CYBERSECURITY LABORATORY REPORT

Persistence Detection Laboratory Report

Finding hidden Windows start-up tricks with Sysmon, Wazuh and Splunk

AUTHOR	Aigbokhaode Hope Imomoh, SOC Analyst Trainee, Abuja, Nigeria

DATE	10 November 2026 (lab performed 9 November 2026)
VERSION	1.0
ENVIRONMENT	Isolated virtual lab (HopeLabs.Local): Windows 10 endpoint, Ubuntu server
SCOPE	Scheduled tasks, Windows services and registry Run keys on one lab host
CLASSIFICATION	Portfolio / training document. Lab-only, harmless test payloads.

Every result in this report is taken from the screenshots, logs and commands recorded during the lab. Unverified items are labelled.

Table of Contents

Table of Contents 2

List of Figures 5

List of Tables 7

About this report (read this first) 8

How the chapters are organised . . . 8

Evidence integrity statement . . . 8

Abbreviations and Acronyms 10

Chapter 1: Executive Summary 11

1.1 What this lab was about . . . 11

1.2 What was done . . . 11

1.3 Main verified results . . . 11

1.4 The most important lessons . . . 11

1.5 Things that were not proven . . . 12

Chapter 2: Introduction and Objectives 13

2.1 Background in plain language . . . 13

2.2 The assignment . . . 13

2.3 Objectives and success criteria . . . 14

Chapter 3: Technical Background 15

3.1 The three persistence methods . . . 15

3.2 Logs and events . . . 15

3.3 Sysmon . . . 15

3.4 From event to dashboard: the full chain . . . 15

3.5 Wazuh rules: parents and children . . . 16

3.6 Alert levels . . . 16

3.7 MITRE ATT&CK; . . . 16

3.8 Splunk versus Wazuh . . . 16

Chapter 4: Lab Environment and Architecture 17

4.1 The machines . . . 17

4.2 How the pieces fit together . . . 17

4.3 Data stores in Splunk . . . 18

Chapter 5: Prerequisites and Safety 19

5.1 What you need . . . 19

5.2 Safety rules followed . . . 19

5.3 How to restore the lab if something goes wrong . . . 19

Chapter 6: Step-by-Step Implementation 20

6.1 Step 1: Check that the logs will reach Wazuh . . . 20
6.2 Step 2: Turn on logging for scheduled-task creation . . . 21
6.3 Step 3: Create the three harmless persistence items . . . 22
6.4 Step 4: Confirm the items exist . . . 25
6.5 Step 5: Read the Windows evidence directly on the computer . . . 26
6.6 Step 6: Find out who launched the commands . . . 29
6.7 Step 7: Write the Wazuh detection rules . . . 30
6.8 Step 8: Trigger the persistence again and look for the alerts . . . 34
6.9 Step 9: Fix the Run key detection . . . 36
6.10 Step 10: Investigate in the Wazuh dashboard . . . 42
6.11 Step 11: Investigate in Splunk . . . 46
6.12 Step 12: Contain and clean up . . . 53

Chapter 7: Configuration and Detection Logic 56

7.1 The final rules . . . 56
7.2 Line-by-line explanation . . . 56
7.3 How the rules work together . . . 57
7.4 Original, changed and recommended configuration . . . 57
7.5 Rule version history . . . 58

Chapter 8: Testing and Results 59

8.1 Test results . . . 59
8.2 What was not tested . . . 59

Chapter 9: Evidence and Screenshot Analysis 60

Chapter 10: Troubleshooting and Root-Cause Analysis 64

10.1 Problems 1, 2 and 4 on Windows . . . 64
10.2 Problem 6: the Run key rule that never fired . . . 65
10.3 Problem 7: logtest tested the wrong thing . . . 65
10.4 Problem 8: invalid XML after a paste . . . 66
10.5 Problems 9 and 10: Splunk zero results . . . 67

Chapter 11: Security Findings and Risk Analysis 69

11.1 Answers to the six analyst questions . . . 69
11.2 Real-SOC response plan (recommended, not performed beyond the lab clean-up) . . . 69
11.3 Findings . . . 69

Chapter 12: Mitigation and Remediation 71

12.1 What was actually implemented . . . 71
12.2 Recommended improvements (not yet implemented) . . . 71

Chapter 13: What I Learned 72

13.1 Concepts . . . 72
13.2 Practical skills . . . 72

13.3 Detection engineering . . . 72
13.4 Troubleshooting and honesty . . . 72
13.5 Areas I still need to develop . . . 72
Chapter 14: Practical Knowledge and Competencies 73
Chapter 15: Reproduction Guide 74
15.1 Starting state . . . 74
15.2 Windows 10: prepare logging (Administrator PowerShell) . . . 74
15.3 Ubuntu: install the rules . . . 74
15.4 Windows 10: create the persistence . . . 74
15.5 Verify the evidence . . . 74
15.6 Investigate . . . 75
15.7 Clean up and roll back . . . 75
15.8 Troubleshooting quick list . . . 75
Chapter 16: Limitations and Future Improvements 76
Chapter 17: Conclusion 77
Appendix A: Final Wazuh Rules 78
Appendix B: Command Reference by Machine 79
B.1 Windows 10 (Administrator PowerShell) . . . 79
B.2 Ubuntu (sudo) . . . 79
B.3 Splunk searches . . . 79
Appendix C: Consolidated Timeline (WAT) 80
Appendix D: Glossary 81
Appendix E: References 82
Appendix F: Evidence Handling and Redaction Log 83

List of Figures

Figure 1. The Day 8 lab brief: objective, six lab tasks, four required deliverables and six analyst questions. (source file 1000497576.jpg) 13

Figure 2. Schematic of the lab (drawn from the documented environment; not a screenshot). Events flow from the Windows endpoint to the Wazuh manager (alerts) and to Splunk (raw searchable events). 17

Figure 3. The agent settings file contains the Sysmon channel (line 50). (source file 1000499351.jpg) 21

Figure 4. The audit policy command succeeded and Security event 4698 (A scheduled task was created) was returned for task \WindowsUpdateCheck by HOPELABS\Administrator at 10:38:42. (source file 1000499231.jpg) 22

Figure 5. First run (about 09:53). The task and the Run key were created successfully; the service command failed because of a PowerShell alias (explained below). (source file 1000499109.jpg) 23

Figure 6. The service was created successfully with sc.exe. (source file 1000499122.jpg) 24

Figure 7. Task details: author, run-as SYSTEM, logon trigger and the command it will run. (source file 1000499144.jpg) 25

Figure 8. Service configuration and Run key listing (also shows the first 4698 query returning nothing). (source file 1000499145.jpg) 26

Figure 9. Service installation event 7045 for UpdSvc (10:11:45), followed by unrelated Sysmon registry noise. (source file 1000499146.jpg) 27

Figure 10. Sysmon event 13 at 09:53:44: reg.exe wrote the Updater Run value (calc.exe) as HOPELABS\Administrator. (source file 1000499236.jpg) 28

Figure 11. Sysmon event 1 (10:38:42): schtasks.exe launched by powershell.exe PID 6324, Administrator, High integrity. (source file 1000499257.jpg) 29

Figure 12. Example of benign noise: sc.exe start pushtoinstall launched by svchost (Windows Store maintenance), not by our PowerShell window. (source file 1000499254.jpg) 30

Figure 13. Parent rules found: 60228 (scheduled task created) and 61138 (new Windows service created). (source file 1000499282.jpg) 31

Figure 14. Parent rule 61615: Sysmon event 13 registry base rule (level 0). (source file 1000499283.jpg) 31

Figure 15. The first version of the persistence rules in nano, using if_sid with the parents 60228, 61138 and 61615. (source file 1000499293.jpg) 32

Figure 16. Configuration test at 11:27: only the old, unrelated 999xx warnings; no errors for the Day 8 rules. (source file 1000499294.jpg) 33

Figure 17. The 11:53 re-creation: sc.exe delete/create, task delete/create, Run key delete/add all succeeded. (source file 1000499329.jpg) 35

Figure 18. Wazuh alert for rule 100621 (level 12, T1543.003), service UpdSvc, 11:50. (source file 1000499311.jpg) 36

Figure 19. Wazuh alert for rule 100620 (level 10, T1053.005), task \WindowsUpdateCheck, 11:53. (source file 1000499330.jpg)

Figure 20. Each Day 8 rule ID appears once in local_rules.xml (one persistence group). (source file 1000499405.jpg) 38

Figure 21. Config test at 13:08 after the clean-up: only the old 999xx warnings. (source file 1000499408.jpg) 39

Figure 22. Manager restarted: 'active (running) since 2026-11-09 13:12:22 WAT'. (source file 1000499413.jpg) 39

Figure 23. The final Run key test on Windows at 13:15: reg.exe and PowerShell writes. (source file 1000499419.jpg) 40

Figure 24. Wazuh dashboard: rule 92302 now level 12 at 13:14:57.184 (T1547.001). (source file 1000499639.jpg) 40

Figure 25. Wazuh dashboard: rule 100622 level 12 at 13:14:57.216, written by powershell.exe. (source file 1000499653.jpg) 41

Figure 26. Terminal confirmation (a photograph of the screen): the 92302 and 100622 alerts at level 12. (source file 20261109_131709288422770388905 6016.jpg) 41

Figure 27. 11 hits for the four rules over the last 24 hours (the bars show when the tests happened). (source file 1000499597.jpg) 42

Figure 28. Alert table: 92302 level 6 alerts (09:53 to 12:41), then 100621 (11:50), 100620 (11:53), and the level 12 alerts at 13:14. (source file 1000499598.jpg) 43

Figure 29. Alert details for the service (rule 100621): service UpdSvc, image path cmd.exe /c whoami, account LocalSystem, start type auto. (source file 1000499605.jpg) 43

Figure 30. Alert details for rule 100620. (source file 1000499627.jpg) 44

Figure 31. Task alert fields: subjectUserName Administrator, subjectLogonId 0x180b06, SID ending -500. (source file 1000499618.jpg) 45

Figure 32. Task alert fields: clientProcessId 6596 and parentProcessId 6324. (source file 1000499617.jpg) 46

Figure 33. Splunk holds Security, Sysmon, System, Application, PowerShell and Defender logs for the host. (source file 1000499658.jpg) 47

Figure 34. Splunk: two 4698 events (11:53:18 and 10:38:42) for WindowsUpdateCheck. (source file 1000499711.jpg) 48

Figure 35. Splunk: two 7045 events for UpdSvc. (source file 1000499763.jpg) 49

Figure 36. Splunk: 12 Sysmon event 13 results for Updater*. (source file 1000499760.jpg) 50

Figure 37. 34 process-create events match the search. (source file 1000500396.jpg) 51

Figure 38. A reg.exe process event: ParentImage powershell.exe, ParentProcessId 6324, LogonId 0x180B06, IntegrityLevel High. (source file 1000500397.jpg) 51

Figure 39. All 28 commands launched by PowerShell 6324, with the same logon ID. (source file 1000500440.jpg) 52

Figure 40. Combined timeline: a create command in Sysmon followed within milliseconds by the matching Windows event. (source file 1000500443.jpg) 53

Figure 41. Clean-up and verification at 13:19: the task is not found, the service does not exist (error 1060), and only OneDrive remains in the Run key. (source file 1000499422.jpg) 54

Figure 42. Splunk also recorded the clean-up and verification commands (13:18 to 13:19), all from PowerShell 6324. (source file 1000500439.jpg) 55

Figure 43. Fix for problem 4: sc.exe delete UpdSvc followed by sc.exe create UpdSvc, both SUCCESS. (source file 1000499310.jpg) 64

Figure 44. Searching for the custom rule IDs returned nothing, while the Updater events were still alerts from rule 92302 at level 6. (source file 1000499380.jpg) 65

Figure 45. The built-in rule 92302 (child of 92300, matches image reg.exe, level 6) and, below, my attempted child rule 100623. (source file 1000499392.jpg) 65

Figure 46. logtest output: the pasted JSON went through generic syslog rules (a photograph of the screen). (source file 20261109_12533378819480738731828 47.jpg) 66

Figure 47. wazuh-analysisd -t at 13:05: ERROR (1226) 'Attribute EOF has no value (line 163)' and CRITICAL (1220). A photograph of the screen. (source file 20261109_13060663825238423391592 23.jpg) 67

Figure 48. Splunk search with lower-case 'or' returned 0 events. (source file 1000499756.jpg) 67

Figure 49. The PowerShell-log search also returned 0 events. Because the query itself was wrong, this does not show that PowerShell logging is off. The log does exist (221 events, Section 4.3). (source file 1000499755.jpg) 68

List of Tables

Table 1. Reading guide 8
Table 2. Abbreviations used in this report 10
Table 3. Summary of verified outcomes 11
Table 4. Objectives and whether they were achieved 14
Table 5. Persistence methods tested 15
Table 6. Event IDs used 15
Table 7. How the two tools were used 16
Table 8. Machines and roles 17
Table 9. Splunk indexes observed (5,180 events for this host in 24 hours) 18
Table 10. What each part of a rule means 56
Table 11. Detection logic summary 57
Table 12. Configuration changes 57
Table 13. How the Run key rules evolved 58
Table 14. Tests, expectations and outcomes 59
Table 15. Evidence register 60
Table 16. Problems encountered 64
Table 17. Analyst questions from the lab brief 69
Table 18. Findings and risk rating 70
Table 19. Implemented measures 71
Table 20. Recommended mitigations 71
Table 21. Skills mapped to the work 73
Table 22. Timeline of the lab (Splunk, Wazuh and Windows evidence) 80
Table 23. Plain-language glossary 81

About this report (read this first)

This report describes one laboratory exercise from a 14-day SOC (Security Operations Centre) training programme. The exercise is called **Day 8: Persistence Detection**. It was carried out on 9 November 2026 in a private, isolated virtual lab.

The report is written so that a person with **no technical background** can follow the story, while a technical reader can still find every exact command, rule and log entry. To make this possible, three habits are used throughout:

- **Plain words first.** Each new idea is explained with an everyday comparison before the technical detail appears.
- **Every command is explained.** For each command you will see where it is typed, what each piece means, what we expected, what really happened, and what can go wrong.
- **Honest results.** Things that worked, things that failed, and things that were never tested are all labelled. Nothing is presented as successful unless a screenshot or log proves it.

> **IN SIMPLE TERMS**
> Think of this lab like a fire-alarm test in a building. We deliberately light a small, safe, harmless 'fire' (fake persistence) and then check that the alarms ring, that we can tell who lit it and how, and that we can put it out and prove the building is clean again.

How the chapters are organised

Table 1. Reading guide

If you want to...	Read
Understand the lab in two minutes	Chapter 1 (Executive Summary)
Learn the basic ideas without jargon	Chapters 2 and 3
See the computers and how they connect	Chapter 4
Follow what was done, command by command	Chapter 6 (the main body of the report)
Read the detection rules and why they work	Chapter 7
See which tests passed, failed or were skipped	Chapter 8
Repeat the lab yourself	Chapter 15 (Reproduction Guide)
Look up a word	Appendix D (Glossary)

Evidence integrity statement

All screenshots in this report are the **original screenshots taken during the lab**. For presentation, copies were only resized to fit the page (never enlarged beyond their original size). No text, number, timestamp, alert, error message or value was edited, redrawn or added. Some screenshots are photographs of a computer screen and appear slightly tilted; they were left as taken so that they remain authentic. The original files are kept unchanged. Each figure caption shows its source file name so that any figure can be checked against the original.

No passwords are visible in any screenshot (password prompts appear empty). Machine names, IP addresses and Windows account identifiers (SIDs) are shown because they belong to a private lab and are essential evidence; no redaction was applied. If this report is shared publicly, these values may be replaced

with placeholders.

Abbreviations and Acronyms

Table 2. Abbreviations used in this report

Short form	Meaning
AD	Active Directory: Microsoft's system for managing users and computers in an organisation
ATT&CK	MITRE ATT&CK: a public catalogue of attacker behaviours
CLI	Command-line interface: typing commands instead of clicking
DQL	Dashboards Query Language: the search language in the Wazuh dashboard
EDR / SIEM	Endpoint Detection and Response / Security Information and Event Management (a central place that collects and analyses security logs)
GPO	Group Policy Object: a central Windows setting pushed to many computers
HKCU / HKLM	Registry sections: HKEY_CURRENT_USER (settings for one user) / HKEY_LOCAL_MACHINE (settings for the whole computer)
IOC	Indicator of Compromise: a clue that something bad happened
PID	Process ID: the number Windows gives a running program
SID	Security Identifier: the unique number behind an account name (the Administrator account ends in -500)
SOC	Security Operations Centre: the team that watches for attacks
SPL	Search Processing Language: Splunk's search language
SYSTEM / LocalSystem	The most powerful built-in Windows account
VM	Virtual machine: a computer simulated inside another computer (VirtualBox here)
WAT	West Africa Time (UTC+1). All times in this report are WAT unless stated

Chapter 1: Executive Summary

1.1 What this lab was about

When an attacker breaks into a computer, one of their first goals is to make sure they can **come back** even if the computer is restarted or the user logs out. Setting something up so that it starts again by itself is called **persistence**. Common hiding places on Windows are **scheduled tasks** (a built-in timer that runs programs), **services** (programs that start with Windows) and **registry Run keys** (a list of programs that start when a user logs in).

The Day 8 assignment asked me to create harmless examples of these three hiding places in an isolated lab, detect them using security tools, work out **who** created them and **how**, match them to the MITRE ATT&CK catalogue, write a detection rule, and then remove them and prove they were gone.

1.2 What was done

- Created one harmless scheduled task, one harmless service and one harmless Run key on a Windows 10 computer using an administrator account.
- Collected detailed logs with **Sysmon** and the normal Windows logs, and sent them to **Wazuh** (alerts) and **Splunk** (searching).
- Wrote four Wazuh detection rules, tested them, fixed problems, and verified that they raised alerts.
- Investigated the events in both the Wazuh dashboard and Splunk, and traced every action back to one PowerShell window.
- Removed the test items and verified the clean-up.

1.3 Main verified results

Table 3. Summary of verified outcomes

Persistence type	What the logs showed	Alert raised	Status
Scheduled task `WindowsUpdateCheck`	Security event 4698, created by HOPELABS\Administrator, parent PowerShell process 6324	Rule 100620, level 10 (MITRE T1053.005)	Verified
Service `UpdSvc`	System event 7045, runs `cmd.exe /c whoami` as LocalSystem	Rule 100621, level 12 (T1543.003)	Verified
Run key `Updater` (written by reg.exe)	Sysmon event 13, reg.exe, user HOPELABS\Administrator	Rule 92302 raised from level 6 to level 12 (T1547.001)	Verified after tuning
Run key `Updater3` (written by PowerShell)	Sysmon event 13, powershell.exe PID 6324	Rule 100622, level 12 (T1547.001)	Verified
Clean-up	Task, service and the three Run key values removed	Verification queries show not found / only OneDrive remains	Verified

1.4 The most important lessons

- **One PowerShell window explains everything.** Splunk showed that every persistence command came from PowerShell process 6324, under the same logon session (0x180B06). That is how an investigator connects separate alerts into one story.

- **A missing setting hides evidence.** The very first task creation left no Security log entry because the 'Other Object Access Events' audit setting was off. After switching it on, the event appeared.
- **Built-in rules can block your own rules.** My first Run-key rule never fired because Wazuh's built-in rule 92302 matched the event first. The fix was to raise that built-in rule's level and rebuild my rule around the right parent.
- **Different tools see different things.** Wazuh gave fast, labelled alerts; Splunk kept the raw events, including failed attempts and the exact command lines.

1.5 Things that were not proven

This lab used one host, one administrator account and harmless commands. It did **not** test other autostart locations (Startup folder, WMI, RunOnce, machine-wide Run keys), real malware, attackers using non-administrator accounts, or how many false alarms the rules would produce in a busy network. The PowerShell command-logging search returned no results, but the search itself was written incorrectly (see Chapter 10), so the real state of PowerShell command logging is **unverified**.

Chapter 2: Introduction and Objectives

2.1 Background in plain language

> **IN SIMPLE TERMS**
> Imagine someone sneaks into your house and wants to be able to get back in later. They might copy your key (a hidden service), set an alarm clock that unlocks a window every morning (a scheduled task), or leave a note on your 'things to do when I get home' list (a Run key). Persistence is the digital version of those tricks. Finding them quickly is a core job for a SOC analyst.

Persistence detection matters because attackers often reuse the same few techniques. If a defender notices the 'alarm clock' (the scheduled task), they can find and stop the intruder before real damage is done.

2.2 The assignment

The lab brief (Figure 1) lists the tasks, the four required deliverables and six analyst questions. I used the brief's simple navy-and-blue heading style as the visual reference for this report, then improved spacing, tables and navigation.

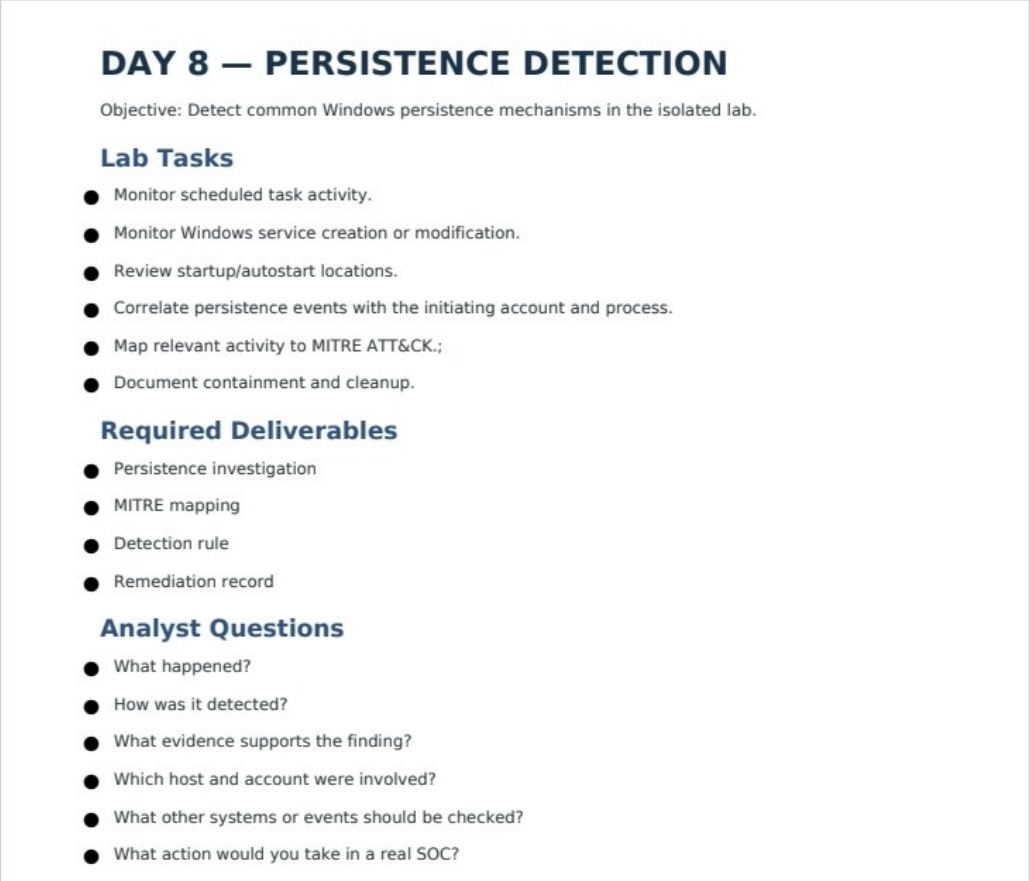


Figure 1. *The Day 8 lab brief: objective, six lab tasks, four required deliverables and six analyst questions. (source file 1000497576.jpg)*

2.3 Objectives and success criteria

Table 4. Objectives and whether they were achieved

Objective (from the brief)	Success criterion	Result
Monitor scheduled task activity	Task creation produces a log and an alert	Achieved: event 4698, rule 100620
Monitor service creation or change	Service install produces a log and an alert	Achieved: event 7045, rule 100621
Review startup / autostart locations	Run key change produces a log and an alert	Partly achieved: only the current-user Run key was tested; Startup folder, RunOnce, machine-wide Run keys and WMI were not
Correlate with account and process	Identify who and which program created each item	Achieved: HOPELABS\Administrator, powershell.exe PID 6324, logon 0x180B06
Map to MITRE ATT&CK	Each item linked to a technique ID	Achieved: T1053.005, T1543.003, T1547.001
Document containment and clean-up	Items removed and removal proven	Achieved on the lab host; verified 13:19 (see Section 6.12)
Deliverables: investigation, MITRE mapping, detection rule, remediation record	Four documented outputs	Achieved: Chapters 6, 7, 11 and 12

Chapter 3: Technical Background

This chapter explains the ideas used in the lab. Each term is described in simple words first, then with the technical meaning.

3.1 The three persistence methods

Table 5. Persistence methods tested

Method	Simple explanation	Technical detail	MITRE ID
Scheduled task	A calendar entry that tells Windows 'run this program at this time or event'	Created with `schtasks`; stored by the Task Scheduler. Ours ran at user logon as SYSTEM	T1053.005
Windows service	A program that Windows starts in the background, even before anyone logs in	Created with `sc.exe create`; recorded in the System log as event 7045	T1543.003
Registry Run key	A 'start these programs when I log in' list	Value under `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`	T1547.001

3.2 Logs and events

Computers write a diary of what happens. Each diary entry is an **event** with a number (the Event ID) that says what kind of thing happened. The events used in this lab are:

Table 6. Event IDs used

Source	Event ID	What it means
Windows Security log	4698	A scheduled task was created (only logged when the 'Other Object Access Events' audit setting is on)
Windows System log	7045	A service was installed
Sysmon	1	A process was started (who started it, with what command, and its parent)
Sysmon	13	A registry value was set (what changed, by which program)

3.3 Sysmon

Sysmon (System Monitor) is a free Microsoft tool that adds a much more detailed diary to Windows. Normal Windows logs say *that* something happened; Sysmon also says *which program* did it, *who started that program* (the 'parent'), and the *exact command typed*.

3.4 From event to dashboard: the full chain

A beginner often thinks 'the alert appeared' is one step. It is really several, and a failure in any one of them means no alert:

1. **Generation:** Windows or Sysmon writes the event to its log.
2. **Collection:** the Wazuh agent on the Windows computer reads that log.
3. **Forwarding:** the agent sends the event across the network to the Wazuh manager.
4. **Decoding:** the manager breaks the raw text into named fields (user, image, targetObject...).

5 **Rule evaluation:** the manager checks the fields against its rules, one by one.

6 **Alert generation:** a matching rule writes an alert with a level (0 to 15) and a MITRE tag.

7 **Display:** the alert is stored and shown in the dashboard; Splunk separately stores the raw events.

3.5 Wazuh rules: parents and children

Wazuh rules form a family tree. A **parent** rule recognises a broad type of event (for example 'a Sysmon registry event'). **Child** rules, written with `<if_sid>PARENT</if_sid>`, only run after the parent has matched and look for something more specific. When several rules could match, **the first one that matches wins**, which becomes important in Chapter 10.

3.6 Alert levels

Each rule has a level from 0 (ignore) to 15 (critical). In this lab: level 4 to 6 means 'worth recording', level 10 means 'important', level 12 means 'high severity, investigate now'.

3.7 MITRE ATT&CK

MITRE ATT&CK is a public encyclopaedia of the techniques attackers use. Each technique has an ID such as T1053.005. Tagging alerts with these IDs lets analysts speak a common language and see which attack stages their detections cover.

3.8 Splunk versus Wazuh

Table 7. How the two tools were used

	Wazuh	Splunk
Main job	Detects: applies rules and raises alerts	Searches: stores raw events so you can ask any question later
Strength	Instant labelled alerts with MITRE tags	Full command lines, failed attempts, process parents
Weakness	Only stores events that triggered a rule	Does not alert by itself unless you build alerts

Chapter 4: Lab Environment and Architecture

4.1 The machines

The lab runs entirely inside VirtualBox on one physical computer, on a private network that is not connected to the internet-facing systems of any real organisation. Two virtual machines were used.

Table 8. Machines and roles

Item	Windows 10 endpoint	Ubuntu server
Name	DESKTOP-T6T1N2V (FQDN DESKTOP-T6T1N2V.HopeLabs.Local)	Ubuntu (Wazuh manager name: Ubuntu)
IP address	192.168.56.109 (Wazuh agent ID 006)	192.168.56.101 (Wazuh dashboard address); Splunk reached locally at 127.0.0.1:8000
Operating system	Windows 10 (build 10.0.19041)	Ubuntu Linux (VirtualBox window title: Ubuntu (Ubuntu_SnapShot))
Roles	The 'victim': runs Sysmon, the Wazuh agent, and the persistence tests	Wazuh manager + dashboard, and Splunk Enterprise 9.4.2
Accounts used	HOPELABS\Administrator (SID ends -500); other accounts seen: HR-STAFF1, svc_backup, aigbo	Local user 'hope' (sudo)
Domain	HOPELABS / HopeLabs.Local	n/a

> **NOTE**
> Exact version numbers for Sysmon and for Wazuh (shown only as 4.x in the index name wazuh-alerts-4.x) were not captured in the evidence, so they are not stated here. Splunk 9.4.2 is visible in the browser tab titles.

4.2 How the pieces fit together

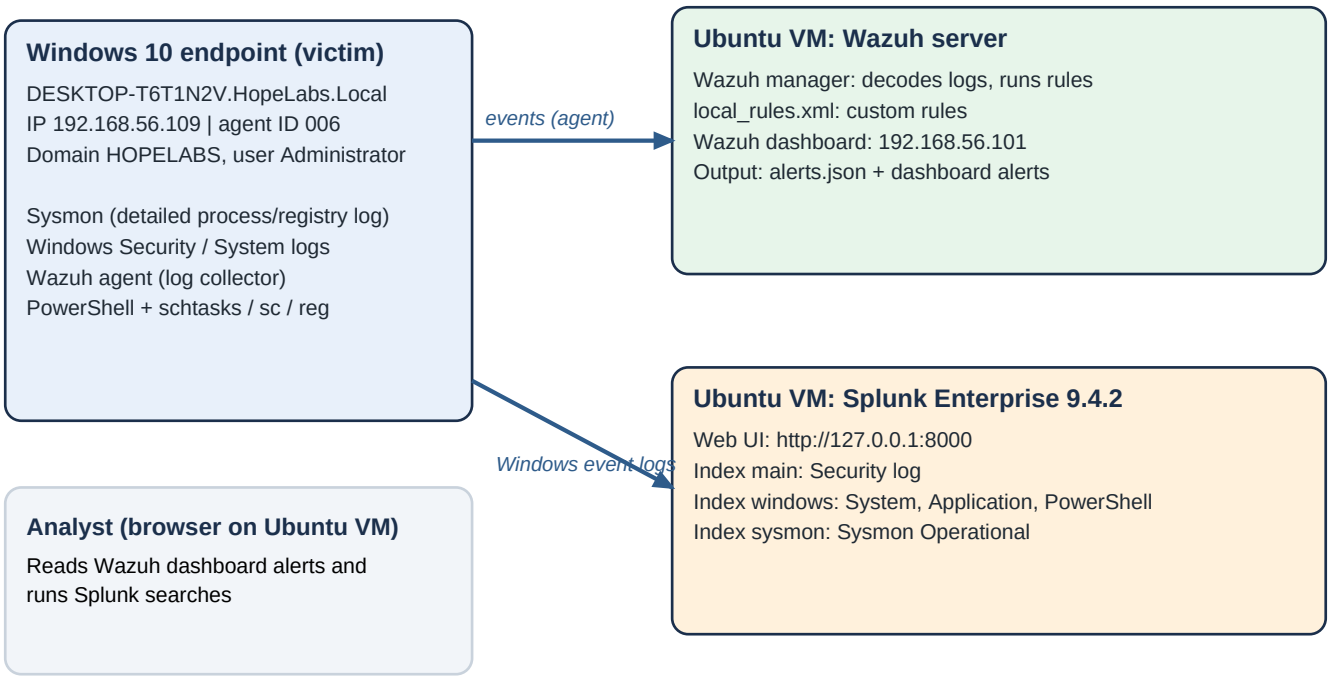


Figure 2. *Schematic of the lab (drawn from the documented environment; not a screenshot). Events flow from the Windows endpoint to the Wazuh manager (alerts) and to Splunk (raw searchable events).*

Reading the diagram from left to right: the Windows computer generates the logs. The Wazuh agent sends them to the Wazuh manager, which applies rules and produces alerts. The same Windows event logs are also stored in Splunk. The analyst sits at the Ubuntu VM and uses a web browser to read the Wazuh dashboard and run Splunk searches.

> **NOTE**
> How Splunk receives the Windows logs (for example a Universal Forwarder) is not shown in the captured evidence. The Splunk sourcetypes are named WinEventLog:Security, WinEventLog:System and WinEventLog:Microsoft-Windows-Sysmon/Operational, which tells us they are Windows event-log inputs.

4.3 Data stores in Splunk

Table 9. Splunk indexes observed (5,180 events for this host in 24 hours)

Index	Sourcetype	Events
main	WinEventLog:Security	1,168
sysmon	WinEventLog:Microsoft-Windows-Sysmon/Operational	3,422
windows	WinEventLog:System	248
windows	WinEventLog:Microsoft-Windows-PowerShell/Operational	221
windows	WinEventLog:Application	77
windows	WinEventLog:Microsoft-Windows-Windows Defender/Operational	44

Chapter 5: Prerequisites and Safety

5.1 What you need

- A Windows 10 virtual machine with **Sysmon** installed and a Wazuh agent that is connected to the manager.
- An Ubuntu virtual machine with the **Wazuh manager and dashboard** and **Splunk**.
- Administrator rights on Windows and `sudo` rights on Ubuntu.
- A way to take screenshots, and a text editor on Ubuntu (this lab used `nano`).

5.2 Safety rules followed

- **Isolated lab only.** Never practise persistence techniques on a real or shared computer.
- **Harmless payloads.** The commands run only `whoami` (prints the current account name) or `calc.exe` (the Calculator). Nothing is stolen, encrypted or downloaded.
- **Obvious names, easy to remove.** Everything created was named WindowsUpdateCheck, UpdSvc or Updater so it could be found and deleted.
- **Backups before edits.** Before replacing the Wazuh rules, the rules file was copied to `local_rules.xml.bak`.
- **Clean-up verified.** Removal was checked with query commands, not just assumed.

> **IN SIMPLE TERMS**
> Why `whoami` and `calc.exe`? Defenders watch the *act of setting up* persistence, not the program it runs. Using harmless programs lets us test the alarm without any danger.

5.3 How to restore the lab if something goes wrong

1. On Ubuntu: restore the rules file with `sudo cp /var/ossec/etc/rules/local_rules.xml.bak /var/ossec/etc/rules/local_rules.xml`, run the config test, then restart the manager.
2. On Windows: run the clean-up commands in Section 6.12 to remove the task, service and Run keys.
3. As a last resort, revert the virtual machine to its VirtualBox snapshot.

Chapter 6: Step-by-Step Implementation

This is the main chapter. Steps are in the order they were performed. Each command appears in a 'command card' that tells you where to type it, what each piece means, what we expected, what actually happened, and what can go wrong. Long commands wrap onto a second line on the page, but each is typed as a single line.

> **NOTE**
> Two machines are used. Commands marked **Windows 10** are typed in an **Administrator PowerShell** window on the endpoint. Commands marked **Ubuntu** are typed in the Terminal on the server. Mixing them up is the most common beginner mistake.

6.1 Step 1: Check that the logs will reach Wazuh

Why: if the Windows computer does not send its Sysmon log to Wazuh, no Sysmon-based alert can ever appear. This is the 'collection' stage from Section 3.4.

Command: Search the Wazuh agent settings for Sysmon

Where it runs: Windows 10, PowerShell **Privileges needed:** Administrator

```
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "Sysmon"
```

Breaking the command into pieces

Part	What it means in plain words
Select-String	Searches a text file for words, like 'Find' in a document
-Path "...\ossec.conf"	The Wazuh agent's settings file
-Pattern "Sysmon"	The word to look for

Why we ran it

To confirm the agent is told to read the Sysmon log.

What we expected

One line showing the Sysmon log location.

What actually happened

One match on line 50: `<location>Microsoft-Windows-Sysmon/Operational</location>`. The agent is configured to collect Sysmon events.

Common problems and how to recover

If nothing is printed, add a `<localfile>` block for the Sysmon channel and restart the agent with `Restart-Service WazuhSvc` (not needed in this lab).

Why this matters for security

A log source that is not collected is a blind spot. Attackers benefit from every blind spot.

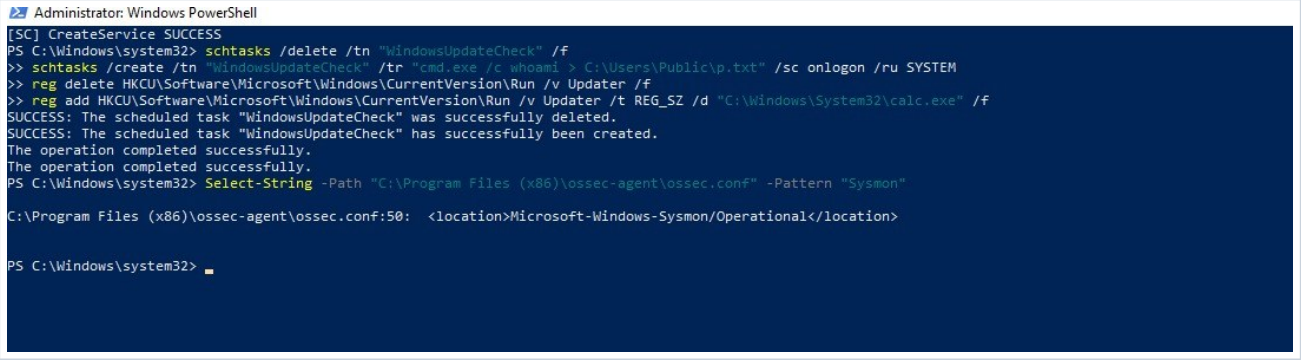


Figure 3. *The agent settings file contains the Sysmon channel (line 50).* *(source file 1000499351.jpg)*

6.2 Step 2: Turn on logging for scheduled-task creation

Windows does not record 'a scheduled task was created' by default. When the first task was created at about 09:53, no Security event 4698 existed (shown in Section 6.5). The setting below fixed that.

Command: Enable the audit policy for task events

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
auditpol /set /subcategory:"Other Object Access Events" /success:enable /failure:enable
```

Breaking the command into pieces

Part	What it means in plain words
auditpol	The Windows audit policy tool
/set	Change a setting
/subcategory:"Other Object Access Events"	The category that includes scheduled-task changes
/success:enable /failure:enable	Log both successful and failed attempts

Why we ran it

Without it, creating a task leaves no Security log entry.

What we expected

The message 'The command was successfully executed.'

What actually happened

It printed that message. The task was then re-created and event 4698 appeared at 10:38:42 (Figure below).

Common problems and how to recover

'Access is denied' means PowerShell was not opened as Administrator. Close it and reopen with 'Run as administrator'.

Why this matters for security

In a real company this is set once for all computers with a Group Policy, otherwise important events are silently missing.

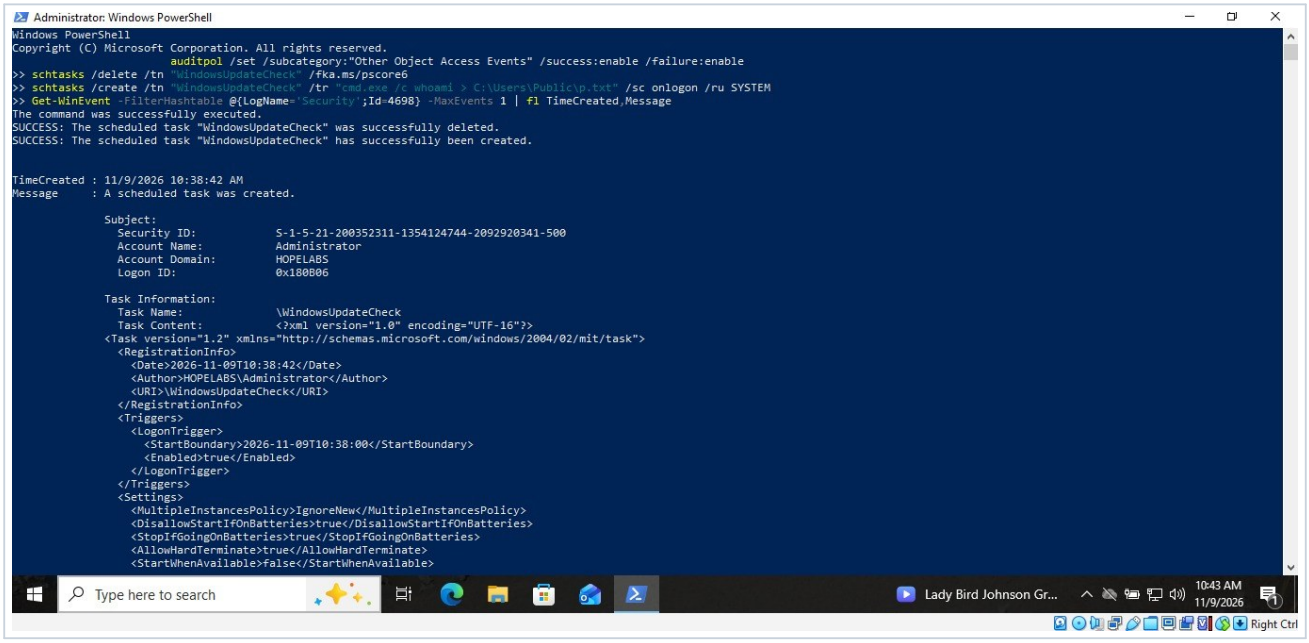


Figure 4. *The audit policy command succeeded and Security event 4698 (A scheduled task was created) was returned for task \WindowsUpdateCheck by HOPELABS\Administrator at 10:38:42.* *(source file 1000499231.jpg)*

6.3 Step 3: Create the three harmless persistence items

All three items were created from one Administrator PowerShell window. The harmless programs are `whoami` and `calc.exe`.

Command: Create a scheduled task that runs at every logon

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
schtasks /create /tn "WindowsUpdateCheck" /tr "cmd.exe /c whoami > C:\Users\Public\p.txt" /sc
    onlogon /ru SYSTEM
```

Breaking the command into pieces

Part	What it means in plain words
schtasks /create	Create a new scheduled task
/tn "WindowsUpdateCheck"	Task Name. A name chosen to look like Windows maintenance, as attackers often do
/tr "cmd.exe /c whoami > C:\Users\Public\p.txt"	Task Run: start the command prompt, run `whoami` and save the answer to a file (`/c` means 'run and then close')
/sc onlogon	Schedule: whenever a user logs on
/ru SYSTEM	Run As User: the powerful SYSTEM account

Why we ran it

To plant the 'alarm clock' persistence method (MITRE T1053.005).

What we expected

The message SUCCESS: The scheduled task has been successfully created.

What actually happened

It printed SUCCESS at about 09:53 (Figure below). Re-running it later shows a warning that the task exists; typing y replaces it.

Common problems and how to recover

If the task already exists you are asked 'replace it (Y/N)?'. Add `/f` to force replacement.

Why this matters for security

A task that runs as SYSTEM at logon survives restarts and gives an attacker top privileges every time.

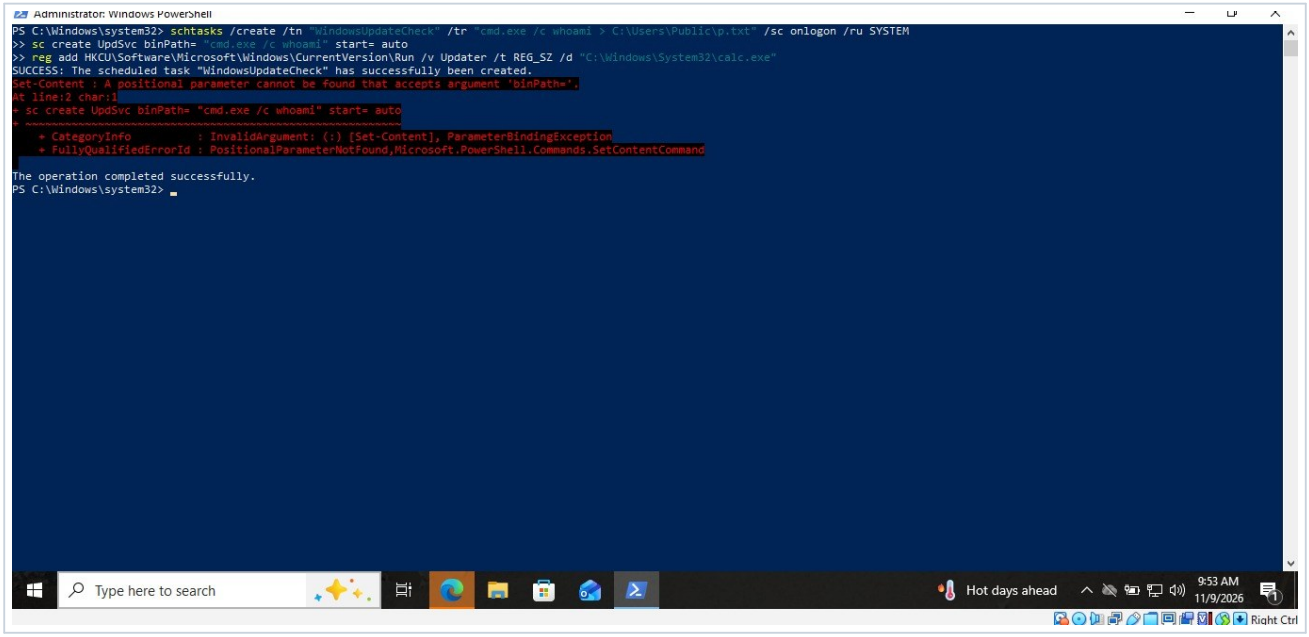


Figure 5. *First run (about 09:53). The task and the Run key were created successfully; the service command failed because of a PowerShell alias (explained below).* *(source file 1000499109.jpg)*

The second command in that screenshot failed. In PowerShell, `sc` is a built-in nickname (an *alias*) for a different command called `Set-Content`. PowerShell therefore tried to write a file instead of creating a service. The fix is to type the full program name `sc.exe`.

Command: Create a Windows service

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
sc.exe create UpdSvc binPath= "cmd.exe /c whoami" start= auto
```

Breaking the command into pieces

Part	What it means in plain words
sc.exe	The Service Control tool. The `.exe` stops PowerShell using its alias
create UpdSvc	Make a new service named UpdSvc
binPath= "cmd.exe /c whoami"	The program the service runs. There must be a space after the equals sign
start= auto	Start automatically every time Windows boots

Why we ran it

To plant the 'service' persistence method (T1543.003).

What we expected

[SC] CreateService SUCCESS.

What actually happened

SUCCESS (Figure below). Windows logged System event 7045 at 10:11:45.

Common problems and how to recover

Error 1073 'The specified service already exists' means a service with that name is already there. Delete it with `sc.exe delete UpdSvc` and create it again. **Missing space after =** gives a usage message.

Why this matters for security

The program `cmd.exe` is not a real service program, so the service will not run properly, but the *installation* is what defenders detect.

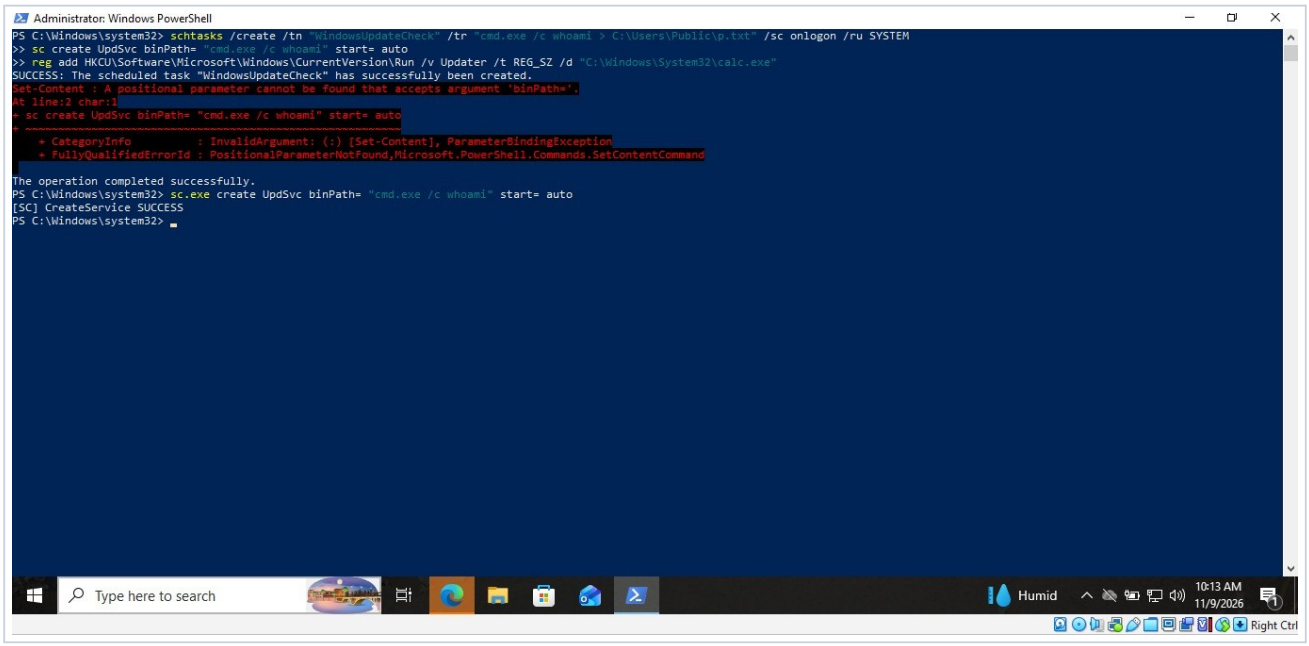


Figure 6. *The service was created successfully with sc.exe.* *(source file 1000499122.jpg)*

Command: Add a Run key value

Where it runs: Windows 10, PowerShell	**Privileges needed:** Current user (Administrator)

```
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d
    "C:\Windows\System32\calc.exe" /f
```

Breaking the command into pieces

Part	What it means in plain words
reg add	Add an entry to the Windows registry (the computer's settings database)
HKCU\...\Run	The 'run at login' list for the current user
/v Updater	Value name: what to call the entry
/t REG_SZ	Type: ordinary text
/d "C:\Windows\System32\calc.exe"	Data: the program to start at login (the Calculator)
/f	Force: overwrite without asking

Why we ran it

To plant the 'Run key' persistence method (T1547.001).

What we expected

'The operation completed successfully.'

What actually happened

That message appeared (first run had no /f; later runs used /f).

Common problems and how to recover

'Access is denied' on HKLM keys needs Administrator. HKCU keys normally do not.

Why this matters for security

Run keys are one of the oldest and most common attacker tricks and are easy to overlook in a long list.

6.4 Step 4: Confirm the items exist

Command: Show the scheduled task in detail

Where it runs: Windows 10, PowerShell **Privileges needed:** Administrator (used here)

```
schtasks /query /tn "WindowsUpdateCheck" /v /fo list
```

Breaking the command into pieces

Part	What it means in plain words
schtasks /query	Look up a task
/tn "WindowsUpdateCheck"	Which task
/v	Verbose: show all details
/fo list	Format as a readable list

Why we ran it

To prove the task is really stored and see what it will run.

What we expected

Task name, author, command, run-as user, schedule.

What actually happened

It showed Author HOPELABS\Administrator, Run As User SYSTEM, Schedule Type 'At logon time', and Task To Run `cmd.exe /c whoami > C:\Users\Public\p.txt`.

Common problems and how to recover

'ERROR: The system cannot find the file specified' means the task does not exist (this is the expected message after clean-up).

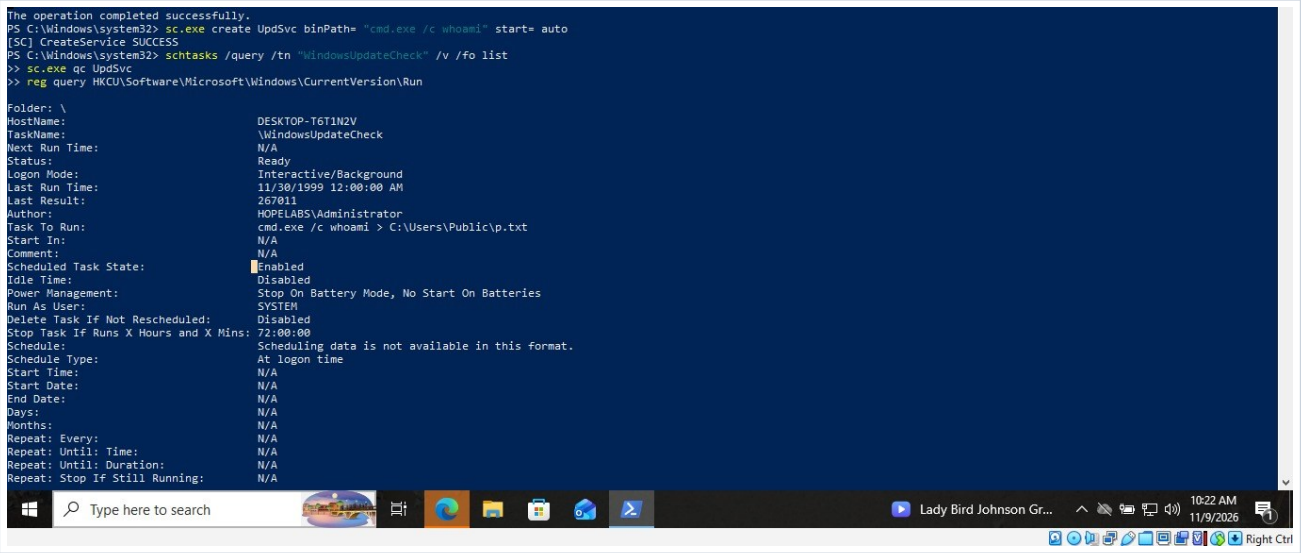


Figure 7. *Task details: author, run-as SYSTEM, logon trigger and the command it will run.* *(source file 1000499144.jpg)*

Command: Show a service's configuration, and the Run key list

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
sc.exe qc UpdSvc
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Breaking the command into pieces

Part	What it means in plain words
sc.exe qc UpdSvc	'Query Config': show how the service is set up
reg query KEY	List what is stored under a registry key

Why we ran it

To confirm the service and Run key values exist.

What we expected

Service details and a list of Run values.

What actually happened

The service showed AUTO_START, BINARY_PATH_NAME `cmd.exe /c whoami`, SERVICE_START_NAME LocalSystem. The Run key listed OneDrive (legitimate) and Updater (ours).

Common problems and how to recover

Error 1060 'The specified service does not exist' means it is not installed.

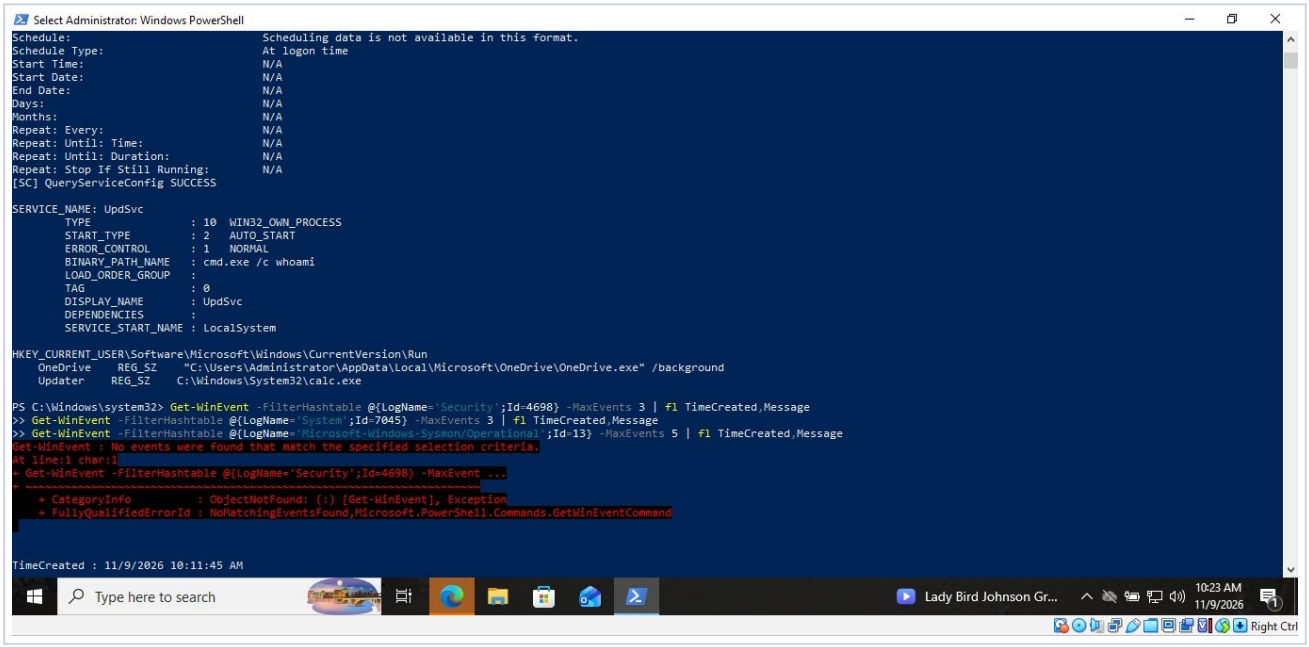


Figure 8. *Service configuration and Run key listing (also shows the first 4698 query returning nothing).* *(source file 1000499145.jpg)*

6.5 Step 5: Read the Windows evidence directly on the computer

Before trusting any SIEM, an analyst should check that the raw event exists on the computer itself.

Command: Read events from a Windows log

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4698} -MaxEvents 3 | fl
    TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='System';Id=7045} -MaxEvents 3 | fl
    TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=13}
    -MaxEvents 5 | fl TimeCreated,Message
```

Breaking the command into pieces

Part	What it means in plain words
Get-WinEvent	Reads entries from a Windows event log
-FilterHashtable @{...}	A filter: which log (LogName) and which event number (Id)
-MaxEvents 3	Stop after the newest 3 matches
|	'Pipe': hand the results to the next command
fl TimeCreated,Message	Format-List: show only the time and the message

Why we ran it

To check events 4698 (task), 7045 (service) and 13 (registry) exist locally.

What we expected

Matching events, or 'No events were found'.

What actually happened

At 10:23 the 4698 query returned **'No events were found'** (audit policy was off, Step 2). The 7045 query returned the UpdSvc install at 10:11:45 (service file name `cmd.exe /c whoami`, LocalSystem). The Sysmon query returned only Microsoft telemetry noise (CompatTelRunner.exe) at that moment.

Common problems and how to recover

'No events were found' can mean a wrong filter, a log that is not enabled, or nothing happened.

Why this matters for security

Comparing the three results showed a real visibility gap, exactly the kind of weakness a SOC must find.

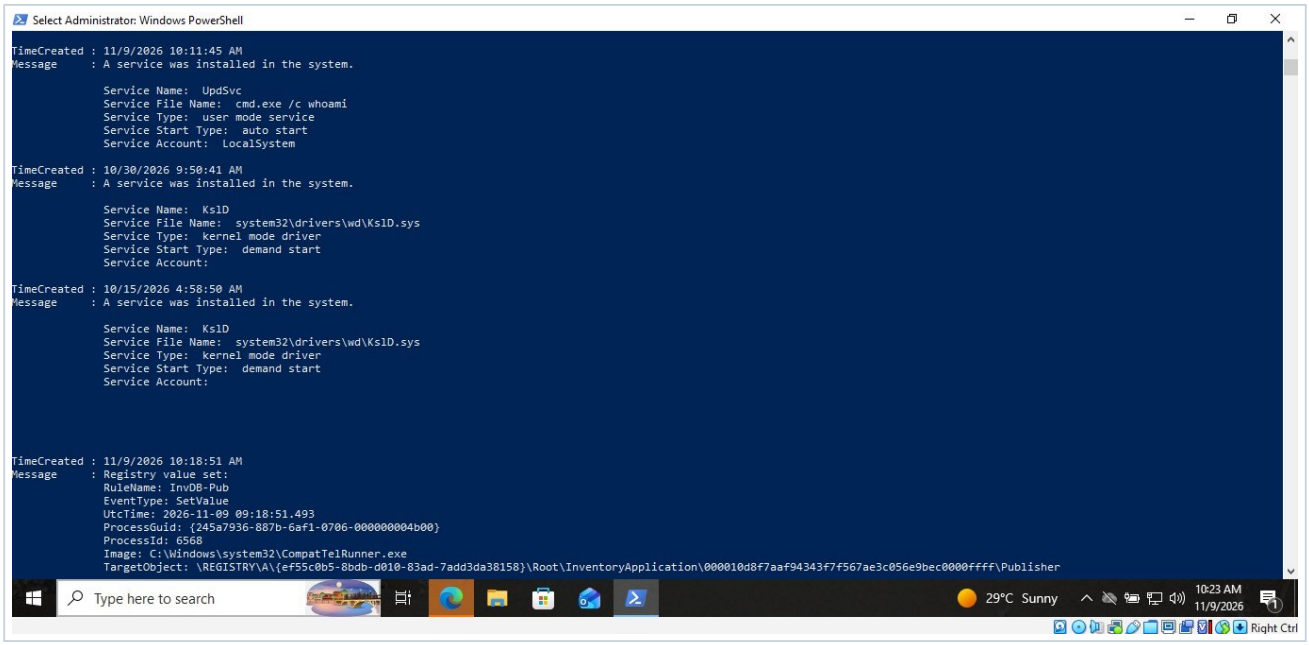


Figure 9. *Service installation event 7045 for UpdSvc (10:11:45), followed by unrelated Sysmon registry noise.* *(source file 1000499146.jpg)*

Command: Filter Sysmon registry events for Run keys only

Where it runs: Windows 10, PowerShell **Privileges needed:** Administrator

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=13} | ?
    {$_.Message -match '\\CurrentVersion\\Run\\'} | fl TimeCreated,Message
```

Breaking the command into pieces

Part	What it means in plain words
Id=13	Sysmon 'registry value set' events
? { ... }	Short for Where-Object: keep only events that satisfy the condition
$_.Message	The text of the current event
-match '\\CurrentVersion\\Run\\'	Keep events whose text contains the path ...\CurrentVersion\Run\ (each backslash is doubled because a backslash is special in patterns)

Why we ran it

The first Sysmon query was full of noise, so this one keeps only Run key changes.

What we expected

Only Run key events.

What actually happened

The 09:53:44 event appeared: `reg.exe` set `...\Run\Updater` to `C:\Windows\System32\calc.exe` as HOPELABS\Administrator. Normal Edge and OneDrive Run entries for other users (HR-STAFF1, svc_backup, aigbo) also appeared and were treated as baseline.

Common problems and how to recover

If everything is returned, check that the backslashes in the pattern are doubled.

Why this matters for security

Knowing what 'normal' looks like (Edge and OneDrive entries) is what lets an analyst spot the one that is not.

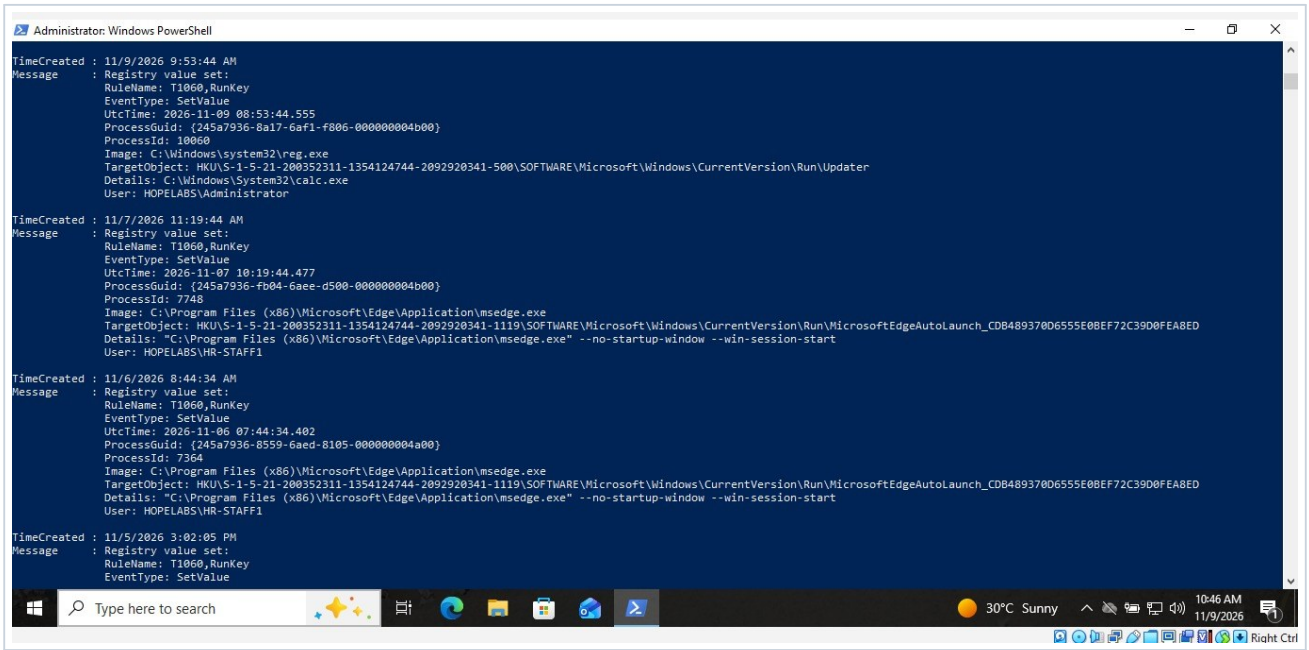


Figure 10. *Sysmon event 13 at 09:53:44: reg.exe wrote the Updater Run value (calc.exe) as HOPELABS\Administrator. (source file 1000499236.jpg)*

6.6 Step 6: Find out who launched the commands

Command: Search Sysmon process-creation events

Where it runs: Windows 10, PowerShell **Privileges needed:** Administrator

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=1} | ?
    {$_.Message -match 'schtasks|sc.exe|reg.exe'} | select -First 6 | fl TimeCreated,Message
```

Breaking the command into pieces

Part	What it means in plain words
Id=1	Sysmon 'a process was started' events
-match 'schtasks|sc.exe|reg.exe'	Keep events mentioning any of these three tools (the bar | means 'or')
select -First 6	Select-Object: keep the first six results

Why we ran it

To learn the parent process (which program started the persistence commands).

What we expected

Process events with a ParentImage field.

What actually happened

The 10:38:42 event showed `schtasks.exe /create /tn WindowsUpdateCheck ...`, user HOPELABS\Administrator, logon ID 0x180B06, integrity High, parent `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` (PID 6324). The same parent also issued `schtasks /delete` one second earlier.

Common problems and how to recover

Not every result is ours. Wazuh's own checks (`reg query ... legalnoticecaption` from the ossec-agent folder) and Windows Store maintenance (`sc.exe start pushtoinstall`, parent svchost) also appear and are normal noise.

Why this matters for security

The 'parent process' is the clue that links an action to a person or script. Here: one PowerShell window.

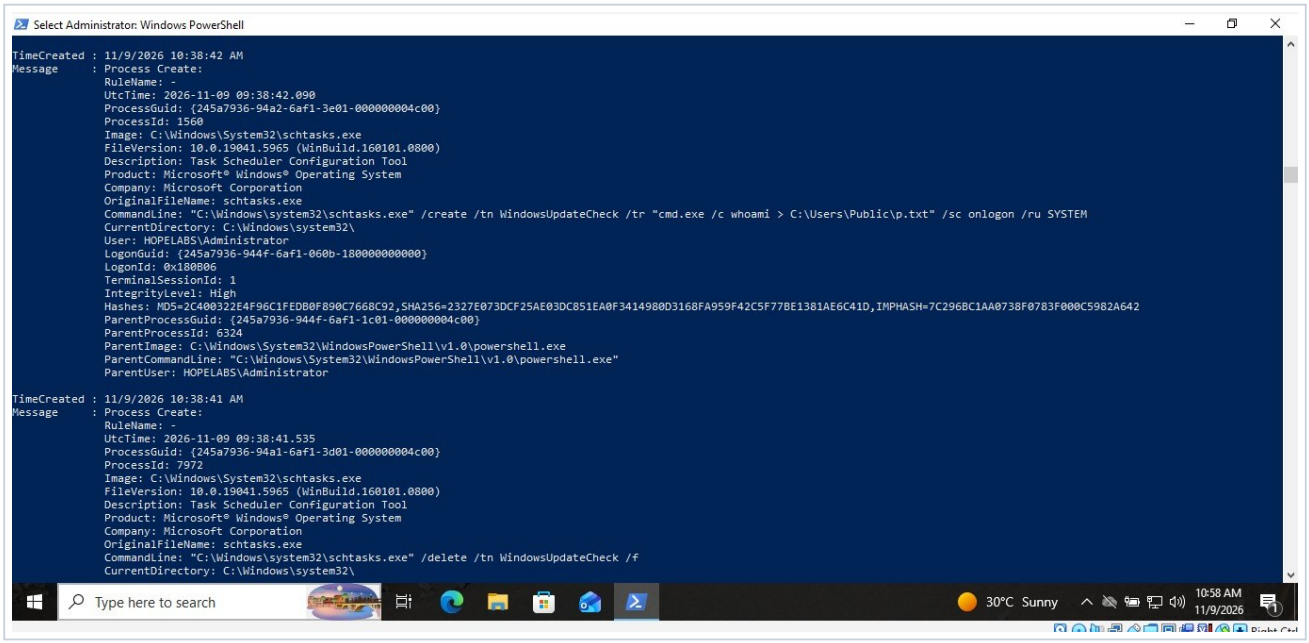


Figure 11. *Sysmon event 1 (10:38:42): schtasks.exe launched by powershell.exe PID 6324, Administrator, High integrity. (source file 1000499257.jpg)*

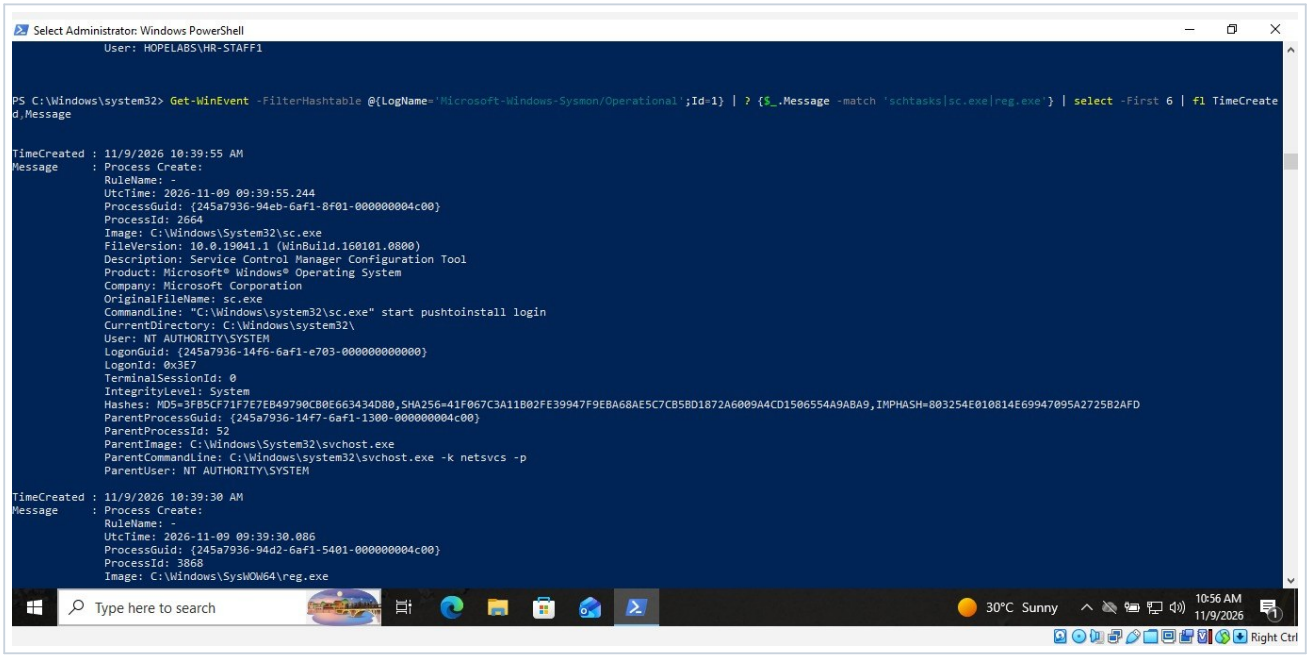


Figure 12. *Example of benign noise: sc.exe start pushtoinstall launched by svchost (Windows Store maintenance), not by our PowerShell window.* *(source file 1000499254.jpg)*

6.7 Step 7: Write the Wazuh detection rules

Now the 'detection rule' deliverable. The goal: when Wazuh sees event 4698, 7045 or a Run key change, raise a clear, labelled alert. A good rule is attached to the correct **parent** rule (Section 3.5), so first I looked up the parents on the Ubuntu server.

Command: Look up the built-in parent rules

Where it runs: Ubuntu terminal	**Privileges needed:** sudo (administrator)

```
sudo sed -n '1322,1332p' /var/ossec/ruleset/rules/0580-win-security_rules.xml
sudo grep -n -B4 -A4 '>\^7045\$<' /var/ossec/ruleset/rules/0590-win-system_rules.xml
sudo grep -n -B4 -A4 '>\^13\$<' /var/ossec/ruleset/rules/0595-win-sysmon_rules.xml
```

Breaking the command into pieces

Part	What it means in plain words
sudo	Run as administrator ('super user do')
sed -n '1322,1332p' FILE	Print only lines 1322 to 1332 of the file
grep -n -B4 -A4 'TEXT' FILE	Search a file for TEXT; show line numbers (-n) and 4 lines Before (-B4) and After (-A4)
'>\^7045\$<'	The pattern for event number 7045 inside the rule file (^ means start, $ means end)

Why we ran it

To find which built-in rule IDs recognise event 4698, 7045 and Sysmon 13.

What we expected

Rule blocks with `id="..."`.

What actually happened

Event 4698 belongs to rule **60228** ('A scheduled task was created', level 4). Event 7045 belongs to rule **61138** ('New Windows Service Created', level 5). Sysmon event 13 belongs to rule **61615** (a level 0 base rule that groups all registry events).

Common problems and how to recover

If grep prints nothing, the pattern or the file name is wrong. List the folder with `ls /var/ossec/ruleset/rules/`.

Why this matters for security

Chaining on the right parent makes a rule faster and more precise.

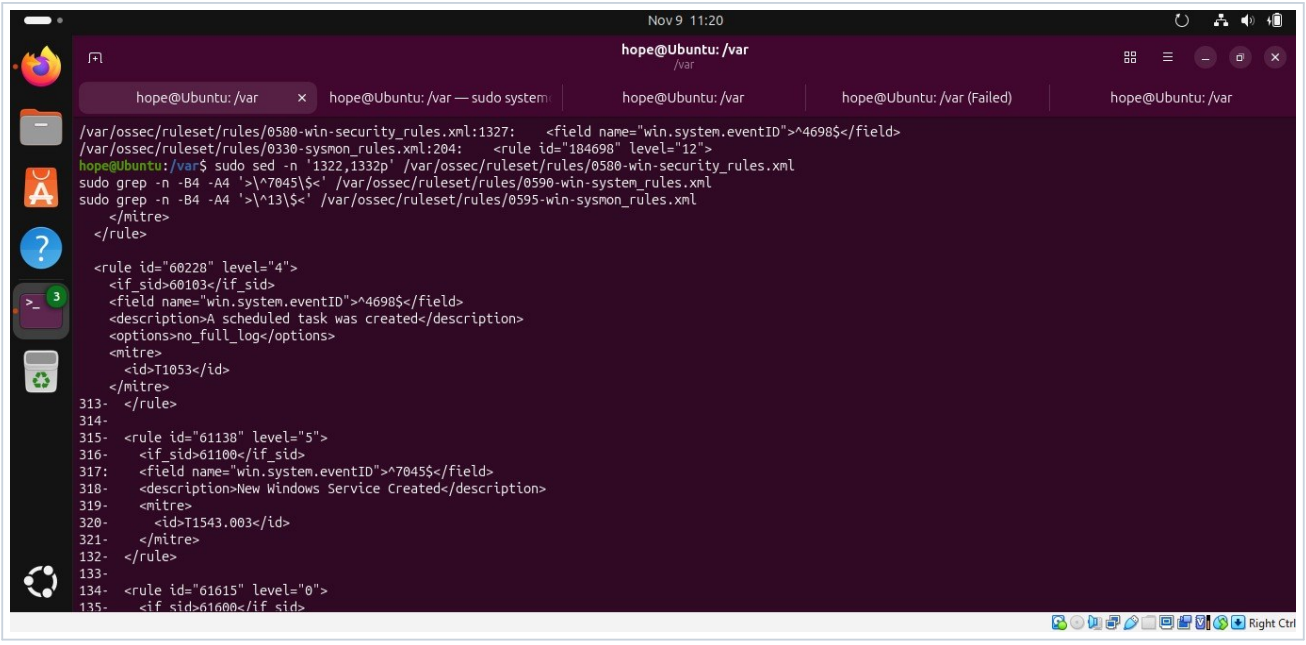


Figure 13. *Parent rules found: 60228 (scheduled task created) and 61138 (new Windows service created).* *(source file 1000499282.jpg)*

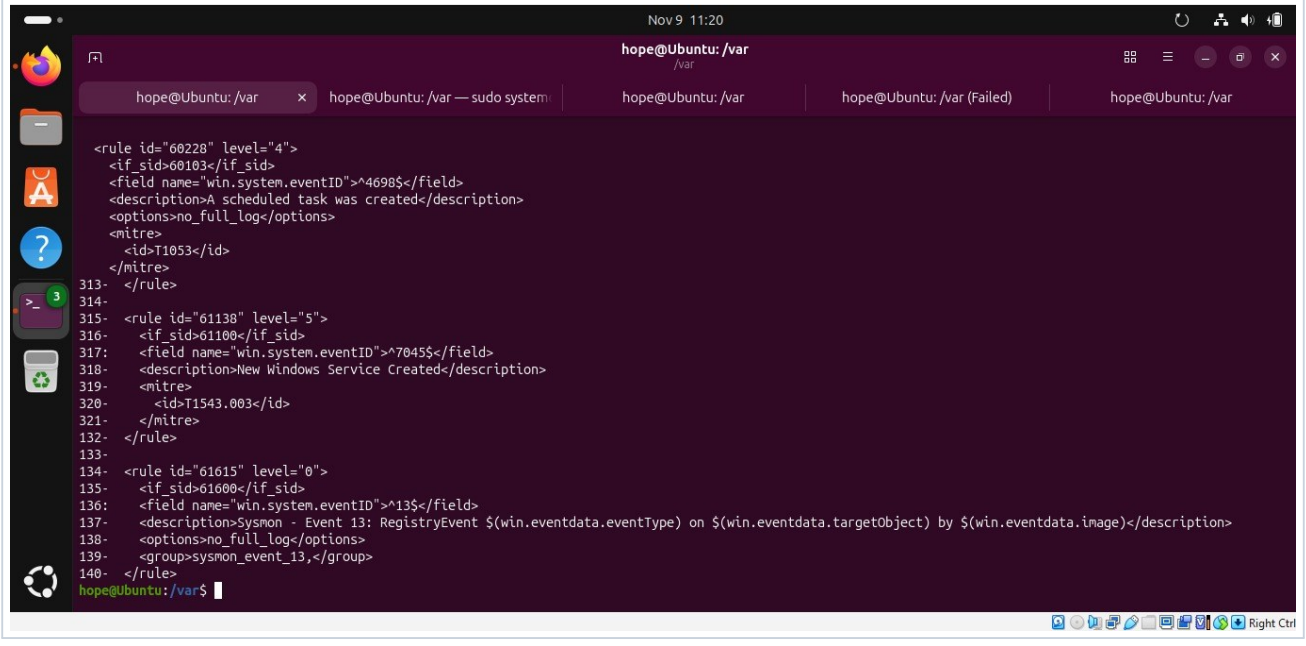


Figure 14. *Parent rule 61615: Sysmon event 13 registry base rule (level 0).* *(source file 1000499283.jpg)*

Command: Edit the custom rules file

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

Breaking the command into pieces

Part	What it means in plain words
nano	A simple text editor in the terminal
/var/ossec/etc/rules/local_rules.xml	The file where your own Wazuh rules live (built-in rules are never edited)

Why we ran it

To add the persistence rules.

What we expected

The file opens. Save with Ctrl+O then Enter; exit with Ctrl+X.

What actually happened

The first version used `<if_group>windows</if_group>`. It was rewritten to use the exact parents found above (Figure below).

Common problems and how to recover

Pasting a command (such as the 'sudo tee' command in Section 6.9) into nano inserts it as text into the file and breaks it (see Chapter 10, Problem 8).

Why this matters for security

Custom rules are code. A typing mistake can stop the whole Wazuh engine from loading.

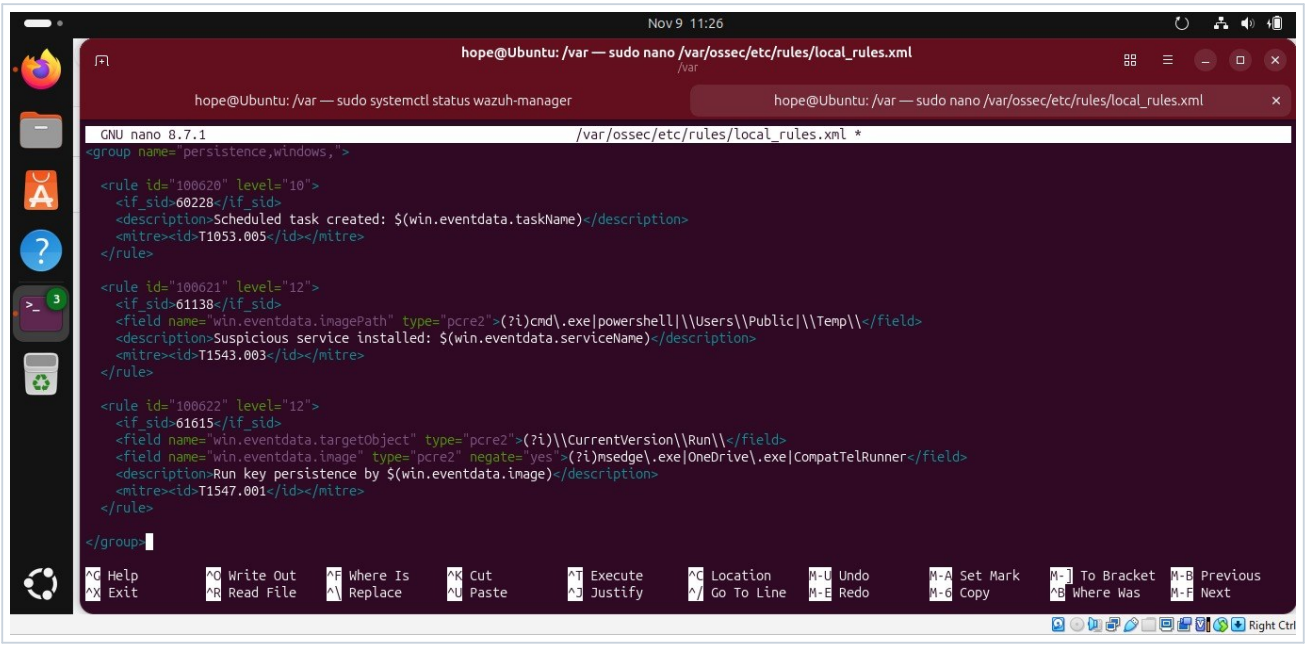


Figure 15. *The first version of the persistence rules in nano, using if_sid with the parents 60228, 61138 and 61615. (source file 1000499293.jpg)*

Command: Test the rules file for errors

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

Breaking the command into pieces

Part	What it means in plain words
wazuh-analysisd	The Wazuh analysis engine that evaluates rules
-t	Test mode: check the configuration and rules, but do not run

Why we ran it

To catch mistakes before restarting a live system.

What we expected

Only warnings, no ERROR or CRITICAL lines.

What actually happened

At 11:27 the test printed only warnings about the lists `malicious-ioc/...` and rules 99901 to 99920 being ignored. These belong to earlier threat-intelligence work (the list files were missing), not to Day 8. There were no errors for rules 100620 to 100622.

Common problems and how to recover

ERROR (1226) / CRITICAL (1220) mean the XML is invalid. Do not restart; fix the file first.

Why this matters for security

Testing before restarting prevents taking down your own monitoring.

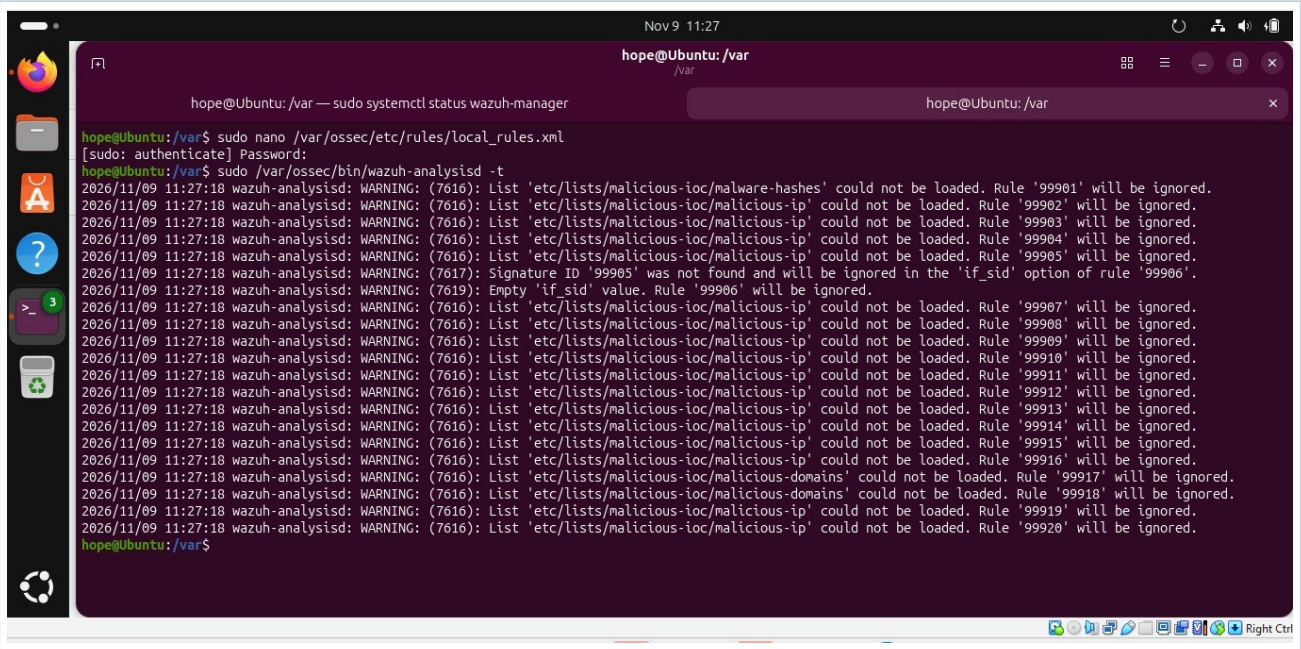


Figure 16. *Configuration test at 11:27: only the old, unrelated 999xx warnings; no errors for the Day 8 rules.* *(source file 1000499294.jpg)*

Command: Restart the Wazuh manager and check it is running

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager | grep Active
```

Breaking the command into pieces

Part	What it means in plain words
systemctl restart wazuh-manager	Stop and start the Wazuh manager so it loads the new rules
systemctl status ...	Show the service state
| grep Active	Keep only the line containing 'Active'

Why we ran it

Wazuh only reads rules when it starts.

What we expected

'Active: active (running)'.

What actually happened

Confirmed running (for example 'active (running) since 2026-11-09 13:12:22' for the final rule set).

Common problems and how to recover

'failed' means a rules error: run the test command and read the last lines.

6.8 Step 8: Trigger the persistence again and look for the alerts

Rules are only believed when a real event makes them fire. The persistence items were deleted and re-created so that fresh events were generated after the rules were loaded.

Command: Delete and re-create the items

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
sc.exe delete UpdSvc
sc.exe create UpdSvc binPath= "cmd.exe /c whoami" start= auto
schtasks /delete /tn "WindowsUpdateCheck" /f
schtasks /create /tn "WindowsUpdateCheck" /tr "cmd.exe /c whoami > C:\Users\Public\p.txt" /sc
    onlogon /ru SYSTEM
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /f
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d
    "C:\Windows\System32\calc.exe" /f
```

Breaking the command into pieces

Part	What it means in plain words
sc.exe delete UpdSvc	Remove the service so it can be created fresh
schtasks /delete ... /f	Remove the task (/f = do not ask)
reg delete ... /v Updater /f	Remove the Run value
the create / add commands	Same as in Step 3; they now generate new events

Why we ran it

Windows only logs a 'created' event when something is genuinely created. Running create on something that already exists fails and logs nothing useful (error 1073).

What we expected

Three SUCCESS messages and three fresh events.

What actually happened

The service was recreated at 11:50 and the task at 11:53 (Figures below). An earlier attempt had printed **'FAILED 1073: The specified service already exists'** because the old service was still there.

Common problems and how to recover

'Marked for deletion' means a window (for example Services or Event Viewer) still has the service open. Close it and retry.

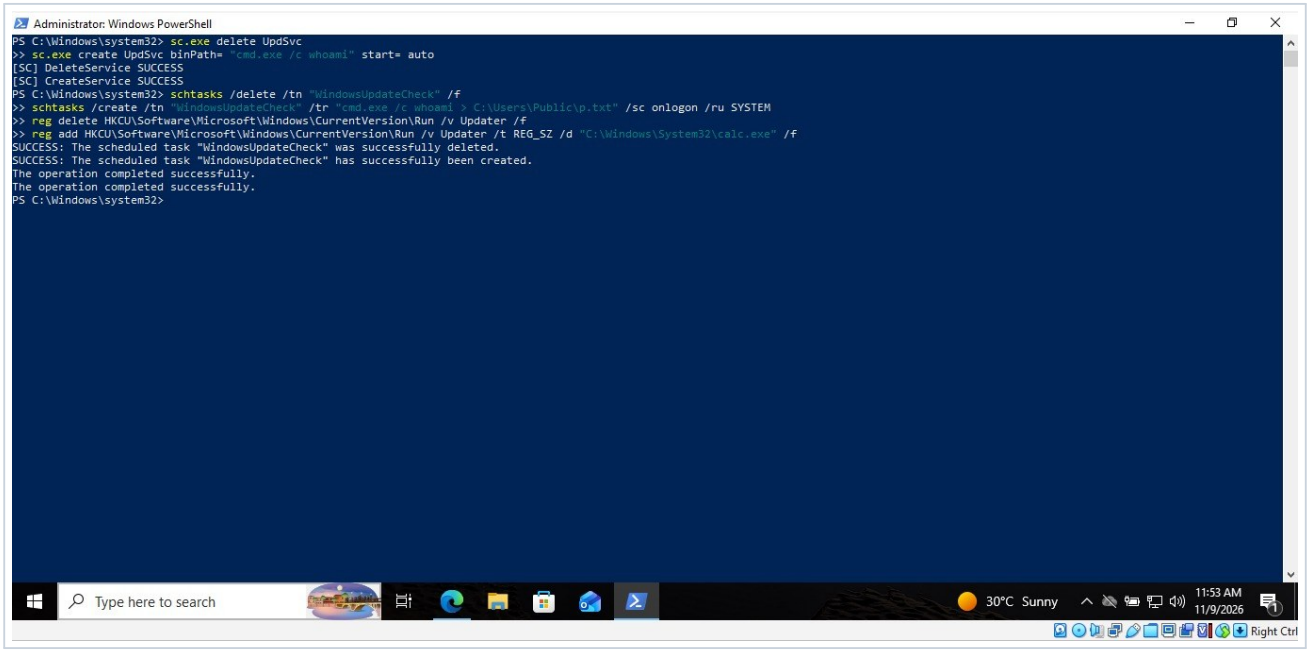


Figure 17. *The 11:53 re-creation: sc.exe delete/create, task delete/create, Run key delete/add all succeeded. (source file 1000499329.jpg)*

Command: Search the Wazuh alert file for the custom rules

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo grep -E '"id":"10062[0-2]"' /var/ossec/logs/alerts/alerts.json | tail -5
```

Breaking the command into pieces

Part	What it means in plain words
grep -E 'PATTERN' FILE	Print the lines of the file that match an extended pattern
"id":"10062[0-2]"	Rule IDs 100620, 100621 or 100622 ([0-2] means one of 0, 1 or 2)
/var/ossec/logs/alerts/alerts.json	Wazuh's raw alert log, one alert per line
| tail -5	Keep only the last 5 matches

Why we ran it

A quick way to see whether the rules fired, before opening the dashboard.

What we expected

One long JSON line per alert.

What actually happened

Rule 100621 fired at 11:50:06 (level 12): 'Suspicious service installed: UpdSvc', agent DESKTOP-T6T1N2V, MITRE T1543.003. **Rule 100620 fired at 11:53:16** (level 10): 'Scheduled task created: \WindowsUpdateCheck', T1053.005.

Common problems and how to recover

An empty result means the rule did not match, the agent is not sending, or you ran the Windows commands before the rules loaded.

Why this matters for security

Alerts with the MITRE tag show an analyst immediately what kind of attacker behaviour this is.

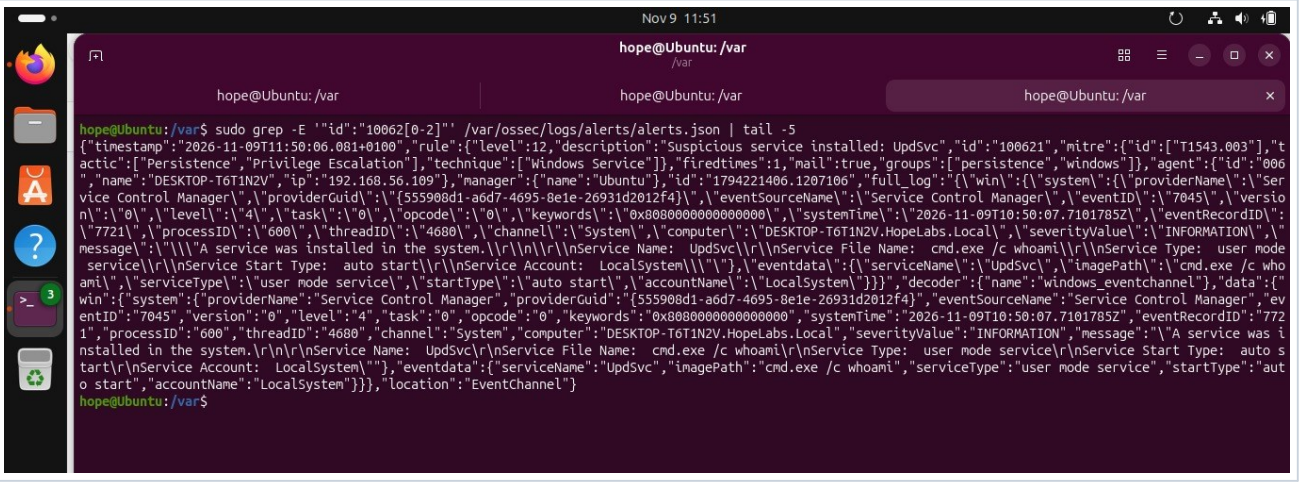


Figure 18. *Wazuh alert for rule 100621 (level 12, T1543.003), service UpdSvc, 11:50.* *(source file 1000499311.jpg)*

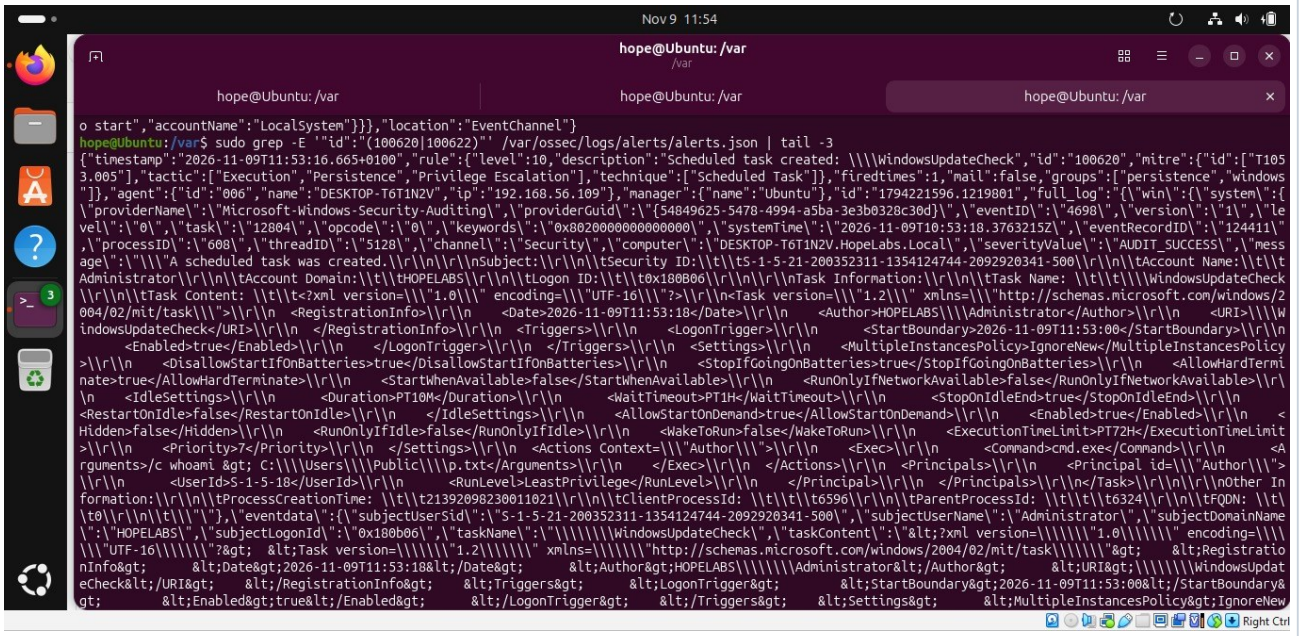


Figure 19. *Wazuh alert for rule 100620 (level 10, T1053.005), task \WindowsUpdateCheck, 11:53.* *(source file 1000499330.jpg)*

> **NOTE**
> Result so far: two of the three Run-key, task and service detections worked. The Run key rule did not fire. Section 6.9 and Chapter 10 explain why and how it was fixed.

6.9 Step 9: Fix the Run key detection

The Run key alerts kept coming from Wazuh's built-in rule **92302** at a low level (6), never from my rule. Only the first rule that matches an event produces the alert, and the built-in rule matched first. Two changes fixed it:

- **Raise the built-in rule's level** by overwriting 92302 in `local_rules.xml` (so Run key writes by reg.exe now alert at level 12).
- **Rebuild rule 100622** on the correct parent (92300) to catch Run key writes by *any other* program, such as PowerShell.

The final rule file was then installed with these commands.

Command: Back up the rules file and delete the old persistence group

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak
sudo sed -i '/<group name="persistence,windows,">/,/<\/group>/d'
    /var/ossec/etc/rules/local_rules.xml
```

Breaking the command into pieces

Part	What it means in plain words
cp A B	Copy file A to file B (a backup)
sed -i	Edit the file in place (no new file)
'/START/,/END/d'	Delete every line from the line matching START to the line matching END
<group name="persistence,windows,"> ... </group>	The block of Day 8 rules, which is replaced wholesale

Why we ran it

To replace the old group cleanly without leaving duplicate rule IDs, which would make Wazuh refuse to start.

What we expected

No output.

What actually happened

The group was removed; other custom rules (100600, 100610) stayed. The check in Figure below shows each ID appears once.

Common problems and how to recover

If you delete too much, restore the backup with the cp command reversed.

Command: Append the final rules (heredoc)

Where it runs: Ubuntu terminal prompt (not inside nano)	**Privileges needed:** sudo

```
sudo tee -a /var/ossec/etc/rules/local_rules.xml > /dev/null << 'EOF'
<group name="persistence,windows,"> ... the four rules shown in Chapter 7 ... </group>
EOF
```

Breaking the command into pieces

Part	What it means in plain words
sudo tee -a FILE	Add (append, -a) the following text to the end of FILE, with administrator rights
> /dev/null	Hide the copy that would otherwise print on screen
<< 'EOF' ... EOF	Everything between these two markers is the text to add

Why we ran it

To add several lines to a file in one paste.

What we expected

Silent success.

What actually happened

The first paste was made inside nano by mistake, which put the words `sudo tee` and `EOF` into the XML file and the next test printed **XMLERR: Attribute 'EOF' has no value (line 163)** (Chapter 10, Problem 8). They were removed with the sed command below.

Common problems and how to recover

Always paste multi-line commands at the `$` prompt, never inside nano.

Command: Remove the accidental text and re-test

Where it runs: Ubuntu terminal	**Privileges needed:** sudo

```
sudo sed -i '/sudo tee -a/d;/^EOF$/d' /var/ossec/etc/rules/local_rules.xml
sudo grep -n 'id="1006[0-9][0-9]"\|id="92302"\|persistence'
    /var/ossec/etc/rules/local_rules.xml
sudo /var/ossec/bin/wazuh-analysisd -t
```

Breaking the command into pieces

Part	What it means in plain words
sed -i '/X/d;/Y/d'	Delete lines matching X, then lines matching Y
^EOF$	A line that contains only the word EOF
grep -n 'A|B|C'	Show lines containing A or B or C, with line numbers

Why we ran it

To clean the file and prove each rule ID exists exactly once.

What we expected

The grep lists 100600, 100610, the group, 100620, 100621, 92302, 100622 once each; the test has no errors.

What actually happened

Confirmed at 13:08 to 13:10 (Figures below), then the manager was restarted at 13:12:22.



Figure 20. *Each Day 8 rule ID appears once in local_rules.xml (one persistence group).* *(source file 1000499405.jpg)*

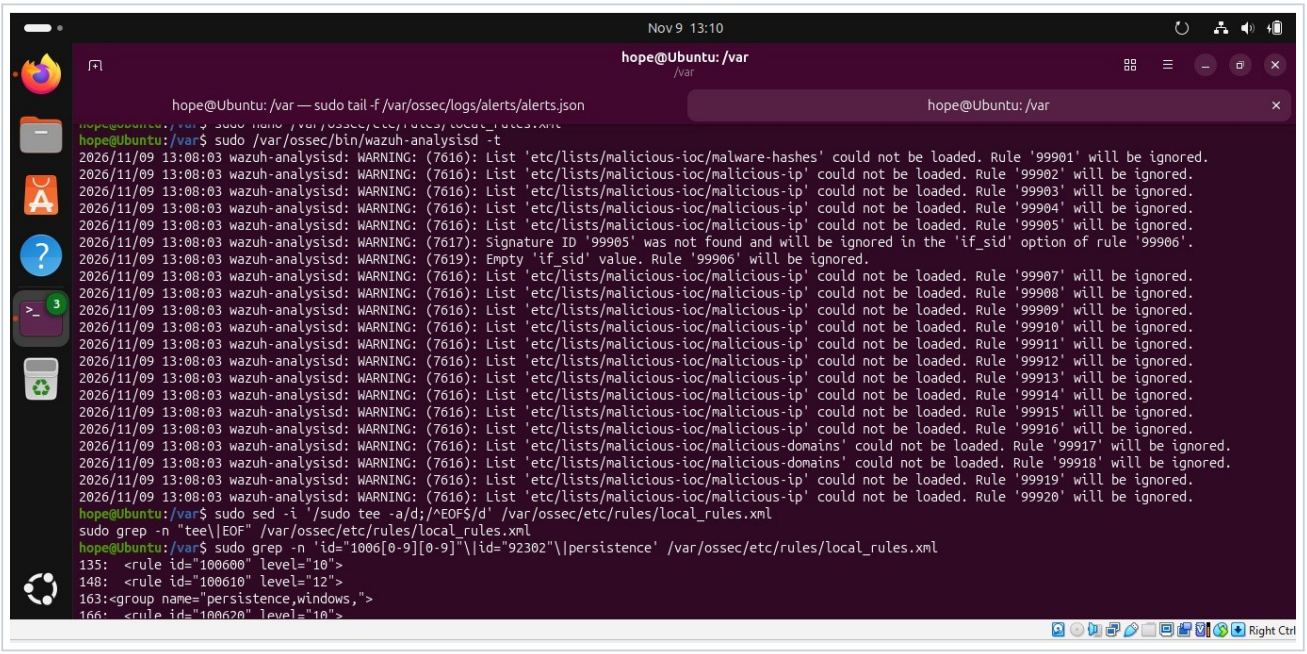


Figure 21. *Config test at 13:08 after the clean-up: only the old 999xx warnings.* *(source file 1000499408.jpg)*

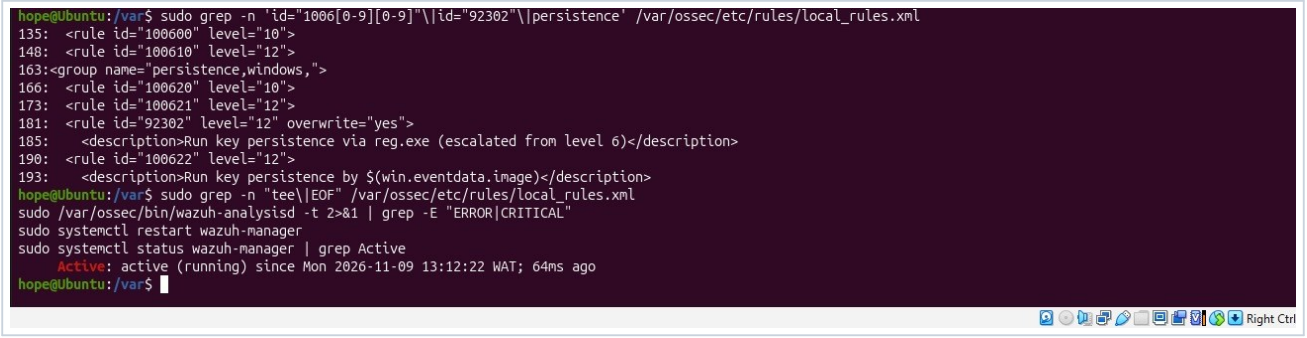


Figure 22. *Manager restarted: 'active (running) since 2026-11-09 13:12:22 WAT'.* *(source file 1000499413.jpg)*

Command: Final trigger: one write by reg.exe and one by PowerShell

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /f
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d
    "C:\Windows\System32\calc.exe" /f
New-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "Updater3"
    -Value "C:\Windows\System32\calc.exe" -Force
```

Breaking the command into pieces

Part	What it means in plain words
reg delete / reg add	Remove and re-add `Updater` using the **reg.exe** program
New-ItemProperty	A PowerShell command that writes a registry value directly (the writer is **powershell.exe**, not reg.exe)
-Path "HKCU:\...\Run"	The Run key, written PowerShell-style with a colon after HKCU
-Name "Updater3" -Value "...calc.exe"	Entry name and the program it launches
-Force	Create it even if it already exists

Why we ran it

To test both detection paths: built-in-style (reg.exe) and any-other-program (PowerShell).

What we expected

Rule 92302 at level 12 for reg.exe; rule 100622 at level 12 for PowerShell.

What actually happened

Both fired at **13:14:57**: rule 92302 ('Run key persistence via reg.exe (escalated from level 6)', level 12) and rule 100622 ('Run key persistence by ...powershell.exe', level 12).

Common problems and how to recover

If a different rule ID appears, another built-in rule is matching first. Use the alert trail to find it.

Why this matters for security

This is the difference between detecting one tool and detecting the behaviour.

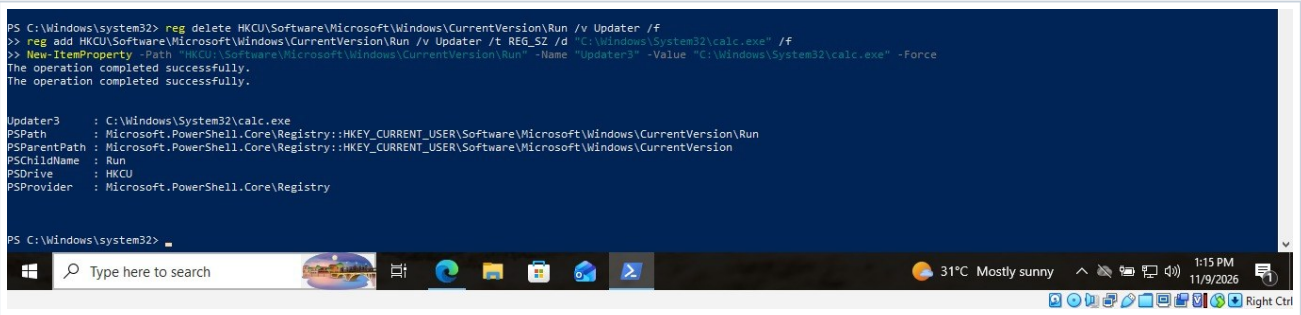


Figure 23. *The final Run key test on Windows at 13:15: reg.exe and PowerShell writes.* *(source file 1000499419.jpg)*

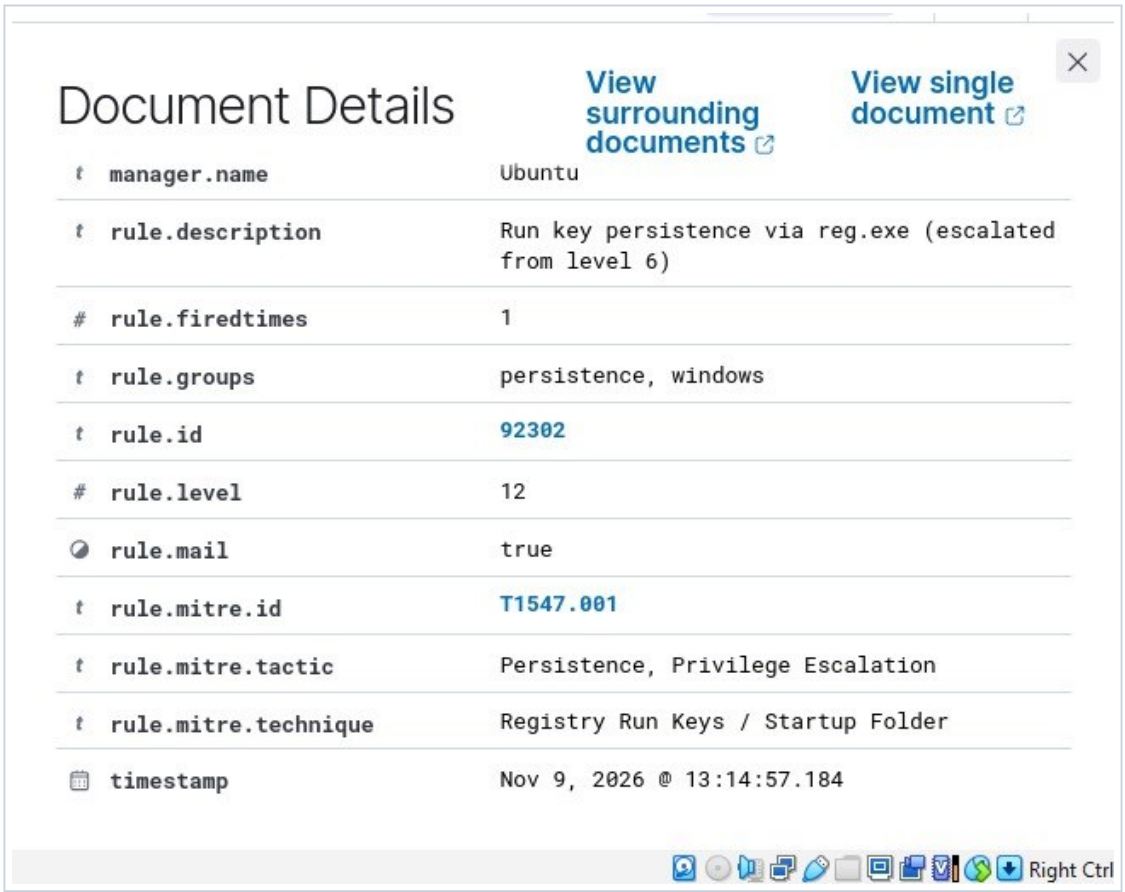


Figure 24. *Wazuh dashboard: rule 92302 now level 12 at 13:14:57.184 (T1547.001).* *(source file 1000499639.jpg)*

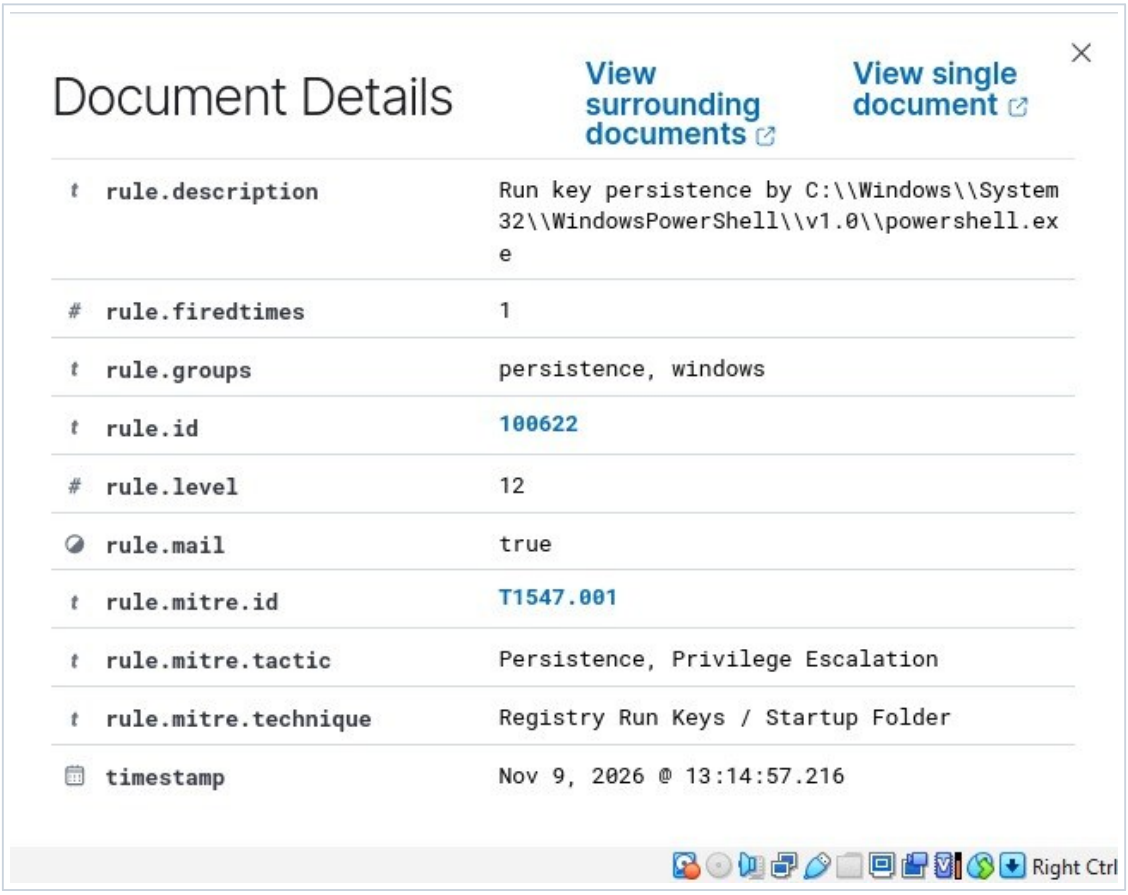


Figure 25. *Wazuh dashboard: rule 100622 level 12 at 13:14:57.216, written by powershell.exe.* *(source file 1000499653.jpg)*

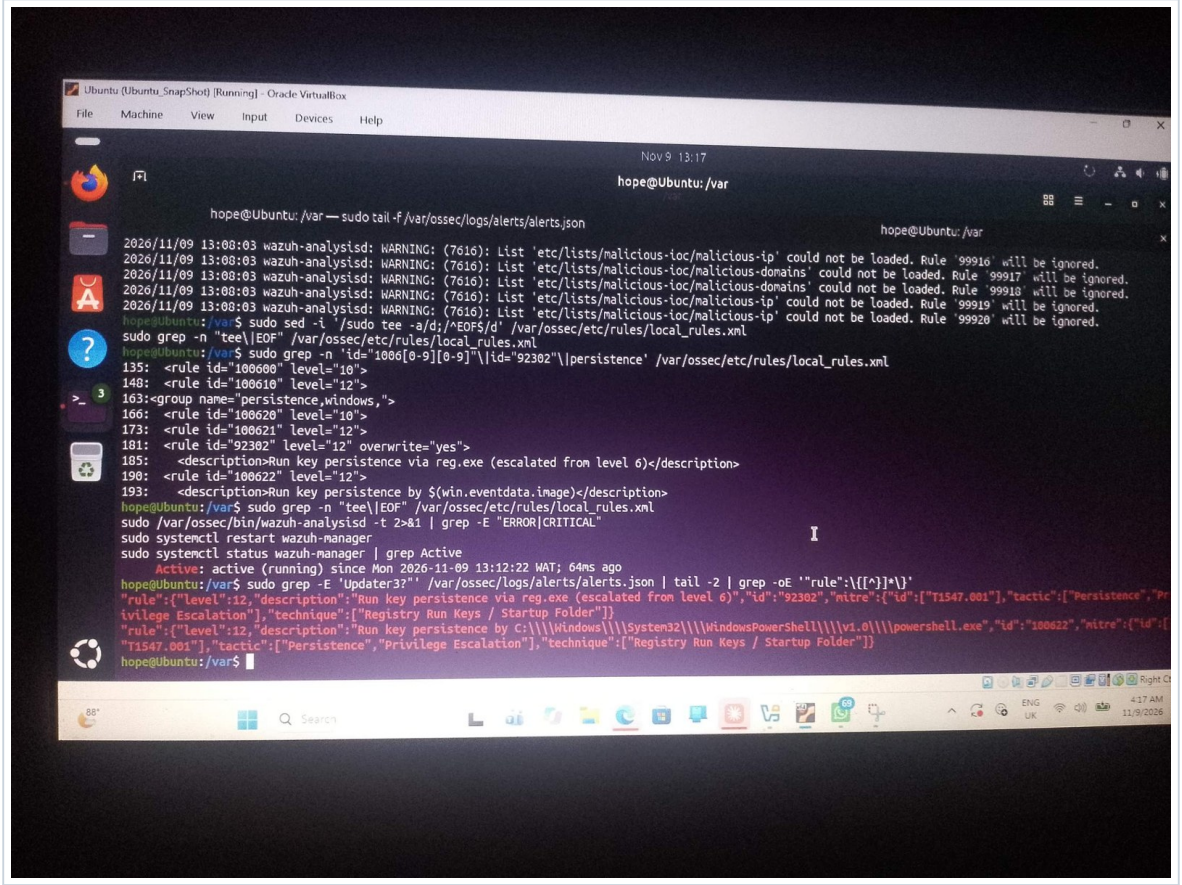


Figure 26. *Terminal confirmation (a photograph of the screen): the 92302 and 100622 alerts at level 12.* *(source file 20261109_131709288422770388905601 6.jpg)*

6.10 Step 10: Investigate in the Wazuh dashboard

An investigation answers the six analyst questions. In the Wazuh dashboard I opened Threat Hunting for the agent `DESKTOP-T6T1N2V (006)` and searched for the four rules.

Command: Filter the alerts in the Wazuh dashboard

Where it runs: Browser: Wazuh dashboard, Threat Hunting > Events

Privileges needed: Dashboard login

```
rule.id:(100620 or 100621 or 92302 or 100622)
```

Breaking the command into pieces

Part	What it means in plain words
rule.id:(...)	Show alerts whose rule ID is any of the listed IDs
or	Either/or between IDs (Wazuh's query language accepts lower case)
Time range: Last 24 hours	Includes the morning tests and the final run

Why we ran it

To list every alert about persistence in one table.

What we expected

A table with the four rule IDs.

What actually happened

11 hits (Figures below).

Common problems and how to recover

Zero hits: widen the time range or check the agent filter (agent.id: 006).

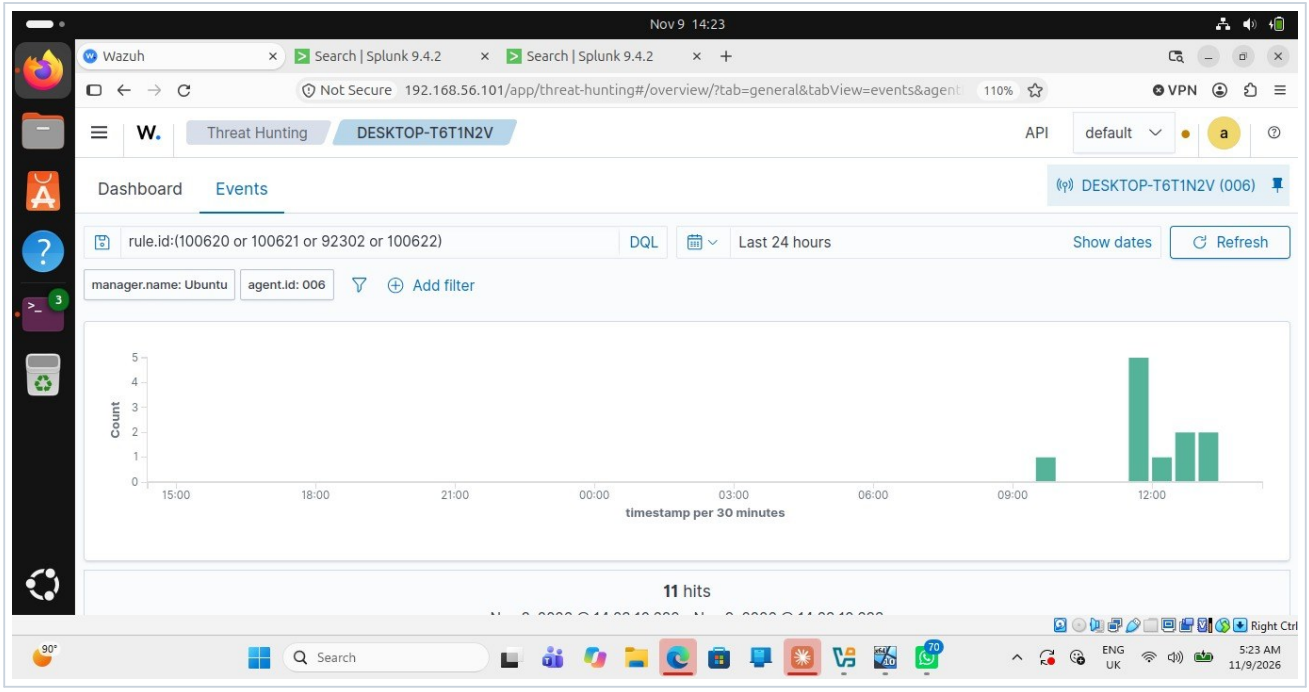


Figure 27. *11 hits for the four rules over the last 24 hours (the bars show when the tests happened).* *(source file 1000499597.jpg)*

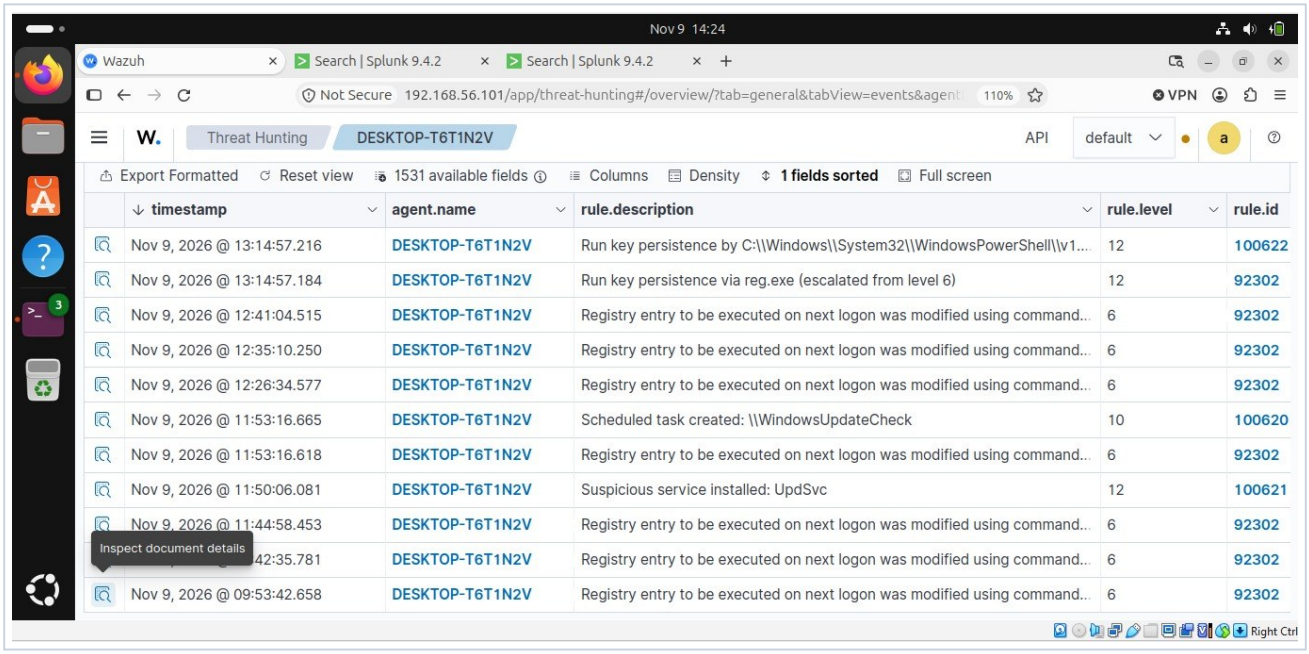


Figure 28. *Alert table: 92302 level 6 alerts (09:53 to 12:41), then 100621 (11:50), 100620 (11:53), and the level 12 alerts at 13:14.* *(source file 1000499598.jpg)*

Opening an alert (the 'inspect document' icon) shows every field. The service alert shows the account, path and name of the service:

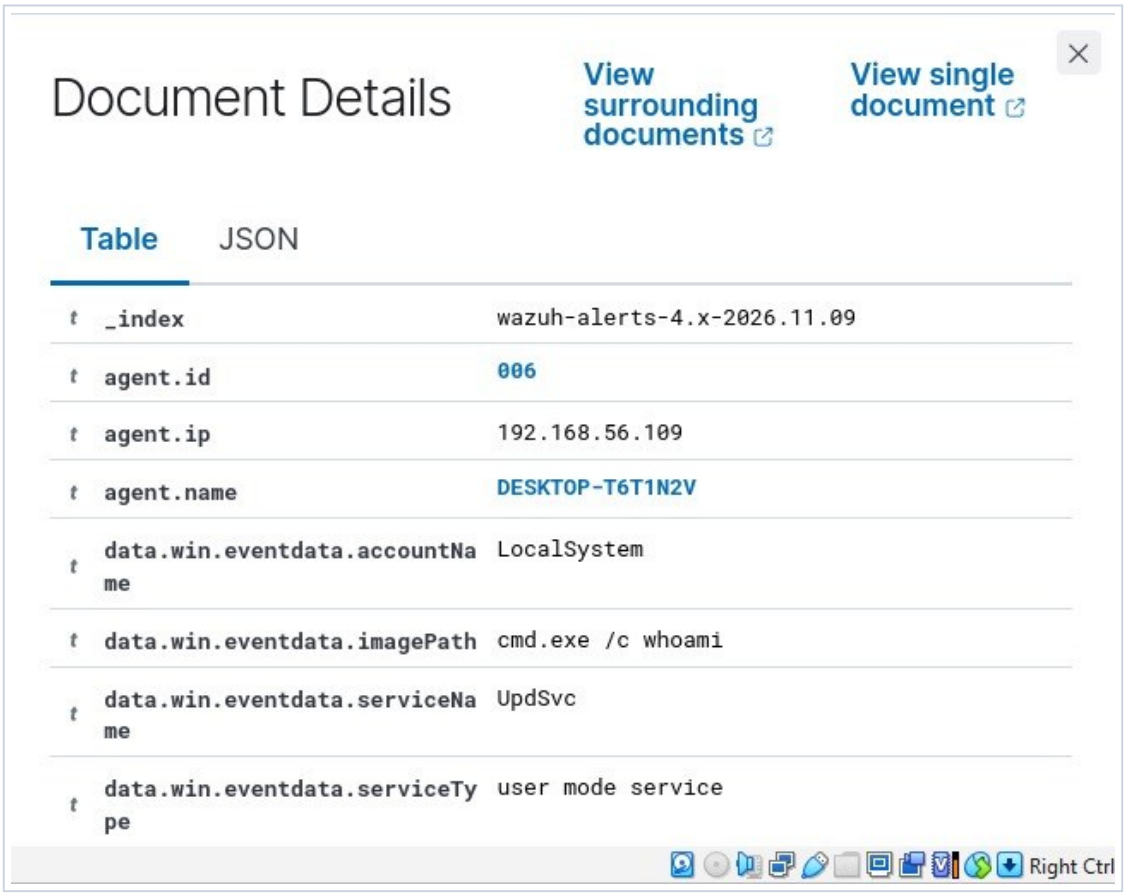


Figure 29. *Alert details for the service (rule 100621): service UpdSvc, image path cmd.exe /c whoami, account LocalSystem, start type auto.* *(source file 1000499605.jpg)*

The scheduled-task alert reports rule 100620, level 10, MITRE T1053.005 (Execution, Persistence, Privilege Escalation) at 11:53:16:

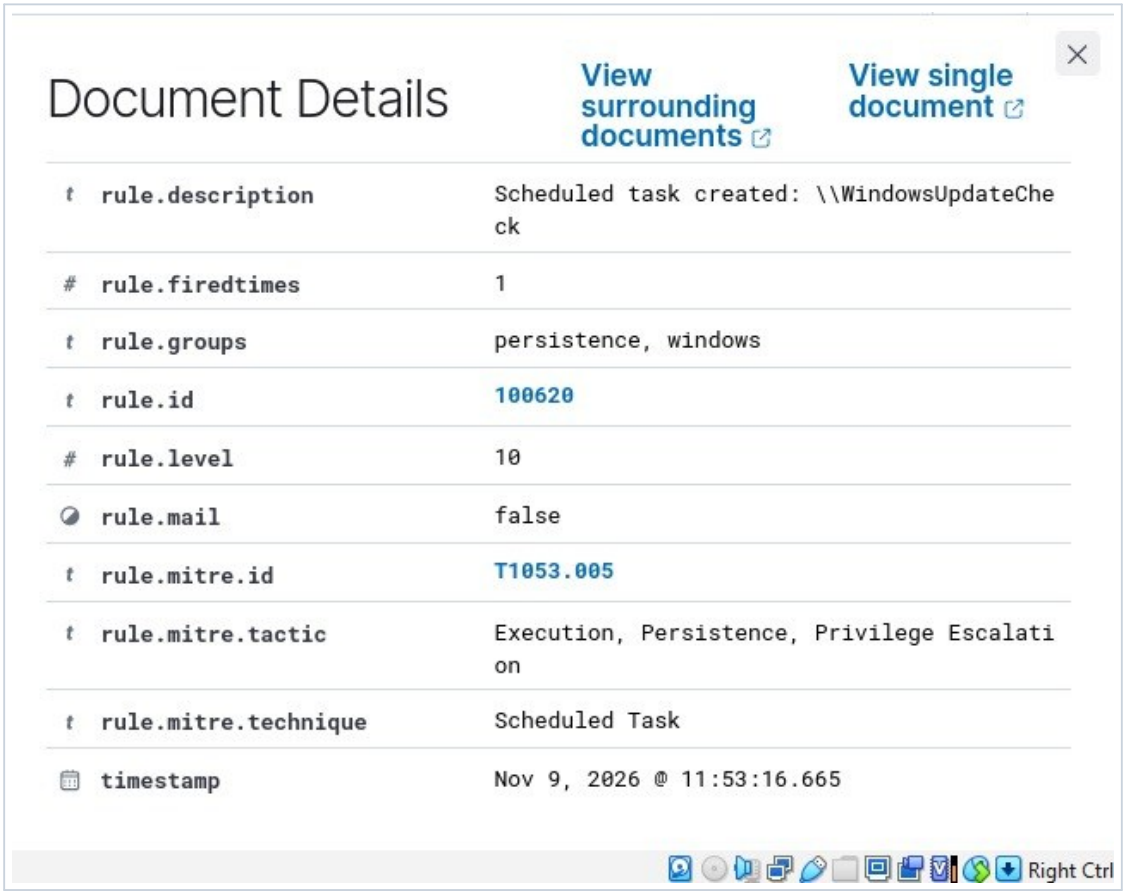


Figure 30. *Alert details for rule 100620.* *(source file 1000499627.jpg)*

The same alert contains the logon ID `0x180b06` and user Administrator of the domain HOPELABS:

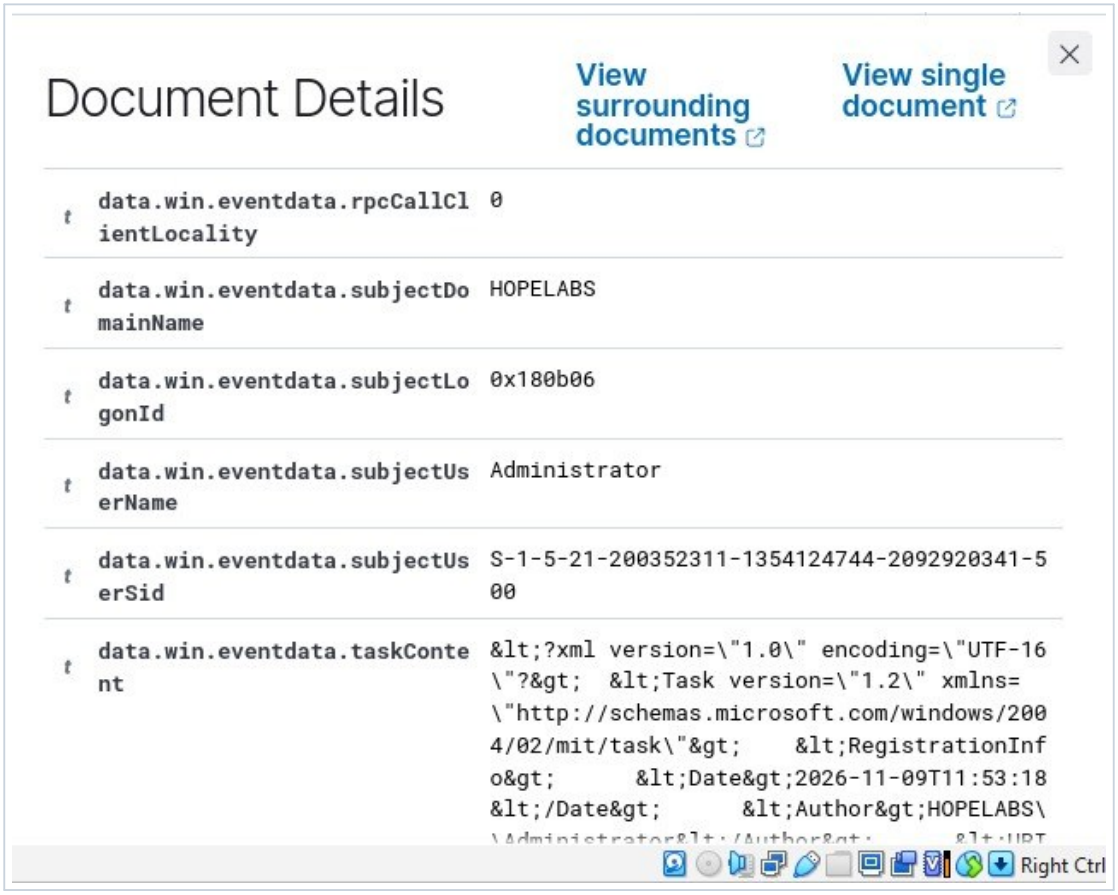


Figure 31. *Task alert fields: subjectUserName Administrator, subjectLogonId 0x180b06, SID ending -500. (source file 1000499618.jpg)*

It also contains the parent process ID 6324, the PowerShell window that launched the task creation:

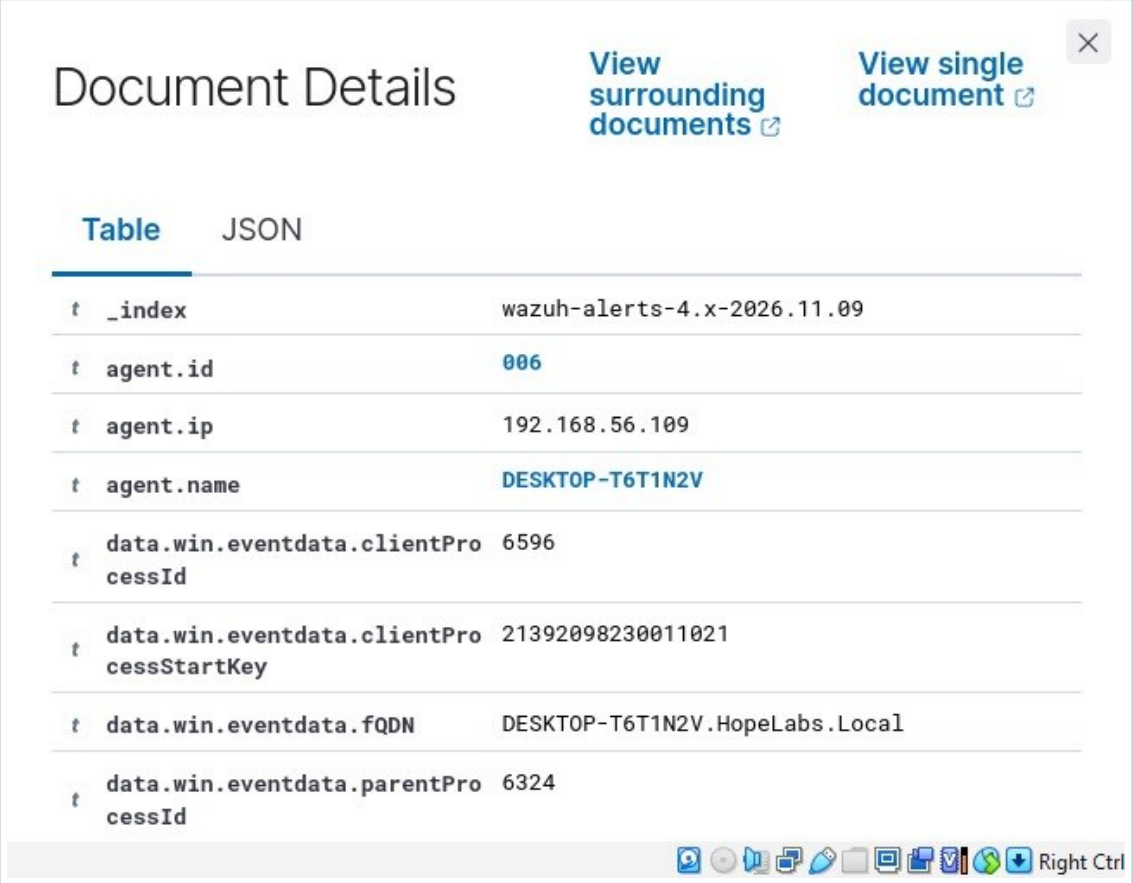


Figure 32. *Task alert fields: clientProcessId 6596 and parentProcessId 6324.* *(source file 1000499617.jpg)*

> **NOTE**
> Wazuh only stores events that triggered a rule, so it cannot show the whole chain of programs. That is why Splunk was used next.

6.11 Step 11: Investigate in Splunk

Splunk keeps every event, so it can answer 'what exactly was typed, and by whom?'. All searches used the time range Nov 9, 09:00 to 14:00 or the last 24 hours.

Command: See what Windows data Splunk holds

Where it runs: Browser: Splunk Search **Privileges needed:** Splunk login

```
index=* host=DESKTOP-T6T1N2V | stats count by index, sourcetype
```

Breaking the command into pieces

Part	What it means in plain words
index=*	Search every index
host=DESKTOP-T6T1N2V	Only events from our Windows computer
| stats count by index, sourcetype	Count events and group the counts by index and log type

Why we ran it

To confirm Splunk really receives the endpoint's raw logs before searching.

What we expected

A small table of indexes.

What actually happened

5,180 events in six rows (Figure below; table in Section 4.3).

Common problems and how to recover

If nothing is returned, the log forwarder or the index name is wrong.

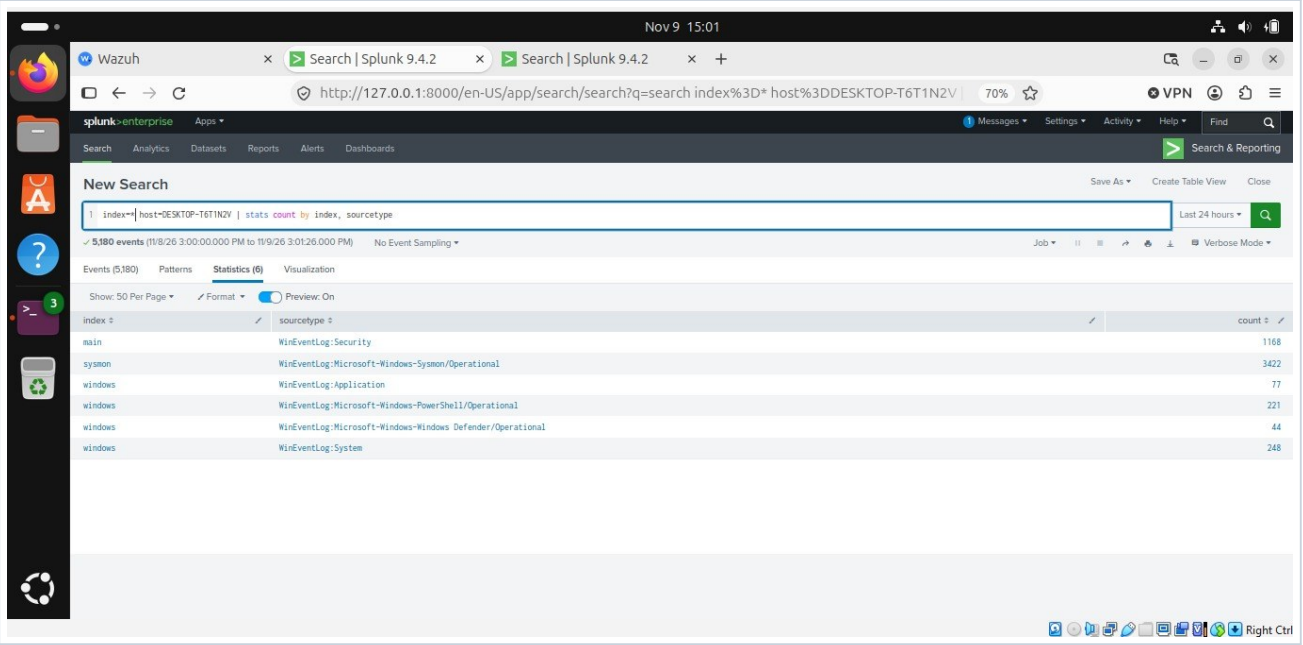


Figure 33. *Splunk holds Security, Sysmon, System, Application, PowerShell and Defender logs for the host.* *(source file 1000499658.jpg)*

Command: Find the scheduled task creation (Security 4698)

Where it runs: Browser: Splunk Search **Privileges needed:** Splunk login

```
index=* EventCode=4698 WindowsUpdateCheck
```

Breaking the command into pieces

Part	What it means in plain words
EventCode=4698	Windows event number for 'task created'
WindowsUpdateCheck	Plain word search for our task name

Why we ran it

To see the raw task-creation events.

What we expected

Task events.

What actually happened

Two events: 10:38:42 (record 124192) and 11:53:18 (record 124411). Both: account Administrator, logon ID 0x180B06, parent process ID 6324, action `cmd.exe /c whoami > C:\Users\Public\p.txt`. There is **no** 4698 for the first creation at about 09:53 (audit policy was off).

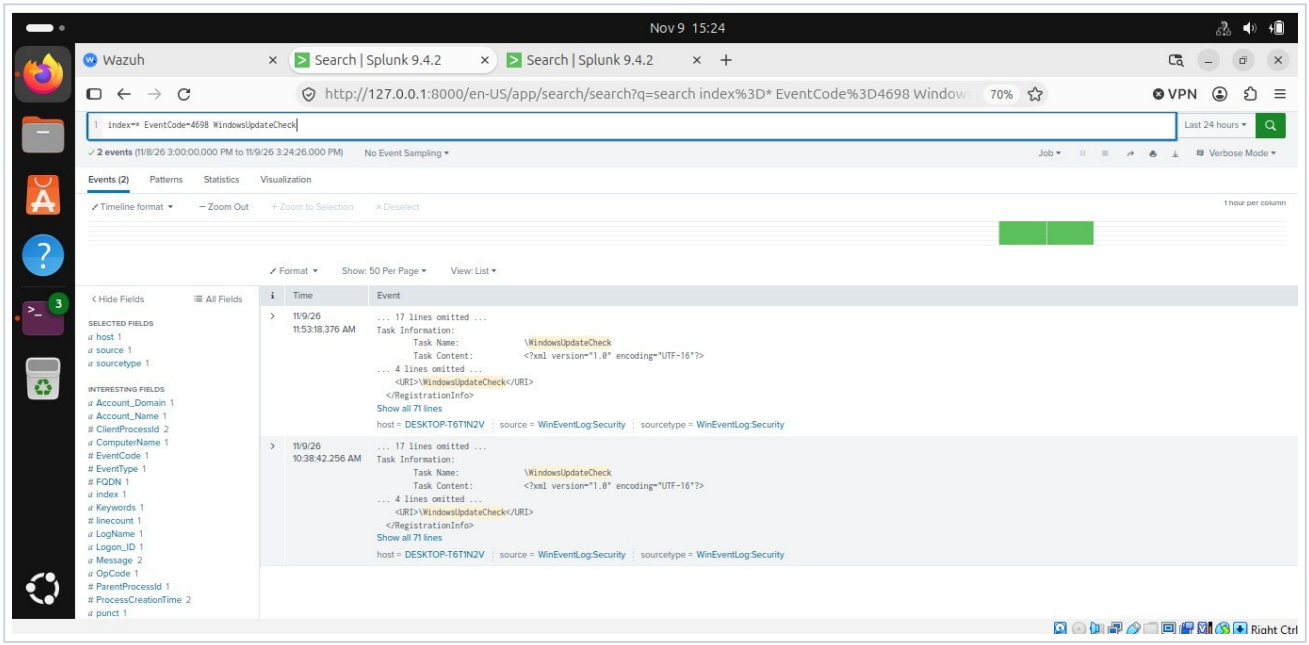


Figure 34. *Splunk: two 4698 events (11:53:18 and 10:38:42) for WindowsUpdateCheck.* *(source file 1000499711.jpg)*

Command: Find the service installs (System 7045)

Where it runs: Browser: Splunk Search	**Privileges needed:** Splunk login

```
index=windows EventCode=7045 UpdSvc
```

Breaking the command into pieces

Part	What it means in plain words
index=windows	The index that holds the System log
EventCode=7045	'A service was installed'
UpdSvc	Our service name

Why we ran it

To see the raw service-install events.

What we expected

Service events.

What actually happened

Two events: 10:11:45 (record 7599) and 11:50:07 (record 7721): UpdSvc, `cmd.exe /c whoami`, auto start, LocalSystem, installed under the Administrator SID (ending -500).

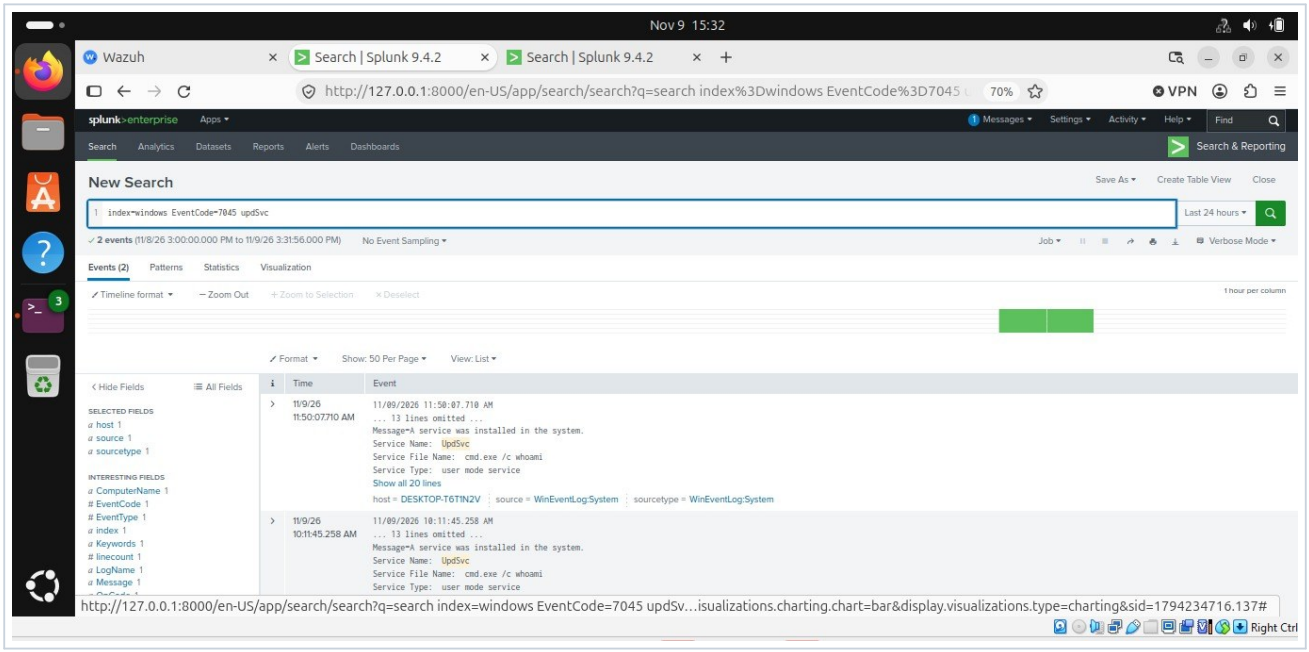


Figure 35. *Splunk: two 7045 events for UpdSvc.* *(source file 1000499763.jpg)*

Command: Find the Run key writes (Sysmon 13)

Where it runs: Browser: Splunk Search	**Privileges needed:** Splunk login

```
index=sysmon EventCode=13 Updater*
```

Breaking the command into pieces

Part	What it means in plain words
index=sysmon	The Sysmon index
EventCode=13	Registry value set
Updater*	Any value name starting with 'Updater' (the * is a wildcard)

Why we ran it

To list every Run key write for our test names.

What we expected

12 events.

What actually happened

12 events: `reg.exe` wrote `Updater` and `powershell.exe` (PID 6324) wrote `Updater2` and `Updater3`.

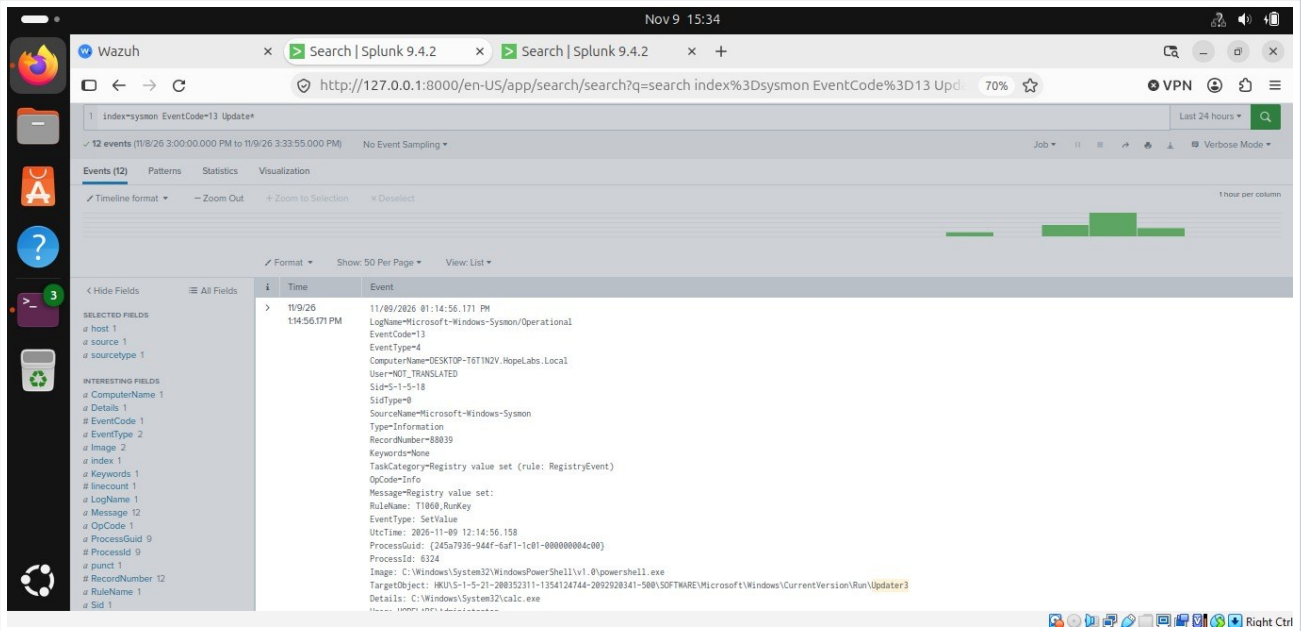


Figure 36. *Splunk: 12 Sysmon event 13 results for Updater*.* *(source file 1000499760.jpg)*

Command: Trace the process chain (Sysmon 1)

Where it runs: Browser: Splunk Search **Privileges needed:** Splunk login

```
index=sysmon EventCode=1 (schtasks.exe OR sc.exe OR reg.exe) powershell.exe
```

Breaking the command into pieces

Part	What it means in plain words
EventCode=1	Process started
(schtasks.exe OR sc.exe OR reg.exe)	Any of the three tools. **OR must be in capital letters** in Splunk
powershell.exe	Also mentions PowerShell (the parent)

Why we ran it

To see which program started the three commands.

What we expected

Process events with parent fields.

What actually happened

34 events (Figure below). Opening one shows `reg.exe` with ParentImage `powershell.exe`, ParentProcessId 6324, user HOPELABS\Administrator, logon ID 0x180B06, integrity level High.

Common problems and how to recover

Lower-case 'or' returned 0 events earlier (Chapter 10, Problem 9).

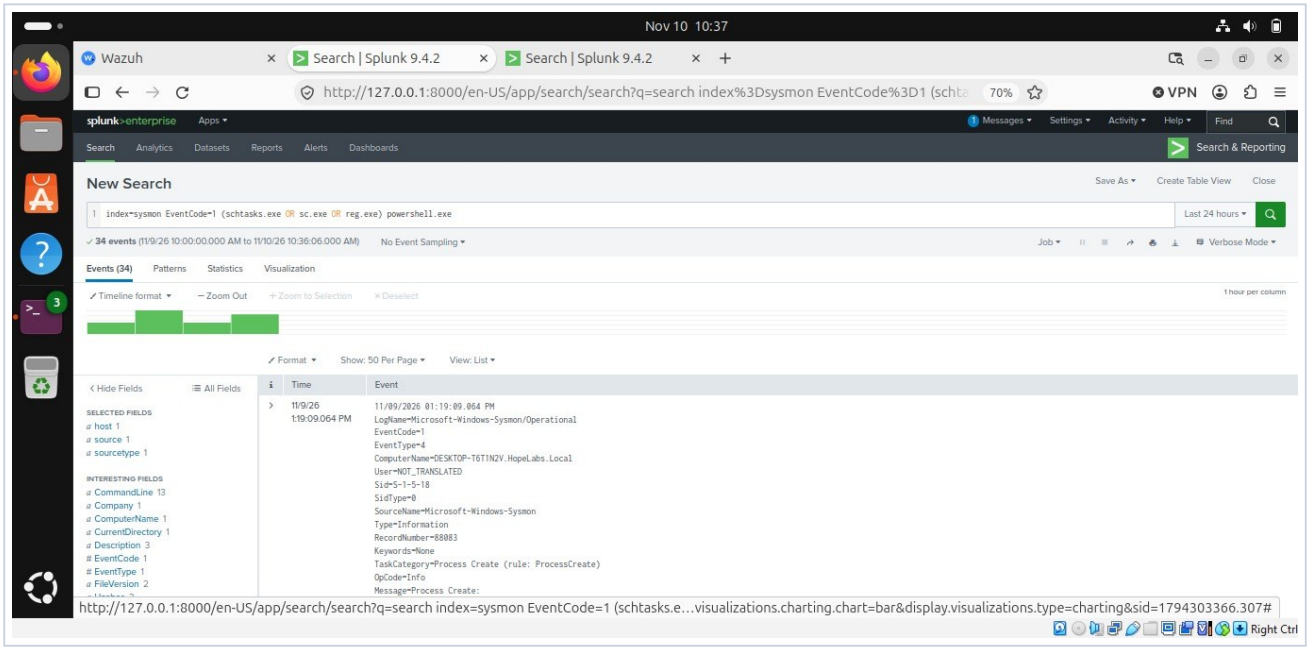


Figure 37. *34 process-create events match the search.* *(source file 1000500396.jpg)*

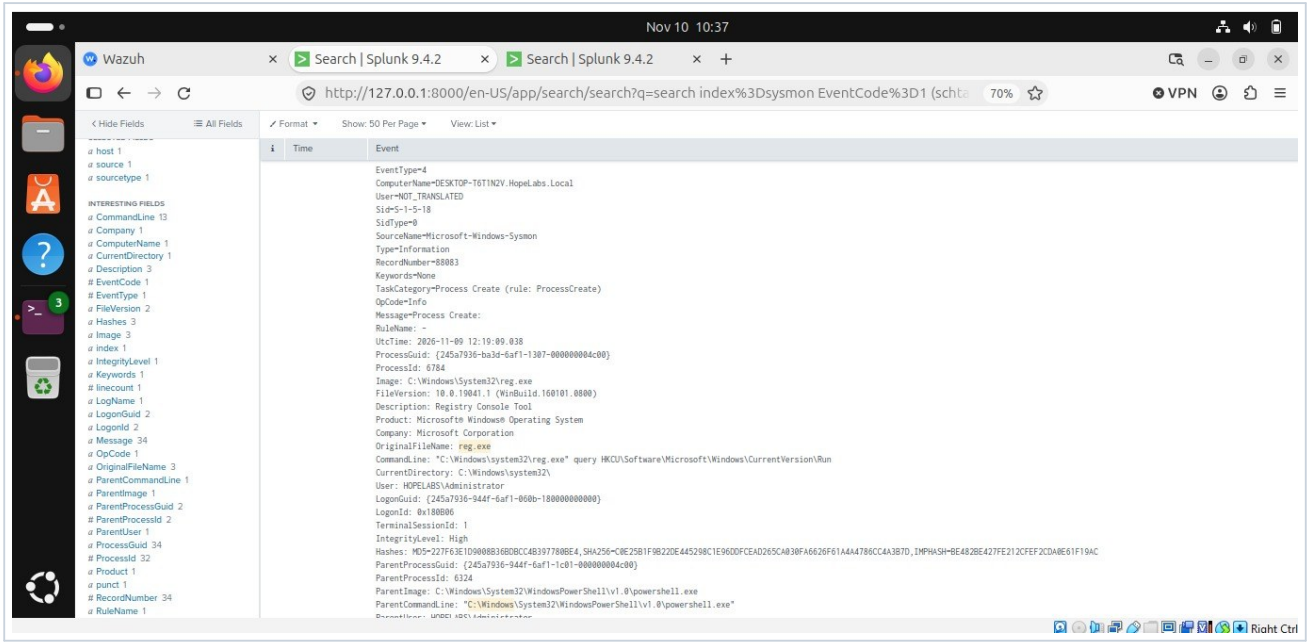


Figure 38. *A reg.exe process event: ParentImage powershell.exe, ParentProcessId 6324, LogonId 0x180B06, IntegrityLevel High.* *(source file 1000500397.jpg)*

Command: List everything PowerShell 6324 launched, in order

Where it runs: Browser: Splunk Search	**Privileges needed:** Splunk login

```
index=sysmon EventCode=1 ParentProcessId=6324 | sort -_time | table _time Image CommandLine
    LogonId ParentProcessId
```

Breaking the command into pieces

Part	What it means in plain words
ParentProcessId=6324	Only programs started by our PowerShell window
| sort -_time	Sort by time, newest first (the minus sign reverses the order)

Part	What it means in plain words
| table ...	Show only these columns

Why we ran it

To get the complete command history of the session.

What we expected

28 rows.

What actually happened

Every row shows the same logon 0x180B06 and parent 6324. They include the create commands (for example `schtasks /create /tn WindowsUpdateCheck ...`) and the clean-up commands.

Why this matters for security

One session, one logon ID: the evidence ties every alert to a single actor.

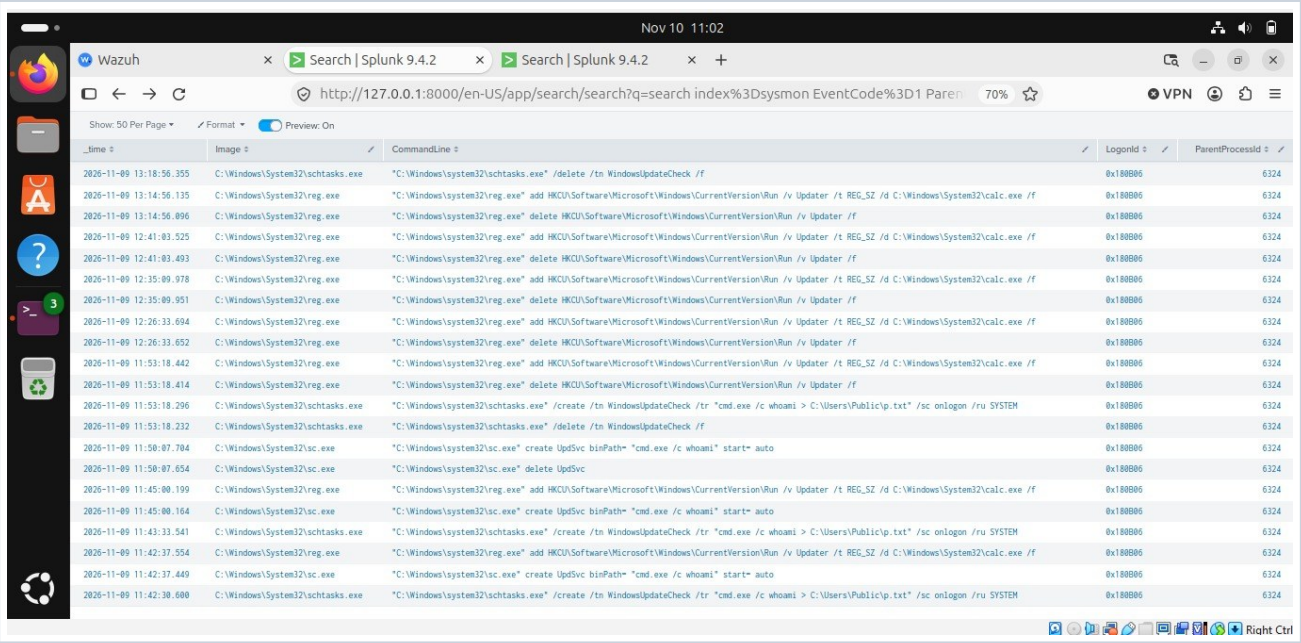


Figure 39. *All 28 commands launched by PowerShell 6324, with the same logon ID.* *(source file 1000500440.jpg)*

Command: Build one combined timeline

Where it runs: Browser: Splunk Search **Privileges needed:** Splunk login

```
(index="main" EventCode=4698 WindowsUpdateCheck) OR (index=windows EventCode=7045 UpdSvc) OR
    (index=sysmon EventCode=13 Updater*) OR (index=sysmon EventCode=1 ParentProcessId=6324
    CommandLine="*create*") | sort -_time | table _time index EventCode Image CommandLine
    TargetObject
```

Breaking the command into pieces

Part	What it means in plain words
(...) OR (...)	Four searches combined into one
CommandLine="*create*"	Only the create commands
| sort / | table	Order by time and show chosen columns

Why we ran it

To see command, event and registry write together in time order.

What we expected

19 events.

What actually happened

19 events. For example `schtasks /create` at 11:53:18.296 is followed 80 ms later by the 4698 event at 11:53:18.376, and `sc.exe create UpdSvc` at 11:50:07.704 is followed by the 7045 event at 11:50:07.710.

Why this matters for security

Showing cause and effect within milliseconds is strong evidence for an incident timeline.

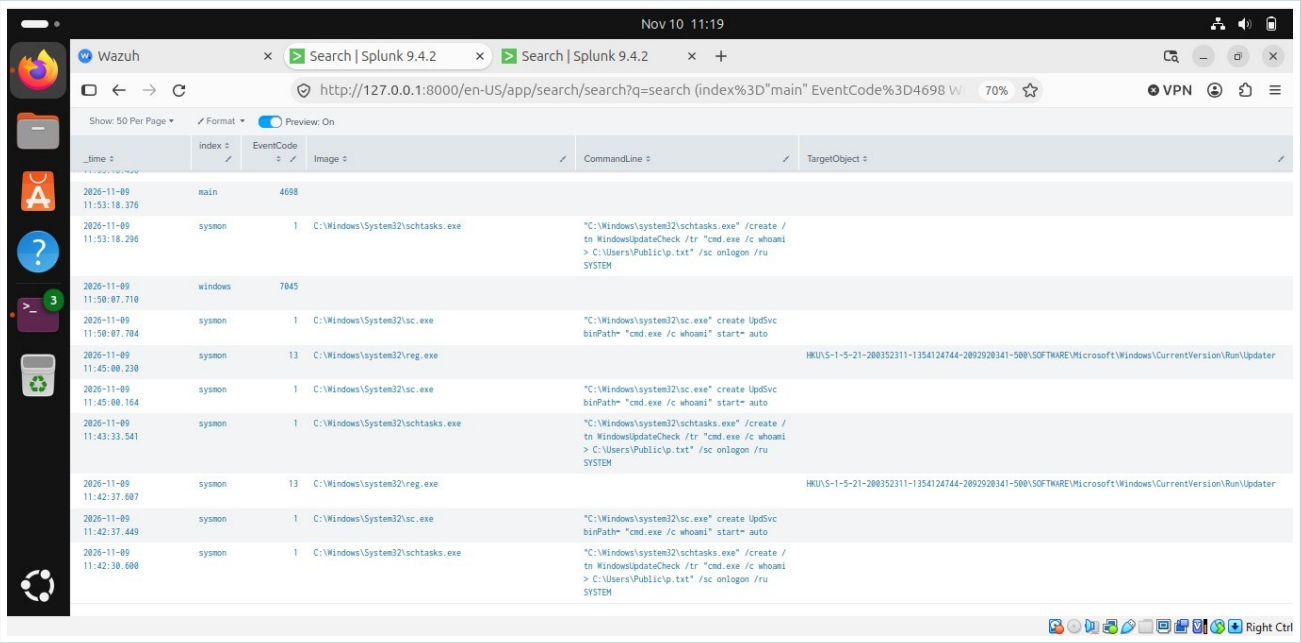


Figure 40. *Combined timeline: a create command in Sysmon followed within milliseconds by the matching Windows event. (source file 1000500443.jpg)*

> **IN SIMPLE TERMS**
> Failed attempts are visible too. The `schtasks /create` at 11:42:30 and 11:43:33 and the `sc create UpdSvc` at 11:42:37 and 11:45:00 have **no** matching 4698 or 7045 event, because those items already existed and the commands failed. Process logs show every attempt; Windows logs record only successes.

6.12 Step 12: Contain and clean up

In a real incident, containment would also mean isolating the computer and disabling the account. In the lab, 'containment' means removing everything that was created and proving it is gone.

Command: Remove the task, service, Run keys and test file

Where it runs: Windows 10, PowerShell	**Privileges needed:** Administrator

```
schtasks /delete /tn "WindowsUpdateCheck" /f
sc.exe delete UpdSvc
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /f
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater2 /f
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater3 /f
Remove-Item C:\Users\Public\p.txt -ErrorAction SilentlyContinue
```

Breaking the command into pieces

Part	What it means in plain words
schtasks /delete ... /f	Delete the scheduled task without asking

Part	What it means in plain words
sc.exe delete UpdSvc	Delete the service
reg delete ... /v NAME /f	Delete one Run value
Remove-Item PATH	Delete a file
-ErrorAction SilentlyContinue	If the file does not exist, do not print an error

Why we ran it

To return the computer to its original state.

What we expected

SUCCESS messages and 'The operation completed successfully'.

What actually happened

All succeeded at about 13:18 (Figure below).

Common problems and how to recover

If a service says 'marked for deletion', close tools that have it open or restart.

Why this matters for security

Removing the item alone is not enough in a real incident: the attacker's access (account, password, malware) must also be found.

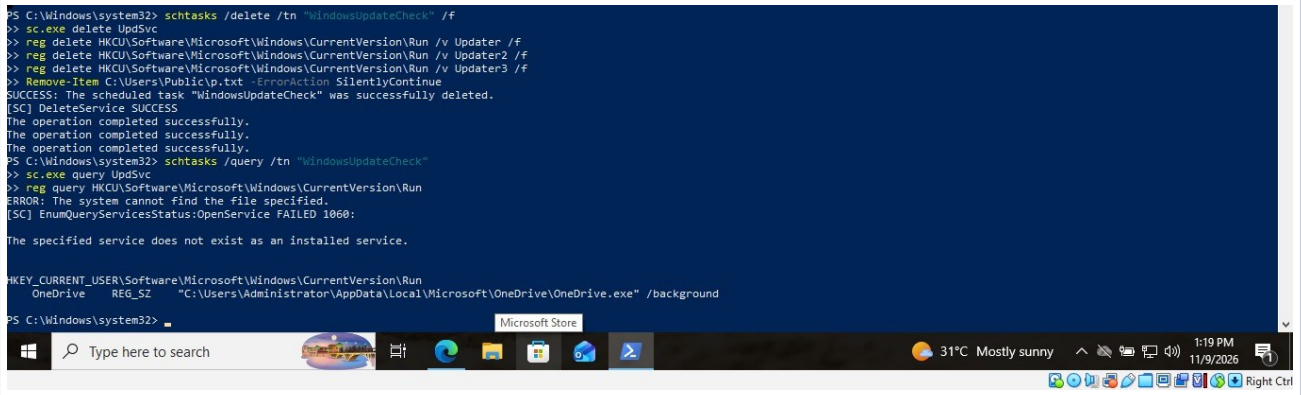


Figure 41. *Clean-up and verification at 13:19: the task is not found, the service does not exist (error 1060), and only OneDrive remains in the Run key.* *(source file 1000499422.jpg)*

Command: Verify the clean-up

Where it runs: Windows 10, PowerShell **Privileges needed:** Administrator

```
schtasks /query /tn "WindowsUpdateCheck"
sc.exe query UpdSvc
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Breaking the command into pieces

Part	What it means in plain words
schtasks /query	Ask for the task
sc.exe query	Ask for the service status
reg query	List the Run key

Why we ran it

To prove, not assume, that everything is gone.

What we expected

Not found / does not exist / only OneDrive.

What actually happened

Exactly that: 'ERROR: The system cannot find the file specified', '[SC] EnumQueryServicesStatus:OpenService FAILED 1060: The specified service does not exist as an installed service', and the Run key lists only OneDrive.

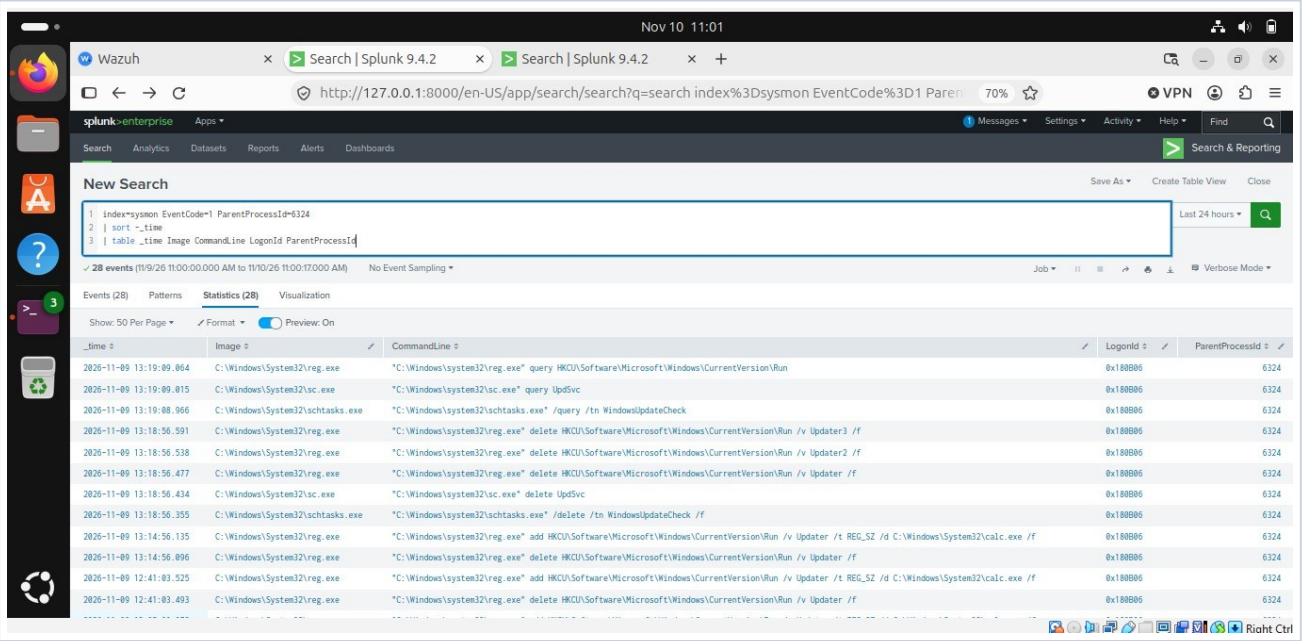


Figure 42. *Splunk also recorded the clean-up and verification commands (13:18 to 13:19), all from PowerShell 6324. (source file 1000500439.jpg)*

Chapter 7: Configuration and Detection Logic

7.1 The final rules

These are the rules that were running at the end of the lab, in `/var/ossec/etc/rules/local_rules.xml` on the Ubuntu server.

```
<group name="persistence,windows,">

  <rule id="100620" level="10">
    <if_sid>60228</if_sid>
    <description>Scheduled task created: $(win.eventdata.taskName)</description>
    <mitre><id>T1053.005</id></mitre>
  </rule>

  <rule id="100621" level="12">
    <if_sid>61138</if_sid>
    <field name="win.eventdata.imagePath"
        type="pcre2">(?i)cmd\.exe|powershell|Users.Public|.Temp.</field>
    <description>Suspicious service installed: $(win.eventdata.serviceName)</description>
    <mitre><id>T1543.003</id></mitre>
  </rule>

  <rule id="92302" level="12" overwrite="yes">
    <if_sid>92300</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)reg\.exe</field>
    <options>no_full_log</options>
    <description>Run key persistence via reg.exe (escalated from level 6)</description>
    <mitre><id>T1547.001</id></mitre>
  </rule>

  <rule id="100622" level="12">
    <if_sid>92300</if_sid>
    <field name="win.eventdata.image" type="pcre2"
        negate="yes">(?i)reg\.exe|msedge\.exe|OneDrive\.exe|CompatTelRunner</field>
    <description>Run key persistence by $(win.eventdata.image)</description>
    <mitre><id>T1547.001</id></mitre>
  </rule>

</group>
```

7.2 Line-by-line explanation

Table 10. What each part of a rule means

Part	Meaning in plain words
<group name="persistence,windows,">	A label for the family of rules so they can be filtered in the dashboard (rule.groups)
<rule id="100620" level="10">	One rule. IDs 100000 and above are reserved for your own rules. The level (0 to 15) sets how serious the alert looks
<if_sid>60228</if_sid>	'Only look at events that rule 60228 already recognised.' This makes 100620 a child of the built-in 'scheduled task created' rule
<field name="win.eventdata.imagePath" type="pcre2">...</field>	A condition on one named field. `pcre2` means the text is a pattern. `(?i)` ignores capital letters; `cmd\.exe|powershell|Users.Public|.Temp.` means 'contains cmd.exe, or powershell, or a path through Users\Public or Temp' (the dot matches any one character, used instead of a backslash)
negate="yes"	Reverse the test: the rule matches when the field does **not** contain the pattern. Used to exclude normal programs

Part	Meaning in plain words
<description>...$(field)...</description>	The alert title. `$(win.eventdata.taskName)` is replaced with the real value, which is why alerts name the task or service
<mitre><id>T1053.005</id></mitre>	The MITRE ATT&CK tag attached to the alert
overwrite="yes"	'Replace the built-in rule that has this same ID with my version.' Used on rule 92302 to raise its level from 6 to 12
<options>no_full_log</options>	Do not store the whole original log line in the alert (kept because the original rule had it)

7.3 How the rules work together

Table 11. Detection logic summary

Rule	Triggers when...	Parent	Level	MITRE
100620	Security event 4698 (task created) arrives and built-in 60228 matches	60228	10	T1053.005
100621	System event 7045 (service installed) arrives and the file path contains cmd.exe, powershell, Users\Public or Temp	61138	12	T1543.003
92302 (overwritten)	Sysmon event 13 writes a Run key and the writing program is reg.exe	92300	12 (was 6)	T1547.001
100622	Sysmon event 13 writes a Run key and the writing program is **not** reg.exe, Edge, OneDrive or CompatTelRunner	92300	12	T1547.001

IN SIMPLE TERMS
Why exclude Edge, OneDrive and CompatTelRunner in rule 100622? Splunk and the local Sysmon query showed that these programs write Run values all the time for ordinary reasons. Excluding them reduces false alarms. The Run key query in Section 6.5 was how that normal baseline was discovered.

7.4 Original, changed and recommended configuration

Table 12. Configuration changes

Item	Original state	Change actually made	Recommended future change
Audit policy 'Other Object Access Events'	Off (no 4698 events)	Enabled on the lab host with auditpol	Enable by Group Policy on all computers
Wazuh agent Sysmon collection	Already configured (line 50)	None	Keep
Rule 100620	Did not exist	Added (child of 60228)	Add an allow-list for known administrative tasks
Rule 100621	Did not exist	Added (child of 61138)	Alert on any service whose path is not in System32 or Program Files
Rule 92302	Level 6, built-in	Overwritten to level 12 in local_rules.xml	Alert only when the value path is not an approved program
Rule 100622	Did not exist	Added (first attempt failed, rebuilt on parent 92300)	Add exclusions only after reviewing each in production
PowerShell Script Block Logging	Unknown / not verified	None	Enable and verify (Event 4104)

Item	Original state	Change actually made	Recommended future change
Time synchronisation	About 2 seconds difference between host and manager	None	Use NTP on all machines

7.5 Rule version history

Table 13. How the Run key rules evolved

Version	What it was	Outcome
v1	All three rules used `if_group windows` (broad)	Worked for neither test until the parents were identified
v2	Rules rewritten with `if_sid` 60228, 61138, 61615	100620 and 100621 fired; 100622 did not
v3	Added 100623 as a child of the built-in rule 92302	Did not fire; root cause unconfirmed; removed
v4 (final)	Overwrite 92302 to level 12; rebuild 100622 on parent 92300	Both fired at 13:14:57

Chapter 8: Testing and Results

8.1 Test results

Table 14. Tests, expectations and outcomes

#	Test	Expected	Actual	Status
T1	Task creation detection	Event 4698 and an alert	Event 4698 at 10:38:42 and 11:53:18; rule 100620 level 10 at 11:53:16	Verified
T2	Service install detection	Event 7045 and an alert	Event 7045 at 10:11:45 and 11:50:07; rule 100621 level 12 at 11:50:06	Verified
T3	Run key by reg.exe	Sysmon 13 and an alert	Built-in 92302 level 6 at 09:53:42; after overwrite level 12 at 13:14:57	Verified (after tuning)
T4	Run key by PowerShell	Sysmon 13 and an alert	Rule 100622 level 12 at 13:14:57, image powershell.exe	Verified
T5	Account and process correlation	Identify who and how	Administrator, logon 0x180B06, parent PowerShell 6324 in Wazuh and Splunk	Verified
T6	MITRE mapping	IDs attached to alerts	T1053.005, T1543.003, T1547.001 shown in alerts	Verified
T7	Clean-up	Items removed	Not found / error 1060 / only OneDrive	Verified
T8	Task creation at about 09:53	Event 4698	No event (audit policy off)	Failed: gap found and fixed
T9	Service creation with 'sc' in PowerShell	Service created	Failed: alias for Set-Content; fixed with sc.exe	Failed then fixed
T10	Original rule 100622 (parent 61615)	Alert	No alert: built-in 92302 matched first	Failed then fixed
T11	Rule 100623 (child of 92302)	Alert	No alert; cause not confirmed	Failed; unresolved
T12	Logtest with pasted JSON	Show rule evaluation	Processed as a syslog message, so it did not test the Windows rules	Inconclusive; not used as evidence
T13	PowerShell command log in Splunk	Show typed commands	0 results, but the query was wrong (lower-case 'or')	Unverified

8.2 What was not tested

- Other autostart locations: Startup folder, RunOnce, machine-wide (HKLM) Run keys, WMI event subscriptions, Winlogon and image-file-execution options.
- Persistence created by a non-administrator account or by a real remote attacker.
- Whether the task or service ever *executed*; the lab verified creation, not execution.
- False-positive rates on a busy network; only one lab host was observed.
- Whether the Wazuh rules still work after a Wazuh upgrade or on other Windows versions.
- Splunk alerting or dashboards; Splunk was used for searching only.

Chapter 9: Evidence and Screenshot Analysis

Each screenshot sits next to the step it supports in Chapter 6. The table below lists every figure in order with what it shows, what it demonstrates and its limitation, so that an assessor can check the claims quickly. Source file names allow each figure to be compared to the original.

Table 15. Evidence register

Fig.	Title	Section	Source file	What it demonstrates	Limitation
1	The Day 8 lab brief: objective, six lab tasks, four required deliverables and six analyst ...	2.2	497576.jpg	Task list, deliverables and questions	Design reference only; no lab result
2	Lab architecture schematic	Chapter 4	drawn schematic	Shows the documented design	Not a screenshot
3	The agent settings file contains the Sysmon channel (line 50).	6.1	499351.jpg	Agent collects the Sysmon channel	Shows configuration, not that events arrived
4	The audit policy command succeeded and Security event 4698 (A scheduled task was created) ...	6.2	499231.jpg	Auditing enabled; event 4698 captured	Shows one event, recorded after the fix
5	First run (about 09:53). The task and the Run key were created successfully; the service c...	6.3	499109.jpg	Task and Run key created; service failed (alias)	Shows creation attempt, not detection
6	The service was created successfully with sc.exe.	6.3	499122.jpg	Service created	Shows creation only
7	Task details: author, run-as SYSTEM, logon trigger and the command it will run.	6.4	499144.jpg	Task details: SYSTEM, at logon, command	Local view, not a SIEM alert
8	Service configuration and Run key listing (also shows the first 4698 query returning nothi...	6.4	499145.jpg	Service and Run key exist; 4698 query empty at 10:23	Screenshot is a snapshot in time
9	Service installation event 7045 for UpdSvc (10:11:45), followed by unrelated Sysmon regist...	6.5	499146.jpg	Event 7045 for UpdSvc at 10:11:45	Local log only
10	Sysmon event 13 at 09:53:44: reg.exe wrote the Updater Run value (calc.exe) as HOPELABS\Ad...	6.5	499236.jpg	reg.exe wrote the Run value at 09:53:44	Local log only
11	Sysmon event 1 (10:38:42): schtasks.exe launched by powershell.exe PID 6324, Administrator...	6.6	499257.jpg	Parent process powershell.exe 6324 for schtasks	Shows one command, others in Splunk
12	Example of benign noise: sc.exe start pushtoinstall launched by svchost (Windows Store mai...	6.6	499254.jpg	Benign noise example	Shows baseline only
13	Parent rules found: 60228 (scheduled task created) and 61138 (new Windows service created)...	6.7	499282.jpg	Parent rules 60228 and 61138	Rule IDs depend on the Wazuh version

Fig.	Title	Section	Source file	What it demonstrates	Limitation
14	Parent rule 61615: Sysmon event 13 registry base rule (level 0).	6.7	499283.jpg	Parent rule 61615	As above
15	The first version of the persistence rules in nano, using if_sid with the parents 60228, 6...	6.7	499293.jpg	First if_sid version of the rules	Early version, later replaced
16	Configuration test at 11:27: only the old, unrelated 999xx warnings; no errors for the Day...	6.7	499294.jpg	Config test passes	Only syntax; does not prove detection
17	The 11:53 re-creation: sc.exe delete/create, task delete/create, Run key delete/add all su...	6.8	499329.jpg	Re-creation at 11:53 succeeded	Commands only; detection shown elsewhere
18	Wazuh alert for rule 100621 (level 12, T1543.003), service UpdSvc, 11:50.	6.8	499311.jpg	Alert 100621 fired	Terminal view; pair with dashboard
19	Wazuh alert for rule 100620 (level 10, T1053.005), task \WindowsUpdateCheck, 11:53.	6.8	499330.jpg	Alert 100620 fired	Terminal view; text is long
20	Each Day 8 rule ID appears once in local_rules.xml (one persistence group).	6.9	499405.jpg	Each rule ID appears once	File check only
21	Config test at 13:08 after the clean-up: only the old 999xx warnings.	6.9	499408.jpg	Config test clean at 13:08	Syntax only
22	Manager restarted: 'active (running) since 2026-11-09 13:12:22 WAT'.	6.9	499413.jpg	Manager running after restart	Does not prove rules fire
23	The final Run key test on Windows at 13:15: reg.exe and PowerShell writes.	6.9	499419.jpg	Final triggers on Windows	Commands only
24	Wazuh dashboard: rule 92302 now level 12 at 13:14:57.184 (T1547.001).	6.9	499639.jpg	92302 now level 12	One alert
25	Wazuh dashboard: rule 100622 level 12 at 13:14:57.216, written by powershell.exe.	6.9	499653.jpg	100622 fired for PowerShell	One alert
26	Terminal confirmation (a photograph of the screen): the 92302 and 100622 alerts at level 1...	6.9	20261109_131 7092884227	Both Run key alerts in terminal	Photograph of screen
27	11 hits for the four rules over the last 24 hours (the bars show when the tests happened).	6.10	499597.jpg	11 persistence alerts in 24 hours	Count only
28	Alert table: 92302 level 6 alerts (09:53 to 12:41), then 100621 (11:50), 100620 (11:53), a...	6.10	499598.jpg	All alerts with levels and times	Truncated descriptions

Fig.	Title	Section	Source file	What it demonstrates	Limitation
29	Alert details for the service (rule 100621): service UpdSvc, image path cmd.exe /c whoami,...	6.10	499605.jpg	Service alert fields	Partial field list
30	Alert details for rule 100620.	6.10	499627.jpg	Rule 100620 level 10, T1053.005	Partial field list
31	Task alert fields: subjectUserName Administrator, subjectLogonId 0x180b06, SID ending -500...	6.10	499618.jpg	Task alert user and logon ID	Partial field list
32	Task alert fields: clientProcessId 6596 and parentProcessId 6324.	6.10	499617.jpg	Task alert parent PID 6324	PID only, no process name
33	Splunk holds Security, Sysmon, System, Application, PowerShell and Defender logs for the h...	6.11	499658.jpg	Splunk holds the host's logs	Collection path not shown
34	Splunk: two 4698 events (11:53:18 and 10:38:42) for WindowsUpdateCheck.	6.11	499711.jpg	Two raw 4698 events	No 4698 for the first task
35	Splunk: two 7045 events for UpdSvc.	6.11	499763.jpg	Two raw 7045 events	Installer SID only
36	Splunk: 12 Sysmon event 13 results for Updater*.	6.11	499760.jpg	12 Run key writes	List view
37	34 process-create events match the search.	6.11	500396.jpg	34 matching process events	List view
38	A reg.exe process event: ParentImage powershell.exe, ParentProcessId 6324, LogonId 0x180B0...	6.11	500397.jpg	reg.exe launched by PowerShell 6324	One event
39	All 28 commands launched by PowerShell 6324, with the same logon ID.	6.11	500440.jpg	All commands from one PowerShell session	Starts at 11:42; earlier commands are outside this window
40	Combined timeline: a create command in Sysmon followed within milliseconds by the matching...	6.11	500443.jpg	Command followed by Windows event within ms	Time window starts at 11:00
41	Clean-up and verification at 13:19: the task is not found, the service does not exist (err...	6.12	499422.jpg	Removal verified	Verifies this host only
42	Splunk also recorded the clean-up and verification commands (13:18 to 13:19), all from Pow...	6.12	500439.jpg	Clean-up commands recorded	Does not prove state; see next figure
43	Fix for problem 4: sc.exe delete UpdSvc followed by sc.exe create UpdSvc, both SUCCESS.	10.1	499310.jpg	Service recreated after delete	Commands only
44	Searching for the custom rule IDs returned nothing, while the Updater events were still al...	10.2	499380.jpg	Built-in rule 92302 level 6 claimed the event	Shows the symptom, not the full cause

Fig.	Title	Section	Source file	What it demonstrates	Limitation
45	The built-in rule 92302 (child of 92300, matches image reg.exe, level 6) and, below, my at...	10.2	499392.jpg	Definition of 92302 and attempted 100623	Cause for 100623 failing is unproven
46	logtest output: the pasted JSON went through generic syslog rules (a photograph of the scr...	10.3	20261109_1253337881948	logtest used syslog path	Photograph; test inconclusive
47	wazuh-analysisd -t at 13:05: ERROR (1226) 'Attribute EOF has no value (line 163)' and CRIT...	10.4	20261109_1306066382523	XML error from pasted text	Photograph
48	Splunk search with lower-case 'or' returned 0 events.	10.5	499756.jpg	Zero results from a lower-case or search	Search was wrong, not the data
49	The PowerShell-log search also returned 0 events. Because the query itself was wrong, this...	10.5	499755.jpg	Zero PowerShell results	Does not prove logging is off

Chapter 10: Troubleshooting and Root-Cause Analysis

Things went wrong several times. Each problem is documented with symptoms, cause, how it was diagnosed, the fix, and whether the fix was verified. Honest troubleshooting is the most valuable part of a lab.

Table 16. Problems encountered

#	Symptom	Cause	Fix	Verified?
1	Service command failed with 'Set-Content: ... binPath='	In PowerShell `sc` is an alias of Set-Content	Use `sc.exe`	Yes: CreateService SUCCESS
2	Get-WinEvent returned 'No events were found' for 4698	Audit subcategory 'Other Object Access Events' was off	`auditpol /set ...` then recreate the task	Yes: event 4698 at 10:38:42
3	Sysmon query full of unrelated registry events	Normal Microsoft and browser activity	Filter with `-match` on the Run key path	Yes
4	Error 1073 'service already exists'	Create run twice	Delete first, then create	Yes
5	Custom rules silent after being written	Events happened before the rules were loaded; manager not restarted	Restart, then generate fresh events	Yes: 100620 and 100621 fired
6	Run key alert never came from rule 100622 or 100623	Built-in rule 92302 (level 6) matched first	Overwrite 92302; rebuild 100622 on parent 92300	Yes: both fired at 13:14:57. Exact reason 100623 failed is unconfirmed
7	wazuh-logtest rejected the test	JSON was treated as a syslog line	Abandoned logtest; verified with live events	Cause probable, not confirmed
8	Config test: XMLERR 'Attribute EOF has no value' line 163	Heredoc command text pasted inside nano	Delete stray lines with sed	Yes: test clean at 13:08
9	Splunk search returned 0 events	Lower-case `or` was read as a search word	Use capital `OR`	Yes: later queries returned events
10	Splunk PowerShell-log search returned 0 events	Same lower-case 'or'; also the word 'exe' was not meaningful	Re-run with capital OR (not yet done)	No: unresolved

10.1 Problems 1, 2 and 4 on Windows

Problem 1 is shown in Figure 'First run' (Section 6.3). Problem 2 was visible as the 4698 error in Section 6.5. Problem 4 appeared when the commands were run twice; the second block reported 'FAILED 1073'.



Figure 43. *Fix for problem 4: sc.exe delete UpdSvc followed by sc.exe create UpdSvc, both SUCCESS. (source file 1000499310.jpg)*

10.2 Problem 6: the Run key rule that never fired

Symptom. After the Windows commands, the Wazuh alert list showed rule 92302 'Registry entry to be executed on next logon was modified using command line application reg.exe' at level 6. Rules 100622 and 100623 never appeared.

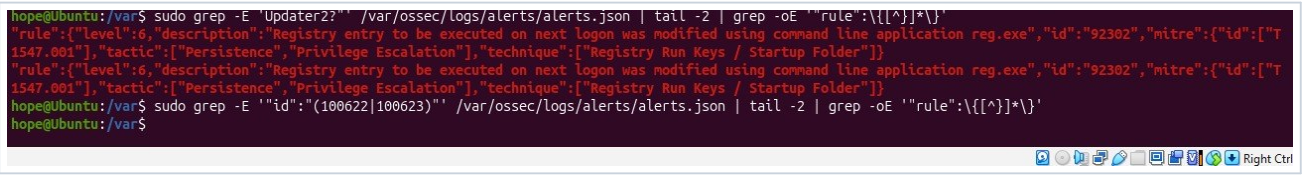


Figure 44. *Searching for the custom rule IDs returned nothing, while the Updater events were still alerts from rule 92302 at level 6.* *(source file 1000499380.jpg)*

Diagnosis. The field values in the alert (image `reg.exe`, targetObject ending `\Run\Updater`, user Administrator) were exactly what my rules looked for, so the rules should have matched. The definition of the built-in rule was then read:

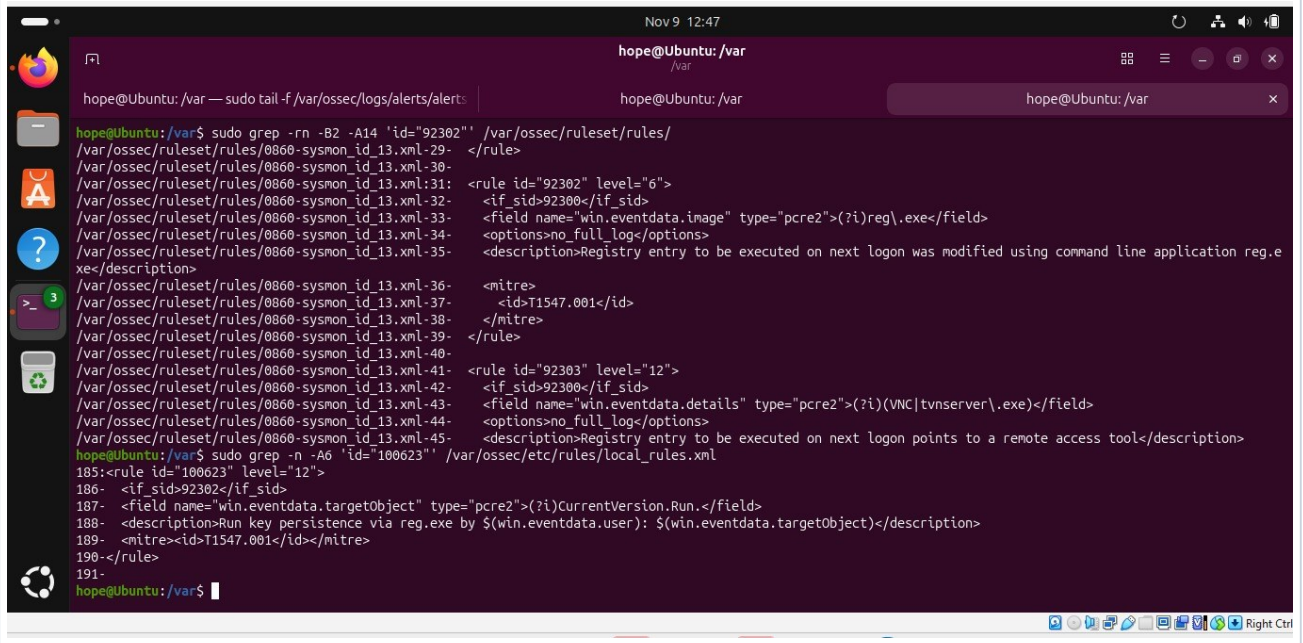


Figure 45. *The built-in rule 92302 (child of 92300, matches image reg.exe, level 6) and, below, my attempted child rule 100623.* *(source file 1000499392.jpg)*

Confirmed. Wazuh already had a rule for reg.exe Run key writes, and its alerts were the ones raised. **Not confirmed.** Why 100623, a child of 92302, never fired. I suspected the double backslashes in my text pattern and replaced them with a dot wildcard, but the rule still did not fire, so that theory remains unproven. **Fix applied.** Rather than keep guessing, I overwrote 92302 at level 12 and rebuilt 100622 on parent 92300 for other writers, then verified with a live test (Section 6.9).

10.3 Problem 7: logtest tested the wrong thing

`wazuh-logtest` is Wazuh's rule tester. The JSON event I pasted was decoded by the generic `json` decoder and then tried against syslog rules (NFS, sendmail and so on), never the Windows rules. This explains the long list of unrelated rule numbers in the output. The result cannot be used as evidence about the Windows rules.

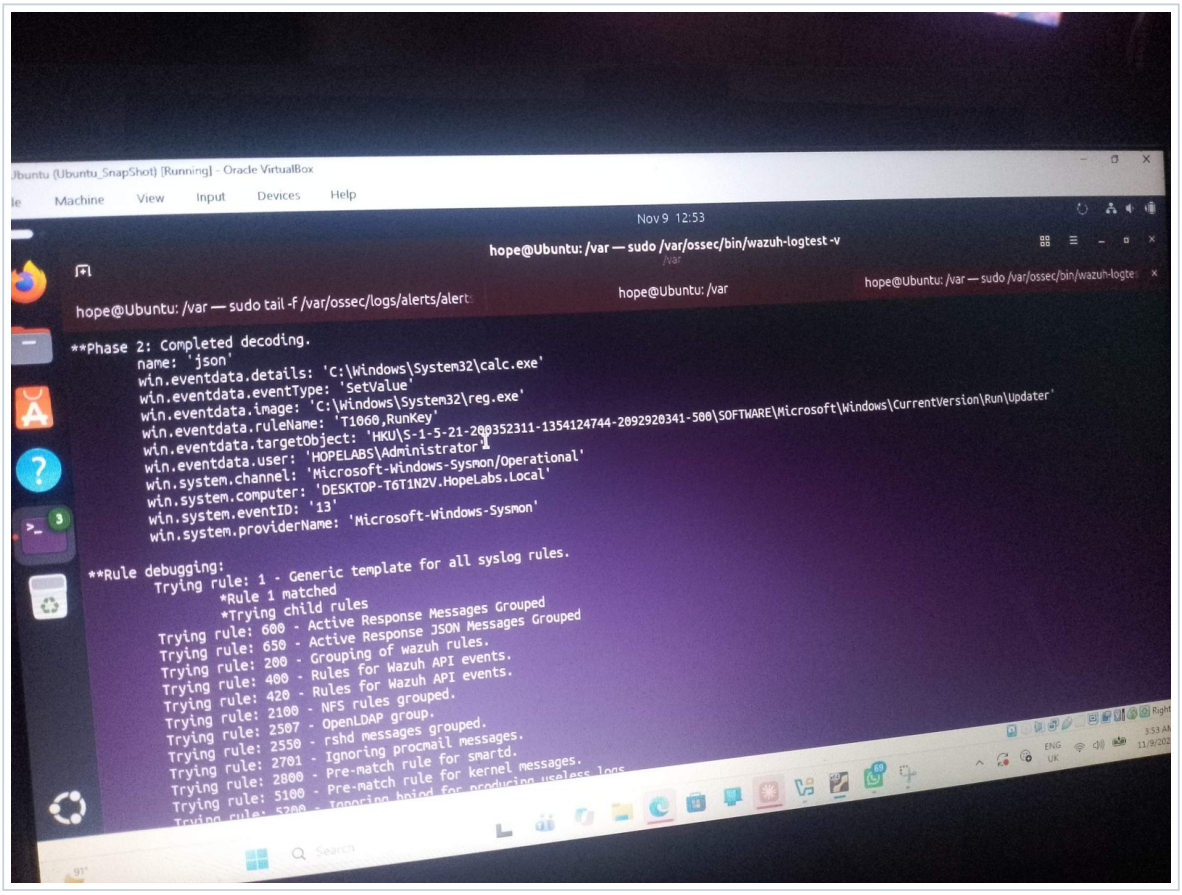


Figure 46. *logtest output: the pasted JSON went through generic syslog rules (a photograph of the screen).* *(source file 20261109_125333788194807387318284 7.jpg)*

10.4 Problem 8: invalid XML after a paste

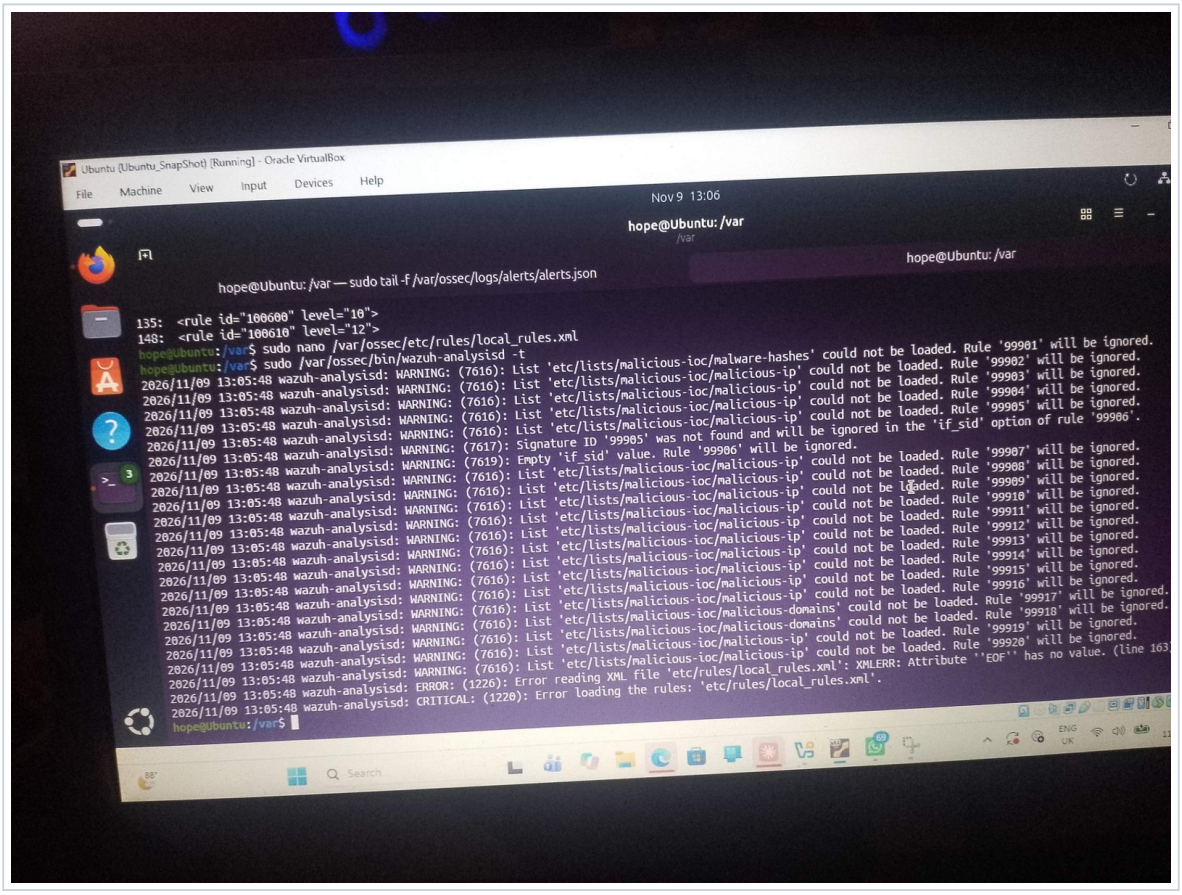


Figure 47. *wazuh-analysisd -t at 13:05: ERROR (1226) 'Attribute EOF has no value (line 163)' and CRITICAL (1220). A photograph of the screen.* *(source file 20261109_130606638252384233915922 3.jpg)*

The stray words `sudo tee` and `EOF` were removed with sed, and the next test at 13:08 was clean (Figure 'cfg test', Section 6.9).

10.5 Problems 9 and 10: Splunk zero results

Splunk requires the words `AND`, `OR` and `NOT` to be in capitals. With lower-case words, Splunk searches for the literal word 'or'. Both searches below returned no events for that reason.

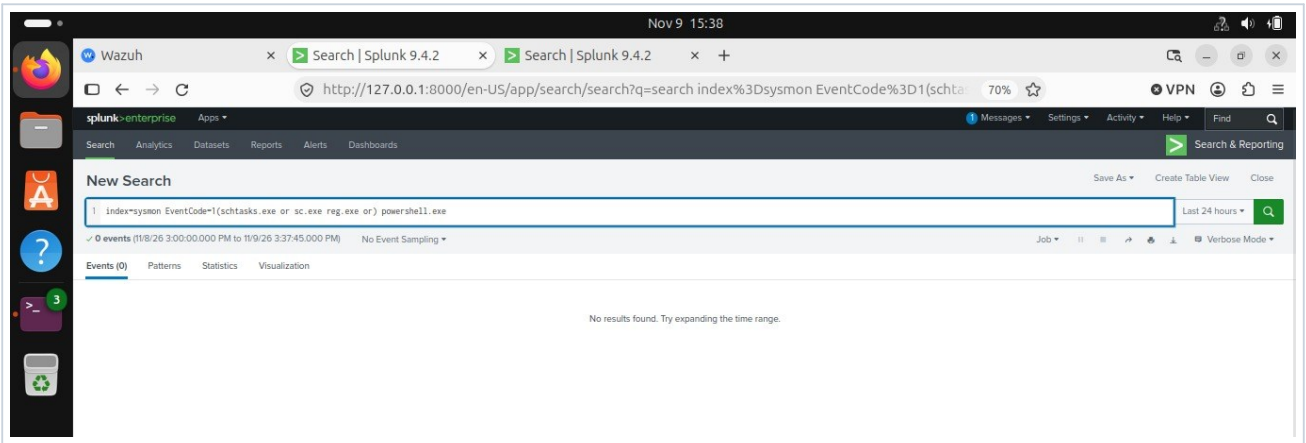


Figure 48. *Splunk search with lower-case 'or' returned 0 events.* *(source file 1000499756.jpg)*

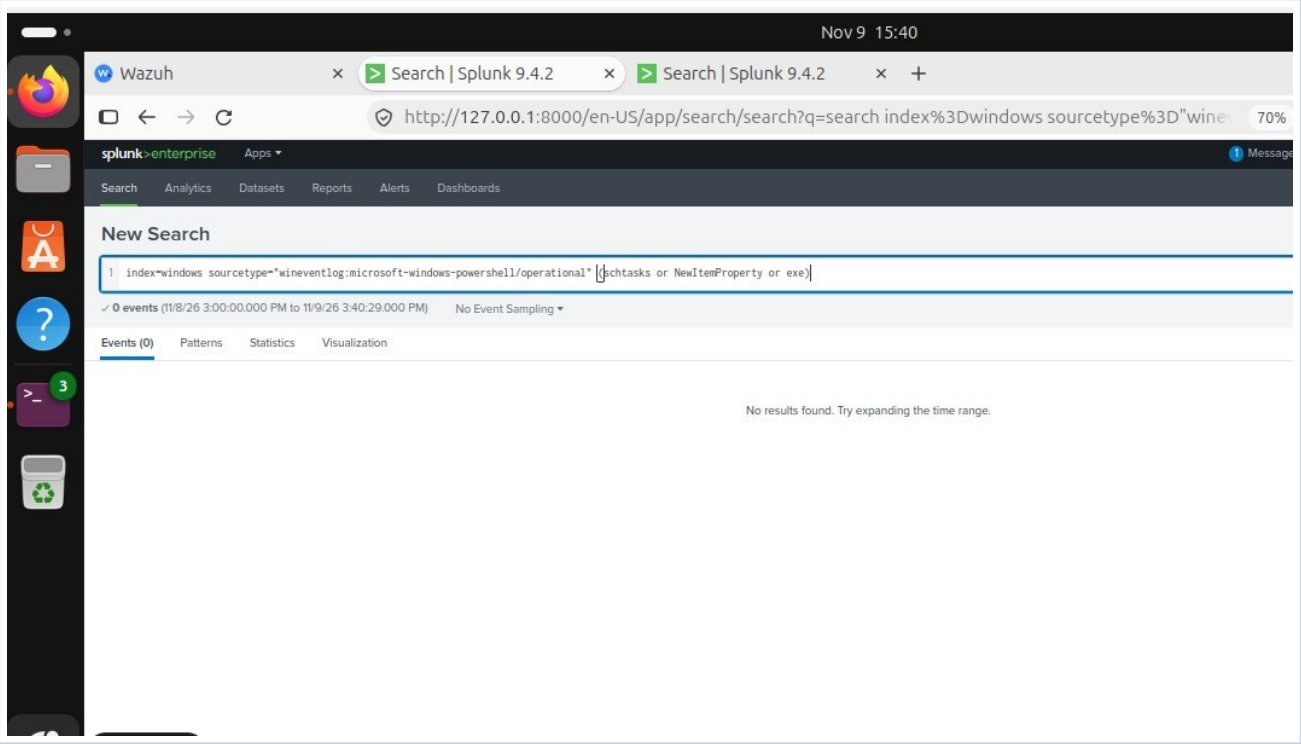


Figure 49. *The PowerShell-log search also returned 0 events. Because the query itself was wrong, this does not show that PowerShell logging is off. The log does exist (221 events, Section 4.3).* *(source file 1000499755.jpg)*

Chapter 11: Security Findings and Risk Analysis

11.1 Answers to the six analyst questions

Table 17. Analyst questions from the lab brief

Question	Answer (with evidence)
What happened?	Using the HOPELABS\Administrator account from an elevated PowerShell window (PID 6324), three persistence mechanisms were created on DESKTOP-T6T1N2V: scheduled task WindowsUpdateCheck (SYSTEM, at logon), service UpdSvc (LocalSystem, auto start) and Run key values Updater, Updater2 and Updater3.
How was it detected?	Security event 4698, System event 7045 and Sysmon event 13 were collected by Wazuh and Splunk. Wazuh rules 100620, 100621, 92302 and 100622 raised level 10 to 12 alerts tagged with MITRE IDs.
What evidence supports the finding?	Wazuh alerts; the raw events in Splunk with record numbers (4698: 124192 and 124411; 7045: 7599 and 7721); Sysmon process events showing the exact commands; matching timestamps within milliseconds (Section 6.11).
Which host and account were involved?	Host DESKTOP-T6T1N2V.HopeLabs.Local (agent 006, 192.168.56.109). Account HOPELABS\Administrator, SID ending -500, logon ID 0x180B06, High integrity.
What else should be checked?	The same task, service and value names on other hosts; where logon 0x180B06 came from (console or remote); other accounts seen on this host (HR-STAFF1, svc_backup, aigbo); other autostart locations; whether the files `p.txt` or any payload exist; and administrator activity at other times.
What would I do in a real SOC?	See the response plan below.

11.2 Real-SOC response plan (recommended, not performed beyond the lab clean-up)

1. **Triage:** SYSTEM-level persistence created by an administrator account is high severity.
2. **Contain:** isolate the computer from the network; disable or reset the account; terminate the PowerShell session.
3. **Scope:** search all computers for the same names and parent processes; check how the logon happened (source address, time).
4. **Preserve evidence:** export the event logs and take a memory image before cleaning.
5. **Eradicate:** remove the task, service and keys; verify, as in Section 6.12.
6. **Recover:** rotate credentials; rebuild the computer if its integrity cannot be proven.
7. **Improve:** keep the detections and fix the gaps in Chapter 12.

11.3 Findings

Table 18. Findings and risk rating

ID	Finding	Evidence	Impact	Confidence
F1	Scheduled task persistence was created and detected	Event 4698, rule 100620	High: runs as SYSTEM at every logon	Confirmed
F2	Service persistence was created and detected	Event 7045, rule 100621	High: LocalSystem, starts at boot	Confirmed
F3	Run key persistence was created and detected, but only after tuning	Rule 92302 level 6, then 12; rule 100622	Medium to High: starts at user logon	Confirmed
F4	Task creation was invisible until auditing was enabled	No 4698 at 09:53; event at 10:38	High: a blind spot hides attacker actions	Confirmed on this host
F5	All activity came from one PowerShell session and one logon ID	Splunk process chain, 28 events	Enables simple correlation	Confirmed
F6	Built-in rules can silently shadow custom rules	Alert trail 92302	Detection gaps if rules are not tested	Confirmed (mechanism); cause for 100623 unconfirmed
F7	PowerShell command logging state is unknown	Query error	Possible loss of command-level evidence	Unverified
F8	Host and manager clocks differ by about 2 seconds	Alert versus event times	Minor correlation error	Observed

Chapter 12: Mitigation and Remediation

12.1 What was actually implemented

Table 19. Implemented measures

Measure	Status	How it was verified
Enabled audit policy for task events (one host)	Implemented	Event 4698 appeared
Created rules 100620, 100621, 100622 and overwrote 92302	Implemented	Alerts fired (Sections 6.8 and 6.9)
Removed task, service and Run keys	Implemented	Verification commands, 13:19
Backed up the rules file	Implemented	local_rules.xml.bak created

12.2 Recommended improvements (not yet implemented)

Table 20. Recommended mitigations

Recommendation	Objective and how	Benefit	Possible impact	Verify by	Residual risk
Enable task auditing by Group Policy	Computer Configuration > Audit Policy > Object Access	Every host logs 4698	More log volume	Create a test task on a new host	Hosts outside the policy
Enable PowerShell Script Block Logging	GPO: Turn on PowerShell Script Block Logging	Exact commands recorded (4104)	More log volume	Run a test command and search 4104	Attackers using other shells
Synchronise clocks (NTP)	Point all VMs to one time source	Accurate timelines	None	Compare event and alert times	Drift if NTP fails
Restrict and monitor administrator accounts	Separate admin accounts, least privilege	Fewer ways to create SYSTEM persistence	Admin inconvenience	Review logon events	Stolen admin credentials
Allow-list known tasks, services and Run values	Maintain a baseline and alert on differences	Fewer false alarms	Upkeep effort	Test with a new unlisted item	Attackers copying approved names
Add detection for other autostart places	Rules for Startup folder, RunOnce, HKLM Run, WMI	Wider coverage	Rule tuning work	Repeat this lab for each location	Unknown techniques
Fix the threat-intel lists warnings	Create the missing CDB lists or remove rules 99901 to 99920	Clean config test	None	Run analysisd -t	None

Chapter 13: What I Learned

These reflections are based on what actually happened in this lab.

13.1 Concepts

- Persistence is about coming back. Scheduled tasks, services and Run keys are three of the simplest ways, and each one leaves a footprint in a different log.
- An alert is the end of a chain (generation, collection, forwarding, decoding, rules, alert, display). When an alert is missing I must check each link, as I did with the audit policy and the Sysmon collection setting.
- A log that is not switched on is a blind spot. The missing 4698 event taught me this more clearly than any explanation could.

13.2 Practical skills

- I learned to read what each command does: flags such as `/tn`, `/tr`, `/sc`, `/ru` now have meaning for me, and I learned that PowerShell aliases (`sc`) can change a command.
- I learned to query logs with `Get-WinEvent`, and to filter them with a pattern so that normal activity (Edge, OneDrive) does not drown the result.
- I learned how to search in Splunk, including that `OR` must be capitalised, and to build a combined timeline from several indexes.

13.3 Detection engineering

- Rules are parents and children, and the first match wins. My first Run key rule failed because I did not check what the ruleset already did with the same event.
- Always test with a real event, and keep a backup before editing. Testing before restarting (`wazuh-analysisd -t`) saved the system when my paste broke the XML.
- A good rule excludes known normal behaviour; otherwise analysts drown in false alarms.

13.4 Troubleshooting and honesty

- I learned to separate what I confirmed (rule 92302 shadowed my rule) from what I only suspected (why 100623 failed), and to say so.
- A tool that gives a surprising answer (logtest, the Splunk zero-result searches) should be questioned before it is believed.

13.5 Areas I still need to develop

- Writing detection rules independently from scratch, rather than from guided instructions.
- Understanding why 100623 failed (Wazuh rule-matching internals).
- Other persistence locations (WMI, Startup folder, HKLM) and how to detect them.
- Reading Windows event XML and Sysmon configuration files in depth.

Chapter 14: Practical Knowledge and Competencies

> **NOTE**
> The commands and rule syntax in this lab were performed with guided, step-by-step instructions during training. The table separates what was performed in the lab from what was only observed or still needs independent practice.

Table 21. Skills mapped to the work

Skill	Evidence in the lab	Level
Creating test persistence safely	Chapter 6.3	Performed (guided)
Windows log analysis (4698, 7045, Sysmon 1 and 13)	Chapter 6.5 and 6.6	Performed (guided)
Enabling audit policy	Chapter 6.2	Performed (guided)
Writing Wazuh rules	Chapter 7	Performed (guided), debugged during the lab
Validating and deploying rules (test, backup, restart)	Chapter 6.7 and 6.9	Performed
Alert triage in Wazuh dashboard	Chapter 6.10	Performed
Searching and correlating in Splunk	Chapter 6.11	Performed (guided)
MITRE ATT&CK mapping	Chapter 3 and Appendix	Performed
Troubleshooting and root-cause analysis	Chapter 10	Performed; one cause unresolved
Containment and verification	Chapter 6.12	Performed
Incident response planning	Chapter 11.2	Studied (not exercised)
Other persistence techniques (WMI, Startup folder)	Not in lab	Not tested

Suggested next exercises: (1) repeat this lab with the commands typed from memory; (2) add rules for the Startup folder and machine-wide Run key; (3) enable and verify PowerShell Script Block Logging; (4) investigate why rule 100623 failed; (5) create a Splunk alert for event 4698.

Chapter 15: Reproduction Guide

This is a stand-alone procedure. Follow the steps in order. Every command is explained in Chapter 6; the short notes here say what each step achieves.

15.1 Starting state

- Windows 10 VM with Sysmon running, a Wazuh agent connected to the manager, and Splunk receiving its logs.
- Ubuntu VM with Wazuh manager and Splunk. Snapshot both VMs first.
- You can open an Administrator PowerShell on Windows and a terminal with sudo on Ubuntu.

15.2 Windows 10: prepare logging (Administrator PowerShell)

```
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "Sysmon"
auditpol /set /subcategory:"Other Object Access Events" /success:enable /failure:enable
```

Expected: a line showing the Sysmon channel; the message 'The command was successfully executed.'

15.3 Ubuntu: install the rules

```
sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak
sudo nano /var/ossec/etc/rules/local_rules.xml
   (paste the rule group from Chapter 7.1 at the end, then Ctrl+O, Enter, Ctrl+X)
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager | grep Active
```

Expected: no ERROR or CRITICAL lines in the test; 'active (running)'. If you want to paste with a heredoc, do it at the prompt, never inside nano.

15.4 Windows 10: create the persistence

```
schtasks /create /tn "WindowsUpdateCheck" /tr "cmd.exe /c whoami > C:\Users\Public\p.txt" /sc
    onlogon /ru SYSTEM
sc.exe create UpdSvc binPath= "cmd.exe /c whoami" start= auto
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d
    "C:\Windows\System32\calc.exe" /f
New-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "Updater3"
    -Value "C:\Windows\System32\calc.exe" -Force
```

Expected: SUCCESS, [SC] CreateService SUCCESS, 'The operation completed successfully', and the property listing for Updater3.

15.5 Verify the evidence

```
# Windows
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4698} -MaxEvents 3 | fl
    TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='System';Id=7045} -MaxEvents 3 | fl
    TimeCreated,Message

# Ubuntu (wait 30 seconds)
sudo grep -E '"id":"(10062[0-2]|92302)"' /var/ossec/logs/alerts/alerts.json | tail -5
```

Expected: events 4698 and 7045 on Windows; alerts for rules 100620, 100621, 92302 (level 12) and 100622 on Ubuntu.

15.6 Investigate

- Wazuh dashboard: Threat Hunting, your agent, query `rule.id:(100620 or 100621 or 92302 or 100622)`; open each alert and record the user, image, logon ID and parent process ID.
- Splunk: run the searches from Section 6.11 and build the combined timeline.

15.7 Clean up and roll back

```
schtasks /delete /tn "WindowsUpdateCheck" /f
sc.exe delete UpdSvc
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /f
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater3 /f
Remove-Item C:\Users\Public\p.txt -ErrorAction SilentlyContinue

schtasks /query /tn "WindowsUpdateCheck"
sc.exe query UpdSvc
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Expected: 'cannot find the file', service error 1060, only legitimate values in the Run key. To roll back the Wazuh rules, copy the .bak file back, run the test, and restart the manager.

15.8 Troubleshooting quick list

- Service already exists (1073): delete, then create.
- No 4698 events: check the audit policy.
- No alerts: restart the manager; run commands *after* the rules are loaded.
- Run key alert at level 6 only: the built-in rule 92302 is matching; use the overwrite.
- Splunk shows 0 events: capital OR, correct index (main, windows, sysmon), wider time range.

Chapter 16: Limitations and Future Improvements

- **Scope:** one lab host, one account, harmless commands. Results do not show how rules behave across a real network.
- **Not tested:** other autostart locations, execution of the persistence, non-administrator attackers, real malware (Section 8.2).
- **False positives and negatives:** not measured. Rule 100622 excludes three programs; a hostile program could use those names. Rule 100621 depends on the service path containing cmd.exe, powershell, Users\Public or Temp; a malicious service in another folder would not match.
- **Missing evidence:** the first task creation (about 09:53) has no 4698 event; PowerShell command logging was not verified; the root cause of rule 100623's failure is unknown; exact Wazuh and Sysmon versions were not captured.
- **Evidence quality:** some screenshots are photographs of the screen and a few have small text; the underlying values were cross-checked against other captures.
- **Environment:** clocks differ by about 2 seconds, and Splunk's collection path is not documented.
- **Effect on conclusions:** the conclusions are limited to 'these three techniques, created this way, were detected on this host after the described tuning'.

Future improvements: add detections for further persistence locations, enable Script Block Logging, synchronise time, build a Splunk alert for event 4698, add allow-lists, and repeat the lab with a non-administrator account.

Chapter 17: Conclusion

In this lab I created three harmless persistence mechanisms on a Windows 10 computer, detected each one with Windows logs and Sysmon, collected the evidence in Wazuh and Splunk, and traced all activity to a single PowerShell session under the Administrator account (logon 0x180B06). I wrote and deployed four Wazuh rules and verified that they fired at level 10 to 12 with MITRE ATT&CK tags (T1053.005, T1543.003, T1547.001). I then removed the items and proved the removal.

The strongest findings are that evidence can be missing if auditing is off, that built-in rules can hide custom rules, and that raw logs in Splunk add the 'how' that alerts alone cannot show. The limits are equally important: one host, harmless commands, an unresolved cause for one failed rule, and an unverified PowerShell command-log state.

Next steps: close those gaps, widen the detections to other autostart locations, and practise the same lab without guided instructions.

Appendix A: Final Wazuh Rules

```
<group name="persistence,windows,">

  <rule id="100620" level="10">
    <if_sid>60228</if_sid>
    <description>Scheduled task created: $(win.eventdata.taskName)</description>
    <mitre><id>T1053.005</id></mitre>
  </rule>

  <rule id="100621" level="12">
    <if_sid>61138</if_sid>
    <field name="win.eventdata.imagePath"
        type="pcre2">(?i)cmd\.exe|powershell|Users.Public|.Temp.</field>
    <description>Suspicious service installed: $(win.eventdata.serviceName)</description>
    <mitre><id>T1543.003</id></mitre>
  </rule>

  <rule id="92302" level="12" overwrite="yes">
    <if_sid>92300</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)reg\.exe</field>
    <options>no_full_log</options>
    <description>Run key persistence via reg.exe (escalated from level 6)</description>
    <mitre><id>T1547.001</id></mitre>
  </rule>

  <rule id="100622" level="12">
    <if_sid>92300</if_sid>
    <field name="win.eventdata.image" type="pcre2"
        negate="yes">(?i)reg\.exe|msedge\.exe|OneDrive\.exe|CompatTelRunner</field>
    <description>Run key persistence by $(win.eventdata.image)</description>
    <mitre><id>T1547.001</id></mitre>
  </rule>

</group>
```

Appendix B: Command Reference by Machine

B.1 Windows 10 (Administrator PowerShell)

```
auditpol /set /subcategory:"Other Object Access Events" /success:enable /failure:enable
schtasks /create /tn "WindowsUpdateCheck" /tr "cmd.exe /c whoami > C:\Users\Public\p.txt" /sc
    onlogon /ru SYSTEM
schtasks /query /tn "WindowsUpdateCheck" /v /fo list
schtasks /delete /tn "WindowsUpdateCheck" /f
sc.exe create UpdSvc binPath= "cmd.exe /c whoami" start= auto
sc.exe qc UpdSvc
sc.exe query UpdSvc
sc.exe delete UpdSvc
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d
    "C:\Windows\System32\calc.exe" /f
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /f
New-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "Updater3"
    -Value "C:\Windows\System32\calc.exe" -Force
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4698} -MaxEvents 3 | fl
    TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='System';Id=7045} -MaxEvents 3 | fl
    TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=1} | ?
    {$_.Message -match 'schtasks|sc.exe|reg.exe'} | select -First 6 | fl TimeCreated,Message
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "Sysmon"
Remove-Item C:\Users\Public\p.txt -ErrorAction SilentlyContinue
```

B.2 Ubuntu (sudo)

```
sudo nano /var/ossec/etc/rules/local_rules.xml
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager | grep Active
sudo grep -E '"id":"10062[0-2]"' /var/ossec/logs/alerts/alerts.json | tail -5
sudo grep -rn -B2 -A14 'id="92302"' /var/ossec/ruleset/rules/
sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak
sudo sed -i '/<group name="persistence,windows,">/,/<\/group>/d'
    /var/ossec/etc/rules/local_rules.xml
sudo sed -i '/sudo tee -a/d;/^EOF$/d' /var/ossec/etc/rules/local_rules.xml
sudo /var/ossec/bin/wazuh-logtest -v
```

B.3 Splunk searches

```
index=* host=DESKTOP-T6T1N2V | stats count by index, sourcetype
index=* EventCode=4698 WindowsUpdateCheck
index=windows EventCode=7045 UpdSvc
index=sysmon EventCode=13 Updater*
index=sysmon EventCode=1 (schtasks.exe OR sc.exe OR reg.exe) powershell.exe
index=sysmon EventCode=1 ParentProcessId=6324 | sort -_time | table _time Image CommandLine
    LogonId ParentProcessId
(index="main" EventCode=4698 WindowsUpdateCheck) OR (index=windows EventCode=7045 UpdSvc) OR
    (index=sysmon EventCode=13 Updater*) OR (index=sysmon EventCode=1 ParentProcessId=6324
    CommandLine="*create*") | sort -_time | table _time index EventCode Image CommandLine
    TargetObject
```

Appendix C: Consolidated Timeline (WAT)

Table 22. Timeline of the lab (Splunk, Wazuh and Windows evidence)

Time	Event	Source
09:53	Run key Updater written by reg.exe (PID 10060); Wazuh alert 92302 level 6; task created (no 4698)	Wazuh, local Sysmon
10:11:45	Service UpdSvc installed (7045, record 7599)	Splunk, local log
10:38:42	Task recreated after enabling auditing: 4698 (record 124192); Sysmon: schtasks launched by powershell.exe 6324	Splunk, Sysmon
11:42 to 11:45	Repeat attempts: create commands failed because items existed; reg add Updater succeeded	Splunk (Sysmon 1)
11:50:07	sc delete then create UpdSvc; 7045 (record 7721); Wazuh 100621 at 11:50:06	Splunk, Wazuh
11:53:18	schtasks delete then create; 4698 (record 124411); Wazuh 100620 at 11:53:16	Splunk, Wazuh
12:26 to 12:41	Run key tests with reg.exe and PowerShell (Updater2); 92302 level 6	Splunk, Wazuh
13:12:22	Wazuh manager restarted with the final rules	Ubuntu
13:14:56 to 13:14:57	reg.exe write: 92302 level 12; PowerShell write Updater3: 100622 level 12	Splunk, Wazuh
13:18:56 to 13:19:09	Clean-up and verification commands from PowerShell 6324	Splunk, Windows

Appendix D: Glossary

Table 23. Plain-language glossary

Term	Meaning
Alert	A message from a security tool saying a rule matched
Audit policy	A Windows setting that decides which actions are written to the Security log
Decoder	The part of Wazuh that splits a raw log line into named fields
Event ID	A number identifying the type of event in a Windows log
Image	In Sysmon, the program file that performed an action
Integrity level	How much trust Windows gives a running program (High = administrator)
Logon ID	A number identifying one login session; it ties actions to the same session
Parent process	The program that started another program
Persistence	Methods that let an attacker keep access after restarts or logouts
Registry	The Windows database of settings
Rule	A condition that, when matched by an event, creates an alert
Sourcetype / index	Splunk labels for the kind of log and the storage bucket it lives in
Sysmon	A Microsoft tool that records detailed process and registry activity
Telemetry	Data that computers report about themselves
Wildcard	A symbol (such as * in Splunk or . in a pattern) that stands for any character(s)

Appendix E: References

- MITRE ATT&CK T1053.005 Scheduled Task: https://attack.mitre.org/techniques/T1053/005/
- MITRE ATT&CK T1543.003 Windows Service: https://attack.mitre.org/techniques/T1543/003/
- MITRE ATT&CK T1547.001 Registry Run Keys / Startup Folder: https://attack.mitre.org/techniques/T1547/001/
- Wazuh documentation, Ruleset and custom rules: https://documentation.wazuh.com/
- Microsoft Sysinternals Sysmon: https://learn.microsoft.com/sysinternals/downloads/sysmon
- Microsoft Learn, Windows security auditing events (4698 and others): https://learn.microsoft.com/
- Splunk documentation, Search Reference: https://docs.splunk.com/

Appendix F: Evidence Handling and Redaction Log

- Original screenshots are kept unchanged. Presentation copies were produced by resizing only (maximum width 1600 pixels; no enlargement).
- No annotations, highlights or edits were added to any screenshot.
- No redactions were applied. Visible private-lab values: computer and domain names, IP addresses 192.168.56.101 and 192.168.56.109, account names and SIDs. Replace them before public sharing.
- Screenshots that are photographs of the screen (logtest, XML error, final terminal confirmation) were not altered or straightened so that they remain authentic.
- Many other lab screenshots exist that were not included because they duplicate information already shown.